INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

ESCOM

Trabajo Terminal

2026-B009

“Manual de Instalación de la Plataforma Web para la Denuncia Segura y Prevención de Riesgos Digitales”

Presentan

Edgar Jair Flores Espinosa

Jose Bryan Omar Jimenez Velazquez

Leilani Shareni Santos Rivera

Directores

Dr. Jessie Paulina Guzman Flores

M. en C. Ulises Velez Saldaña

29 de Diciembre de 2026

INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
SUBDIRECCIÓN ACADÉMICA

No. de TT: 2026-B009

Fecha de presentación de TT-II: 05 de junio de 2026

Documento Técnico

"Manual de Instalación de la iPlataforma Web para la Denuncia Segura y Prevención de Riesgos Digitales"

Presentan:

Edgar Jair Flores Espinosa, Jose Bryan Omar Jimenez Velazquez, Leilani Shareni Santos Rivera

Directores:

Dr. Jessie Paulina Guzman Flores

M. en C. Ulises Velez Saldaña

MANUAL DE INSTALACION

[1] josebryanomar2004@gmail.com

Índice

1. Introducción y Arquitectura Distribuida **1**

1.1. Resumen de la solución . . . 1

1.1.1. Descripción general de la plataforma (microservicios Spring Boot, frontends Angular y modelo de IA) . . . 1

1.1.2. Objetivo del despliegue en 3 VPS . . . 2

1.2. Diagrama de arquitectura en 3 capas . . . 2

1.2.1. Relación entre VPS Frontend (router-nginx y frontends), VPS Backend (microservicios y modelo de IA) y VPS Base de Datos (PostgreSQL) . . . 2

1.2.2. Flujo de una petición del usuario . . . 3

1.3. Matriz de conectividad y puertos . . . 4

1.3.1. Direccionamiento IP público por VPS (169.58.62.111, 156.67.26.73, 169.58.62.99) . . . 4

1.3.2. Mapeo de puertos del host y de los contenedores (80/443 router-nginx; 8090–8095 frontends; 8082–8094 microservicios; 5432 defensoria-db) 4

1.3.3. Rutas públicas de nginx hacia cada servicio . . . 5

2. Preparación de Nodos y Entorno de Ejecución (Común a las 3 VPS) **6**

2.1. Actualización base y acceso a los servidores . . . 6

2.1.1. Acceso por SSH como root con Termius . . . 6

2.1.2. Actualización del sistema y zona horaria (America/Mexico_City) . . . 8

2.2. Instalación y aprovisionamiento de Podman . . . 11

2.2.1. Instalación de Podman y utilerías (podman, curl, iptables, python3, certbot) . . . 11

2.2.2. Imágenes base con nombre completo (docker.io/library/...) . . . 13

2.2.3. Contenedores ejecutados como root con podman run . . . 14

2.2.4. Arranque automático de contenedores (podman-restart.service) . . . 14

2.3. Estructura de directorios y gestión de permisos . . . 15

2.3.1. Árbol de carpetas en /apps (aplicaciones/defensoria, data/postgresql, respaldos, utiles/respaldos) . . . 15

2.3.2. Políticas de propiedad y permisos (root:root, 755 carpetas, 750 scripts, 750/640 config-files, 700 respaldos) . . . 17

3. Seguridad de Red y Configuración de Firewalls (ufw / iptables) **19**

3.1. Política de firewall con Podman . . . 19

3.1.1. ufw: política por defecto y reglas de entrada (IN) y de reenvío (FWD) . . . 19

3.1.2. Capa adicional: regla en la tabla raw de iptables . . . 20

3.2. Reglas perimetrales por nodo . . . 21

3.2.1. VPS Base de Datos: puerto 5432 exclusivamente desde la IP del VPS Backend . . . 21

3.2.2. VPS Backend: puertos de los microservicios exclusivamente desde la IP del VPS Frontend . . . 21
3.2.3. VPS Frontend: tráfico público 80 (HTTP) y 443 (HTTPS) . . . 23
3.2.4. Rutas internas bloqueadas en nginx (403) . . . 23
3.2.5. Persistencia de las reglas tras reiniciar . . . 24
3.3. Diagnóstico de conectividad . . . 26
3.3.1. Verificación de puertos y sockets (ss -tulpn, ufw status, iptables -t raw) 26
3.3.2. Pruebas de conexión entre nodos y desde Internet . . . 27

4. Compilación Local, Empaquetado y Construcción de Contenedores 28
4.1. Código fuente del proyecto . . . 28
4.1.1. Estructura de la carpeta Produccion . . . 29
4.1.2. Revisión de los archivos de configuración de producción (config-files) . 30
4.2. Compilación local . . . 32
4.2.1. Compilación de los 11 JAR con Maven . . . 33
4.2.2. Compilación de los 5 frontends Angular (ng build y baseHref) . . . 35
4.3. Construcción de imágenes con Podman . . . 37
4.3.1. Dockerfile de los microservicios (eclipse-temurin:21-jre-jammy) . . . 37
4.3.2. Dockerfile de admin-service (cliente pg_dump 16) . . . 38
4.3.3. Dockerfile de los frontends (nginx:alpine) . . . 38
4.3.4. Construcción de las imágenes en el servidor con los scripts podman-compose . . . 39
4.4. Transferencia de artefactos a los VPS . . . 39
4.4.1. Copia segura mediante scp hacia cada servidor de destino . . . 39

5. Despliegue del Nodo de Base de Datos (VPS 169.58.62.99) 43
5.1. Imagen de PostgreSQL . . . 43
5.1.1. Descarga de la imagen oficial postgres:16-alpine . . . 43
5.2. Volúmenes persistentes y variables . . . 44
5.2.1. Volumen de datos en /apps/data/postgresql/defensoria_db . . . 44
5.2.2. Script de inicialización 00-init.sql y scripts semilla . . . 44
5.3. Creación y ejecución del contenedor de BD . . . 45
5.3.1. El script podman-compose-bd.sh . . . 45
5.3.2. Creación de defensoria-db (puerto 5432, límite de memoria) . . . 46
5.3.3. Verificación de la base defensoria_db . . . 47
5.4. Arranque automático y respaldos . . . 49
5.4.1. Política --restart always y podman-restart.service . . . 49
5.4.2. Servicio systemd del firewall (firewall-boot) . . . 49

6. Despliegue del Nodo de Backend (VPS 156.67.26.73) 50

6.1. Construcción de las imágenes del Backend 50
6.1.1. Ubicación de los JAR en artifact/ 50
6.1.2. Construcción de las imágenes con podman-compose.sh 51
6.2. Configuración de producción (config-files) 52
6.2.1. Montaje de la configuración en /app/config 52
6.2.2. Conexión a la base de datos (169.58.62.99:5432) 52
6.2.3. Secreto JWT compartido 53
6.2.4. Correo SMTP y URLs entre microservicios 53
6.3. Creación y ejecución de los contenedores del Backend 54
6.3.1. Límites de memoria y heap de la JVM 54
6.3.2. Microservicios de la plataforma 54
6.3.3. Primer Contacto y Subdefensoría 55
6.3.4. Modelo de IA (antecedentes-service y su corpus) 55
6.4. Ciclo de vida de los contenedores 58
6.4.1. Reinicio automático (--restart always) 58
6.4.2. Redespliegue de un servicio (up-container y delete-container) 59
6.5. Carga de datos iniciales 60
6.5.1. Datos semilla (dependencias, chatbot y personal) 60

7. Despliegue del Nodo de Frontend, Nginx y Certificados (VPS 169.58.62.111) 62
7.1. Nginx en contenedor (router-nginx) 62
7.1.1. Contenedor router-nginx con --network host 62
7.1.2. Estructura de /apps/aplicaciones/defensoria/router (config y certbot-webroot) 63
7.2. Despliegue de los frontends 63
7.2.1. Ubicación del build web en dist/browser y asignación de permisos (755/644) 63
7.2.2. Contenedores de los 5 frontends (nginx:alpine) 64
7.3. Emisión de Certificados SSL/TLS con Certbot 66
7.3.1. Instalación de Certbot (método webroot) 66
7.3.2. Dominio en No-IP (defensoria-escom.ddns.net) 66
7.3.3. Reto HTTP-01 y emisión inicial del certificado Let's Encrypt 67
7.3.4. Renovación automática y recarga de nginx (certbot renew --dry-run) . . 69
7.4. Configuración de producción de Nginx (defensoria.conf) 70
7.4.1. Bloque de redirección forzada de HTTP (80) a HTTPS (443) 70
7.4.2. Bloque 443 y certificados TLS 70
7.4.3. Rutas hacia los frontends 71
7.4.4. Reverse proxy: location /api/ hacia el VPS Backend 71
7.4.5. Rutas bloqueadas, tamaño máximo de carga y tiempos de espera 71

7.4.6. Compresión gzip y caché de estáticos 72

Índice de figuras

1.1. Arquitectura de la plataforma en 3 capas 3
2.1. Configuración de los tres hosts en Termius (contraseña oculta) 7
2.2. Sesión SSH como root en las tres VPS dentro de un Workspace de Termius . . 8
2.3. Actualización del sistema en el VPS Frontend 9
2.4. Actualización del sistema en el VPS Backend 9
2.5. Actualización del sistema en el VPS Base de Datos 10
2.6. Zona horaria America/Mexico_City en el VPS Frontend 10
2.7. Zona horaria America/Mexico_City en el VPS Backend 11
2.8. Zona horaria America/Mexico_City en el VPS Base de Datos 11
2.9. Instalación de Podman y versión en el VPS Frontend 12
2.10. Instalación de Podman y versión en el VPS Backend 12
2.11. Instalación de Podman y versión en el VPS Base de Datos 13
2.12. Imagen base descargada en cada VPS (salida de podman images) 14
2.13. Servicio podman-restart habilitado 15
2.14. Dueño y permisos de las carpetas en el VPS Frontend 18
2.15. Dueño y permisos de las carpetas en el VPS Backend 18
2.16. Dueño y permisos de las carpetas en el VPS Base de Datos 19
3.1. Reglas de ufw y regla raw del VPS Base de Datos 25
3.2. Reglas de ufw y regla raw del VPS Backend 25
3.3. Estado del firewall del VPS Frontend 26
3.4. Pruebas de conexión entre nodos y desde Internet 27
4.1. Clonado de la rama Produccion y hash del commit descargado 28
4.2. Estructura de la carpeta Produccion 30
4.3. Contenido de un config-file de ejemplo (quejas-service.yml, con contraseña y secreto ocultos) 32
4.4. Versiones de Java, Maven, Node.js y npm en el equipo de desarrollo 33
4.5. Compilación de los microservicios con Maven 34
4.6. Los 11 JAR generados en las carpetas target 35
4.7. Compilación de los frontends con ng build 36
4.8. Carpetas dist generadas y base href de cada frontend 37
4.9. Contenido de los Dockerfile de microservicios y frontends 39
4.10. Transferencia de archivos al VPS Base de Datos 40
4.11. Transferencia de archivos al VPS Backend 41
4.12. Transferencia de archivos al VPS Frontend 42
5.1. Imagen postgres:16-alpine disponible y versión de PostgreSQL 43
5.2. Scripts de inicialización y semilla en el VPS Base de Datos 45

5.3. Creación de defensoria-db y estado del contenedor 47
5.4. Base defensoria_db, extensión uuid-ossp y zona horaria 48
5.5. Puerto 5432 en escucha y protegido por la regla de firewall 48
5.6. Política de reinicio de defensoria-db y servicio podman-restart 49
5.7. Servicios ufw y defensoria-bd-firewall habilitados y regla raw cargada 50
6.1. Los 11 JAR en la carpeta artifact del VPS Backend 51
6.2. Archivos de configuración en el VPS Backend con sus permisos 52
6.3. Construcción y arranque de los microservicios con podman-compose.sh up . . 55
6.4. Modelo de IA levantado, health check y búsqueda de prueba 56
6.5. Los 11 contenedores del Backend en estado Up 57
6.6. Respuesta de cada microservicio en su puerto 57
6.7. Imágenes defensoria-* construidas en el VPS Backend 58
6.8. Política de reinicio de los contenedores del Backend 58
6.9. Redespliegue de un microservicio con up-container 59
6.10. Tablas creadas por los microservicios en defensoria_db 60
6.11. Resultado de la carga de datos semilla 61
6.12. Número de registros cargados en cada tabla 62
7.1. Carpeta router con defensoria.conf y certbot-webroot 63
7.2. Archivos compilados de los cinco frontends en dist/browser 64
7.3. Construcción y arranque de los cinco frontends 65
7.4. Contenedores de los frontends en estado Up y respondiendo 200 66
7.5. Host defensoria-escom.ddns.net apuntando a 169.58.62.111 en el panel de No-IP 67
7.6. Emisión del certificado de Let's Encrypt 68
7.7. Contenedor router-nginx en ejecución y configuración validada 69
7.8. Prueba de renovación del certificado y fecha de vencimiento 69
7.9. Redirección HTTP a HTTPS, respuesta 200 por HTTPS y ruta interna bloqueada 73

Índice de tablas

1.1. Microservicios de la plataforma . . . 1
1.2. Frontends de la plataforma . . . 2
1.3. Servidores del despliegue . . . 2
1.4. Direccionamiento y accesos permitidos por VPS . . . 4
1.5. Mapeo de puertos del host y del contenedor . . . 4
1.6. Rutas públicas de router-nginx . . . 5
2.1. Hosts registrados en Termius . . . 6
2.2. Imágenes base de la plataforma . . . 13
2.3. Carpetas principales . . . 15
2.4. Política de permisos . . . 17
3.1. Tipos de reglas de ufw . . . 20
3.2. Partes de la regla raw . . . 20
3.3. Resumen de la protección por VPS . . . 21
3.4. Reglas de ufw del VPS Base de Datos . . . 21
3.5. Reglas de ufw del VPS Backend . . . 22
3.6. Puertos del VPS Backend . . . 23
3.7. Servicios de systemd del firewall . . . 24
3.8. Puertos que deben aparecer en escucha con la plataforma instalada . . . 26
3.9. Matriz de pruebas de conectividad (con la plataforma levantada) . . . 27
4.1. Contenido de la carpeta Produccion . . . 29
4.2. Secciones comunes de los config-files . . . 30
4.3. Herramientas requeridas en el equipo de desarrollo . . . 32
4.4. JAR generados . . . 34
4.5. Salida de compilación de los frontends . . . 36
4.6. Imágenes que construye cada script . . . 39
5.1. Scripts SQL de la base de datos . . . 44
5.2. Comandos de podman-compose-bd.sh usados en la plataforma . . . 45
5.3. Parámetros del contenedor defensoria-db . . . 46
6.1. Comandos de podman-compose.sh . . . 51
6.2. Dependencias entre microservicios . . . 53
6.3. Recursos asignados a cada contenedor del Backend . . . 54
6.4. Operaciones del ciclo de vida . . . 59
6.5. Datos semilla . . . 60
7.1. Carpetas de los frontends en el VPS Frontend . . . 64
7.2. Contenedores de los frontends . . . 65

1. Introducción y Arquitectura Distribuida

Este capítulo presenta la plataforma que se instala con este manual, la forma en que sus componentes se reparten entre tres servidores virtuales (VPS) y la manera en que se comunican entre sí. Sirve como mapa de referencia para el resto del documento: cada capítulo posterior instala una de las piezas que aquí se describen.

1.1. Resumen de la solución

1.1.1. Descripción general de la plataforma (microservicios Spring Boot, frontends Angular y modelo de IA)

La Plataforma de Quejas de la Defensoría de los Derechos Politécnicos (DDP) es un sistema web que permite a la comunidad del Instituto Politécnico Nacional presentar quejas y darles seguimiento, y al personal de la Defensoría recibirlas, revisarlas, turnarlas y atenderlas. Está construida con una arquitectura de microservicios y se compone de tres tipos de piezas:

- **Microservicios (backend).** Diez servicios independientes desarrollados en Java 21 con Spring Boot. Cada uno resuelve una parte del negocio, expone una API REST y se ejecuta en su propio contenedor de Podman.
- **Frontends (interfaz web).** Cinco aplicaciones Angular, una por cada tipo de usuario. Se compilan a archivos estáticos (HTML, CSS y JavaScript) y cada una se sirve desde su propio contenedor de nginx.
- **Modelo de inteligencia artificial.** El servicio `antecedentes-service`, también en Java, busca quejas parecidas en el histórico mediante TF-IDF y similitud coseno, y genera un resumen extractivo de cada queja con TextRank. No requiere GPU ni servicios externos.

Todos los servicios guardan su información en una sola base de datos PostgreSQL 16 (`defensoria_db`).

Tabla 1.1: Microservicios de la plataforma

Servicio	Función
`auth-service`	Inicio de sesión, emisión de tokens JWT, activación de cuentas y recuperación de contraseña
`quejas-service`	Registro y consulta de quejas, folios, credenciales y evidencias
`notificaciones-service`	Avisos dentro de la plataforma y envío de correos
`catalogo-service`	Catálogo de dependencias (unidades académicas) del IPN
`admin-service`	Panel de la Defensora: indicadores, personal, configuración y respaldos
`revision-service`	Validación por la recepcionista y turnado de quejas
`chatbot-service`	Preguntas frecuentes del asistente del portal
`denunciado-service`	Registro de la respuesta y evidencias de la persona denunciada
`primer-contacto-service`	Módulo de Primer Contacto (atención inicial y citas)
`subdefensoria-service`	Módulo de la Subdefensoría (atención de quejas turnadas)
`antecedentes-service`	Modelo de IA: búsqueda de antecedentes y resumen de quejas

Tabla 1.2: Frontends de la plataforma

Frontend	Ruta pública	Usuarios
Portal (`defensoria-web`)	`/`	Quejosos y personas denunciadas
Revisión (`revision-web`)	`/revision/`	Login unificado del personal y recepcionistas
Administración (`admin-web`)	`/admin/`	Defensora y administración del sistema
Primer Contacto (`primer-contacto-web`)	`/primer-contacto/`	Analistas de Primer Contacto
Subdefensoría (`subdefensoria-web`)	`/subdefensoria/`	Abogados de la Subdefensoría

1.1.2. Objetivo del despliegue en 3 VPS

El objetivo del despliegue es dejar la plataforma completa en producción y accesible en `https://defensoria-escom.ddns.net`, separando sus componentes en tres servidores virtuales, uno por capa:

- **Seguridad.** Solo el servidor frontend queda expuesto a Internet. El backend acepta conexiones únicamente del frontend, y la base de datos únicamente del backend, de modo que los datos de las quejas nunca quedan directamente al alcance de un atacante externo.
- **Aislamiento de recursos.** La base de datos, los microservicios y los portales no compiten por la misma memoria y CPU; si un servicio consume más de lo previsto, no afecta a las otras capas.
- **Mantenimiento independiente.** Cada capa se puede actualizar, reiniciar o respaldar sin detener a las demás.
- **Reproducibilidad.** Todos los componentes se ejecutan en contenedores Podman creados con scripts, por lo que la instalación se puede repetir de la misma forma en servidores nuevos.

Tabla 1.3: Servidores del despliegue

Capa	Hostname	IP pública	Contenido
Frontend	vmi3633026	169.58.62.111	router-nginx y 5 frontends
Backend	vmi3633021	156.67.26.73	10 microservicios y modelo de IA
Base de datos	vmi3633025	169.58.62.99	PostgreSQL 16 (`defensoria-db`)

1.2. Diagrama de arquitectura en 3 capas

1.2.1. Relación entre VPS Frontend (router-nginx y frontends), VPS Backend (microservicios y modelo de IA) y VPS Base de Datos (PostgreSQL)

La plataforma sigue una arquitectura de tres capas en la que cada capa vive en su propia VPS y solo se comunica con la capa contigua (Figura 1.1).

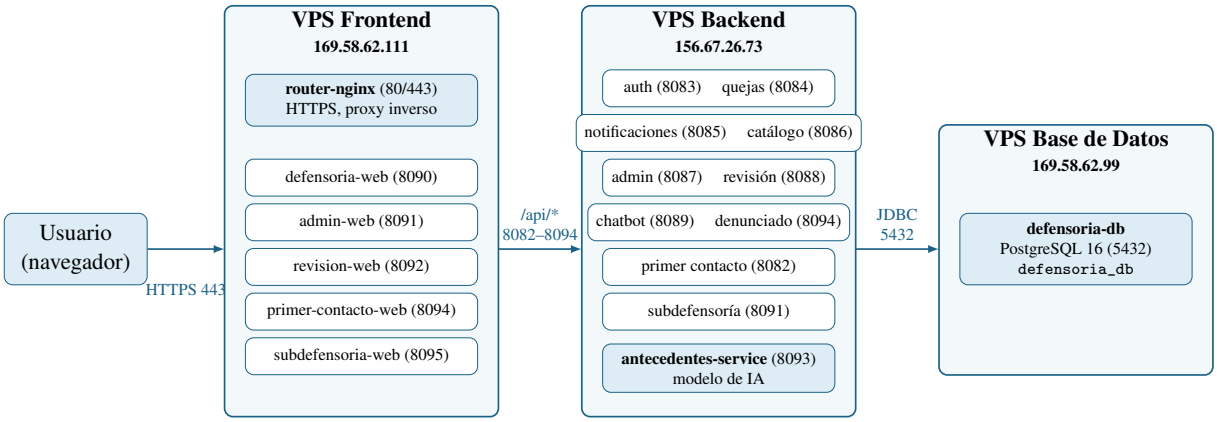


Figura 1.1: Arquitectura de la plataforma en 3 capas

- **VPS Frontend.** Es la única capa visible desde Internet. El contenedor `router-nginx` recibe todo el tráfico en los puertos 80 y 443, termina la conexión cifrada (HTTPS) con el certificado de Let's Encrypt y decide a dónde enviar cada petición: las páginas se las pide a uno de los cinco contenedores de frontend que corren en el mismo servidor, y las llamadas que empiezan con `/api/` las reenvía al backend.
- **VPS Backend.** Ejecuta los diez microservicios y el modelo de IA, cada uno en su contenedor. Recibe peticiones solo del frontend. Los microservicios también se llaman entre sí; por ejemplo, `quejas-service` avisa a `notificaciones-service` cuando se registra una queja, y `primer-contacto-service` consulta a `antecedentes-service` para buscar antecedentes de una queja.
- **VPS Base de Datos.** Ejecuta PostgreSQL 16 en el contenedor `defensoria-db`, con la base `defensoria_db`. Solo acepta conexiones del backend. Las tablas las crea cada microservicio al arrancar (Hibernate).

1.2.2. Flujo de una petición del usuario

Como ejemplo, se sigue el recorrido completo cuando una persona registra una queja desde el portal:

1. El usuario abre `https://defensoria-escom.ddns.net`. El dominio de No-IP resuelve a la IP del VPS Frontend (169.58.62.111).
2. `router-nginx` recibe la petición en el puerto 443, establece la conexión HTTPS y, como la ruta es `/`, la envía a `defensoria-web` (`127.0.0.1:8090`), que devuelve la aplicación Angular del portal.
3. El usuario llena el formulario y lo envía. El navegador hace una petición `POST` a `/api/quejoso/...` con los datos de la queja, su credencial y sus evidencias.
4. `router-nginx` identifica la ruta `/api/quejoso/` y reenvía la petición al VPS Backend, a `156.67.26.73:8084` (`quejas-service`). El firewall del backend la acepta porque proviene de la IP del frontend.
5. `quejas-service` valida la información, genera el folio y guarda la queja y sus archivos en `defensoria_db` a través de JDBC hacia `169.58.62.99:5432`. El firewall de la base la acepta porque proviene de la IP del backend.

6. `quejas-service` llama a `notificaciones-service` (puerto 8085) para registrar el aviso correspondiente.

7. La respuesta con el folio regresa por el mismo camino: backend → `router-nginx` → navegador, y el portal muestra el folio al usuario.

Las peticiones del personal siguen el mismo patrón; la diferencia es que incluyen el token JWT emitido por `auth-service`, que cada microservicio valida con el secreto compartido antes de responder.

1.3. Matriz de conectividad y puertos

1.3.1. Direccionamiento IP público por VPS (169.58.62.111, 156.67.26.73, 169.58.62.99)

Cada VPS cuenta con una sola dirección IP pública. Las tres se comunican entre sí por esas direcciones; no existe una red privada entre ellas, por lo que el aislamiento de cada capa se logra con reglas de firewall que solo aceptan tráfico de la IP de la capa anterior (capítulo 3).

Tabla 1.4: Direccionamiento y accesos permitidos por VPS

VPS	IP pública	Acepta conexiones de	Se conecta a
Frontend	169.58.62.111	Internet (80 y 443)	Backend (8082–8094)
Backend	156.67.26.73	Frontend (8082–8094) y el propio backend	Base de datos (5432); servidor SMTP para correos
Base de datos	169.58.62.99	Backend (5432)	–

El acceso de administración por SSH (puerto 22) está disponible en las tres VPS. El dominio público `defensoria-escom.ddns.net` apunta únicamente al VPS Frontend.

1.3.2. Mapeo de puertos del host y de los contenedores (80/443 router-nginx; 8090–8095 frontends; 8082–8094 microservicios; 5432 defensoria-db)

Cada contenedor escucha en un puerto interno y Podman lo publica en un puerto del servidor (host). Todos los microservicios escuchan internamente en el 8080, y es el puerto del host el que los distingue. `router-nginx` no publica puertos: usa la red del host (`--network host`) para escuchar directamente en el 80 y el 443.

Tabla 1.5: Mapeo de puertos del host y del contenedor

VPS	Contenedor	Puerto host	Puerto contenedor	Imagen base
Frontend	router-nginx	80, 443	red del host	nginx:alpine
Frontend	defensoria-web	8090	8090	nginx:alpine
Frontend	admin-web	8091	80	nginx:alpine
Frontend	revision-web	8092	80	nginx:alpine
Frontend	primer-contacto-web	8094	80	nginx:alpine
Frontend	subdefensoria-web	8095	80	nginx:alpine
Backend	primer-contacto-service	8082	8080	eclipse-temurin:21
Backend	auth-service	8083	8080	eclipse-temurin:21

VPS	Contenedor	Puerto host	Puerto contenedor	Imagen base
Backend	quejas-service	8084	8080	eclipse-temurin:21
Backend	notificaciones-service	8085	8080	eclipse-temurin:21
Backend	catalogo-service	8086	8080	eclipse-temurin:21
Backend	admin-service	8087	8080	eclipse-temurin:21
Backend	revision-service	8088	8080	eclipse-temurin:21
Backend	chatbot-service	8089	8080	eclipse-temurin:21
Backend	subdefensoria-service	8091	8080	eclipse-temurin:21
Backend	antecedentes-service	8093	8080	eclipse-temurin:21
Backend	denunciado-service	8094	8080	eclipse-temurin:21
Base de datos	defensoria-db	5432	5432	postgres:16-alpine

Nota: Los puertos 8091 y 8094 aparecen tanto en el frontend como en el backend. No hay conflicto porque están en servidores distintos: en el frontend corresponden a `admin-web` y `primer-contacto-web`, y en el backend a `subdefensoria-service` y `denunciado-service`.

1.3.3. Rutas públicas de nginx hacia cada servicio

El usuario nunca usa los puertos directamente: todo entra por `https://defensoria-escom.ddns.net` y `router-nginx` decide el destino según la ruta de la URL. Las rutas de los frontends se dirigen a contenedores del mismo VPS (`127.0.0.1`) y las de la API al VPS Backend.

Tabla 1.6: Rutas públicas de router-nginx

Ruta pública	Destino	Servicio
Frontends		
`/`	`127.0.0.1:8090`	defensoria-web
`/admin/`	`127.0.0.1:8091`	admin-web
`/revision/`	`127.0.0.1:8092`	revision-web
`/primer-contacto/`	`127.0.0.1:8094`	primer-contacto-web
`/subdefensoria/`	`127.0.0.1:8095`	subdefensoria-web
API		
`/api/auth/`	`156.67.26.73:8083`	auth-service
`/api/quejoso/`	`156.67.26.73:8084`	quejas-service
`/api/notificaciones/`	`156.67.26.73:8085`	notificaciones-service
`/api/catalogos/`	`156.67.26.73:8086`	catalogo-service
`/api/admin/`	`156.67.26.73:8087`	admin-service
`/api/revision/`	`156.67.26.73:8088`	revision-service
`/api/chatbot/`	`156.67.26.73:8089`	chatbot-service
`/api/primer-contacto/`	`156.67.26.73:8082`	primer-contacto-service
`/api/subdefensoria/`	`156.67.26.73:8091`	subdefensoria-service
`/api/denunciado/`	`156.67.26.73:8094`	denunciado-service

Además, `router-nginx` responde 403 (prohibido) a las siguientes rutas, que son de uso interno entre microservicios:

- `/api/notificaciones/enviar` y `/api/notificaciones/registrar`

- `/api/primer-contacto/ingesta/`
- `/api/subdefensoria/ingesta/`
- `/api/subdefensoria/acuerdos-conclusion/interno/`

El modelo de IA (`antecedentes-service`, puerto 8093) no tiene ruta pública: solo lo consumen otros microservicios dentro del backend. Las rutas bloqueadas existen porque esos endpoints están pensados para llamadas entre servicios; si quedaran expuestos, cualquier persona podría, por ejemplo, enviar correos desde la cuenta de la Defensoría.

2. Preparación de Nodos y Entorno de Ejecución (Común a las 3 VPS)

Antes de instalar cualquier componente de la plataforma, las tres VPS se dejan en un mismo estado base: acceso de administración configurado, sistema actualizado, misma zona horaria, Podman instalado y la estructura de carpetas creada con sus permisos. Los pasos de este capítulo se ejecutan en el VPS Frontend (169.58.62.111), el VPS Backend (156.67.26.73) y el VPS Base de Datos (169.58.62.99); cuando un paso aplica solo a uno de ellos, se indica.

2.1. Actualización base y acceso a los servidores

Las tres VPS son servidores con Ubuntu 24.04.5 LTS contratados con el proveedor Contabo. La administración se realiza de forma remota por SSH con el cliente Termius, usando la cuenta `root`. Esta sección explica primero cómo se configura ese acceso y después la actualización del sistema y la zona horaria, que es lo primero que se ejecuta al entrar a cada servidor.

2.1.1. Acceso por SSH como root con Termius

Cliente utilizado. Termius es un cliente SSH gráfico para Windows, macOS y Linux que guarda la lista de servidores con sus credenciales, abre cada conexión en una pestaña o en paneles dentro de un mismo espacio de trabajo (*Workspace*) e incluye un explorador SFTP para transferir archivos. Se utiliza en lugar de la terminal del sistema para tener en un solo lugar las tres VPS de la plataforma y trabajar con ellas al mismo tiempo.

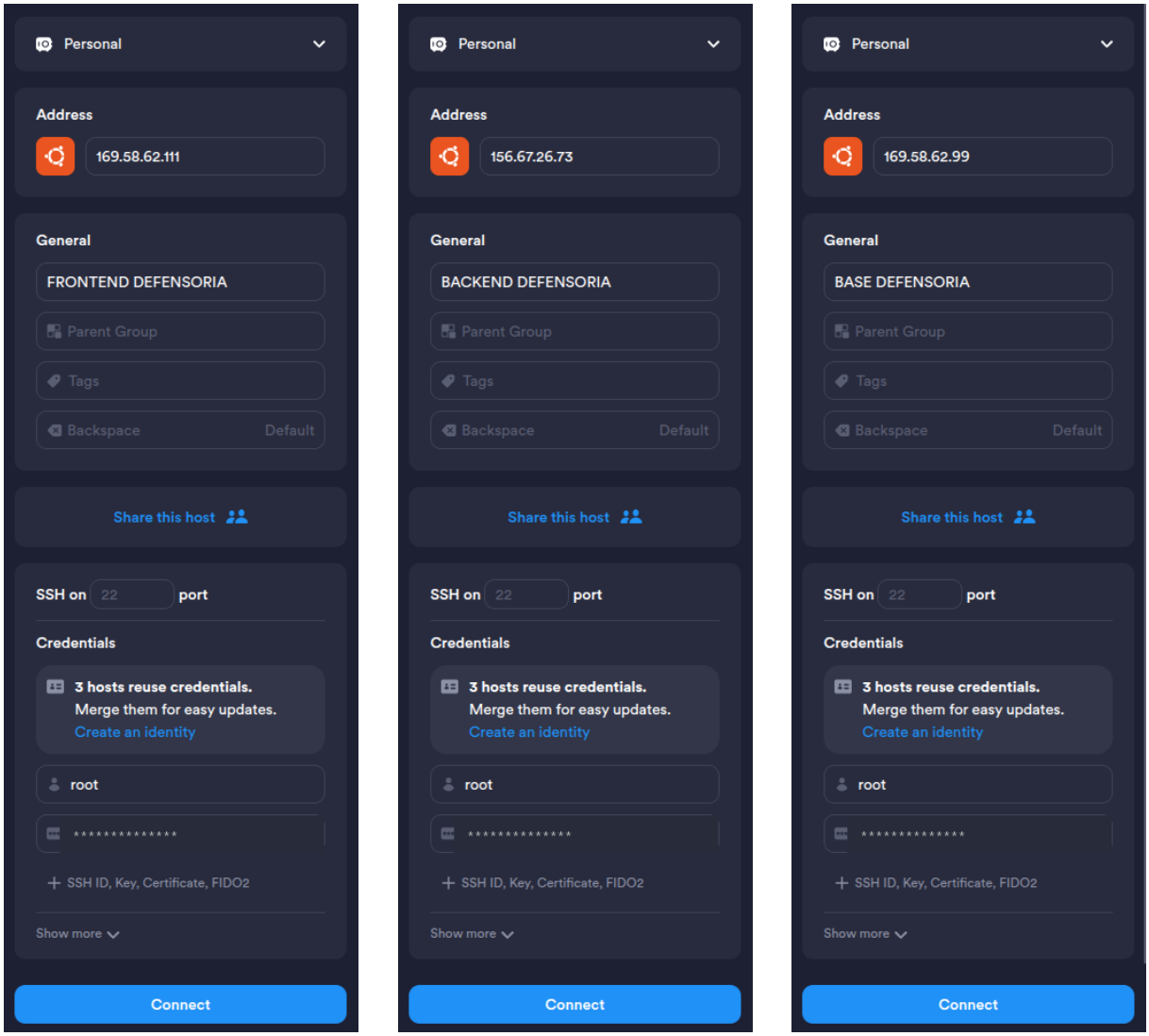
Registro de los servidores. En Termius se crea un *host* por cada VPS desde **Hosts** → **New host**, con los siguientes datos:

Tabla 2.1: Hosts registrados en Termius

Label (nombre)	Address	Puerto SSH	Usuario	Autenticación
FRONTEND DEFENSORIA	169.58.62.111	22	root	Contraseña
BACKEND DEFENSORIA	156.67.26.73	22	root	Contraseña
BASE DEFENSORIA	169.58.62.99	22	root	Contraseña

1. En **Address** se escribe la IP pública de la VPS.

2. En **General** se asigna el nombre con el que aparecerá en la lista (*Label*).
3. En **SSH on port** se deja el puerto 22.
4. En **Credentials** se captura el usuario `root` y su contraseña. Termius la guarda cifrada en su bóveda (*vault*) personal, por lo que no es necesario escribirla en cada conexión.
5. Se guarda el host y se repite para las otras dos VPS.



(a) VPS Frontend (b) VPS Backend (c) VPS Base de Datos

Figura 2.1: Configuración de los tres hosts en Termius (contraseña oculta)

Conexión a las tres VPS. Al seleccionar un host y pulsar **Connect**, Termius abre la sesión SSH. En la primera conexión muestra la huella digital (*fingerprint*) de la llave del servidor y pide confirmarla; al aceptarla la registra en **Known Hosts**. Si en una conexión posterior la huella cambia sin que el servidor se haya reinstalado, se debe rechazar la conexión, porque puede tratarse de un servidor suplantado.

Para la instalación conviene abrir las tres sesiones en un mismo *Workspace*, de modo que las tres VPS quedan visibles a la vez (Figura 2.2). El *prompt* de cada panel muestra el nombre de host del servidor (`root@vmi3633026`, `root@vmi3633021`, `root@vmi3633025`), lo que permite

confirmar en qué VPS se está escribiendo antes de ejecutar un comando. La opción **Broadcast** de Termius envía lo que se escribe a todos los paneles a la vez y sirve para los pasos idénticos en las tres VPS (actualización, zona horaria, instalación de Podman).

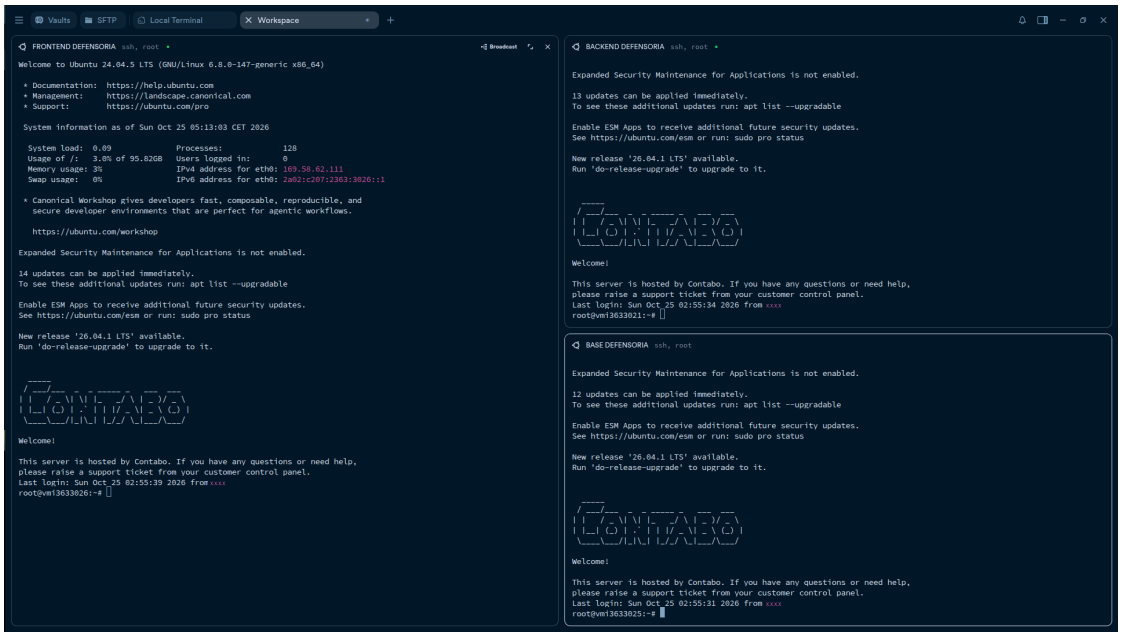


Figura 2.2: Sesión SSH como root en las tres VPS dentro de un Workspace de Termius

Nota: El acceso se hace con usuario `root` y contraseña, sin llaves SSH. Se recomienda que la contraseña sea larga (16 caracteres o más) y distinta en cada VPS: Termius advierte en la Figura 2.1 que los tres hosts reutilizan las mismas credenciales, de modo que si una se filtra quedan expuestos los tres servidores. Las contraseñas no se escriben en este manual y, al tomar capturas de Termius, el campo de contraseña debe quedar oculto.

2.1.2. Actualización del sistema y zona horaria (America/Mexico_City)

Actualización de paquetes. Al conectarse, Ubuntu indica en el mensaje de bienvenida cuántas actualizaciones están pendientes (entre 12 y 14 en la Figura 2.2). En cada VPS se actualiza la lista de paquetes y se instalan:

```
# apt update
# apt upgrade -y
```

Si la actualización incluyó un kernel nuevo, Ubuntu crea el archivo `/var/run/reboot-required`; en ese caso se reinicia la VPS y se vuelve a conectar desde Termius:

```
# [ -f /var/run/reboot-required ] && reboot
```

Nota: El mensaje de bienvenida también ofrece actualizar a Ubuntu 26.04 LTS (`do-release-upgrade`). **No** se aplica: la plataforma se instaló y probó sobre Ubuntu 24.04 LTS, que sigue recibiendo actualizaciones de seguridad; un cambio de versión mayor se planea por separado.

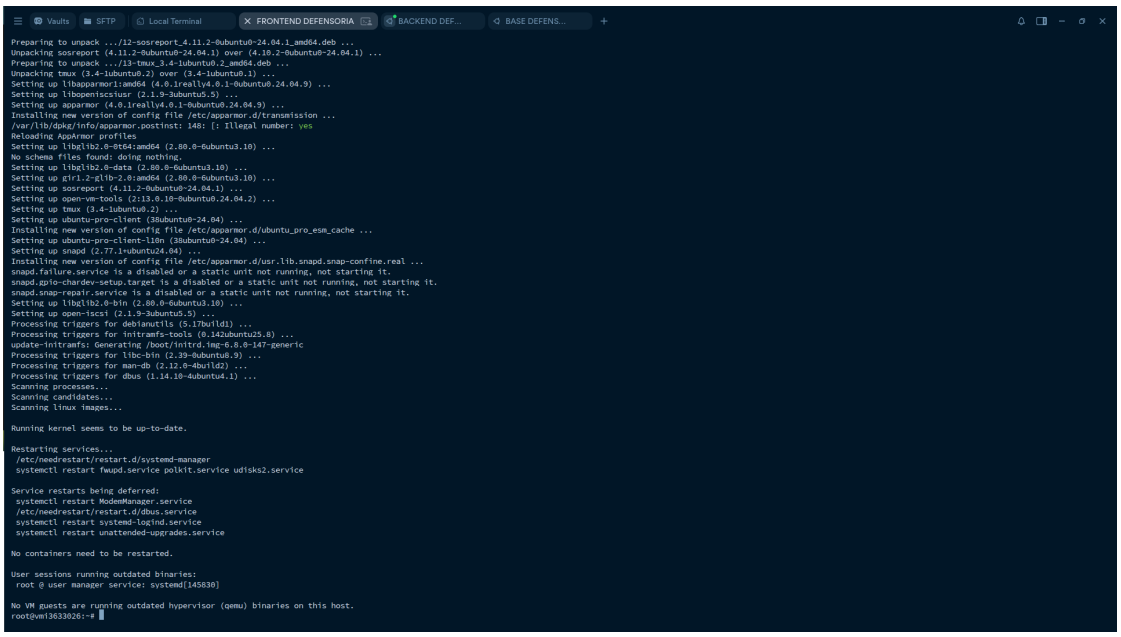


Figura 2.3: Actualización del sistema en el VPS Frontend



Figura 2.4: Actualización del sistema en el VPS Backend

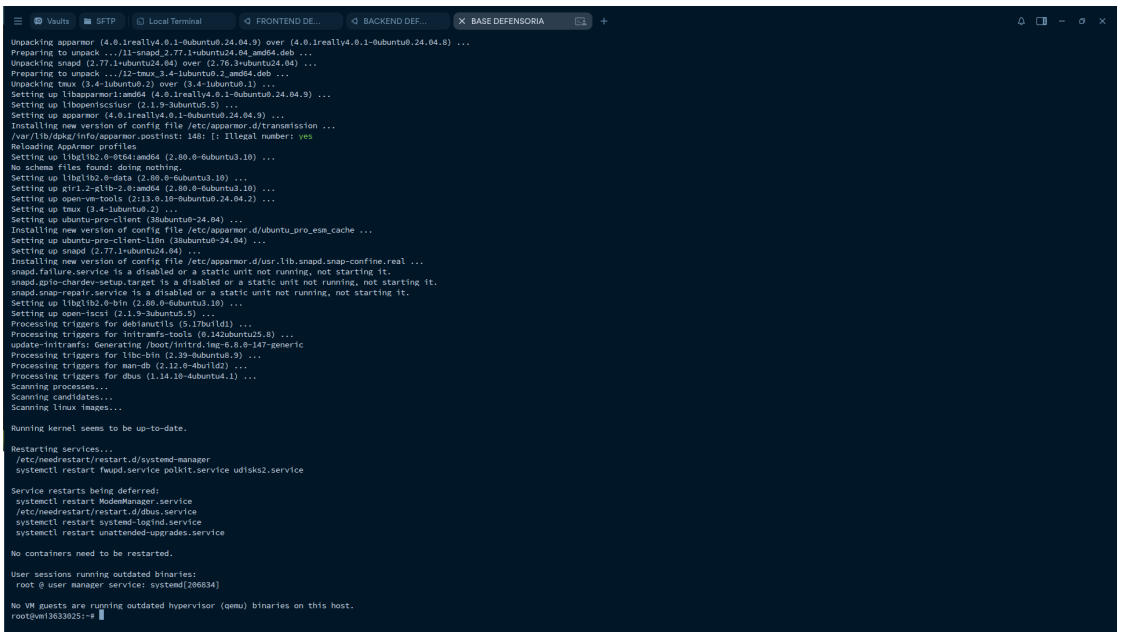


Figura 2.5: Actualización del sistema en el VPS Base de Datos

Zona horaria. Las VPS vienen con la zona horaria del centro de datos de Contabo en Europa; por eso el mensaje de bienvenida de la Figura 2.2 muestra la hora en `CET`. Se cambian a la hora del centro de México para que las fechas de los registros del sistema, de los logs de los contenedores y de los respaldos coincidan con la hora local de la Defensoría (por ejemplo, el respaldo diario programado a las 4:00 se ejecuta a las 4:00 hora de México y no a media noche):

```
# timedatectl set-timezone America/Mexico_City
# timedatectl
```

La salida de `timedatectl` debe mostrar `Time zone: America/Mexico_City (CST, -0600)` y `System clock synchronized: yes`; esto último indica que el reloj se sincroniza automáticamente por NTP.



Figura 2.6: Zona horaria America/Mexico_City en el VPS Frontend

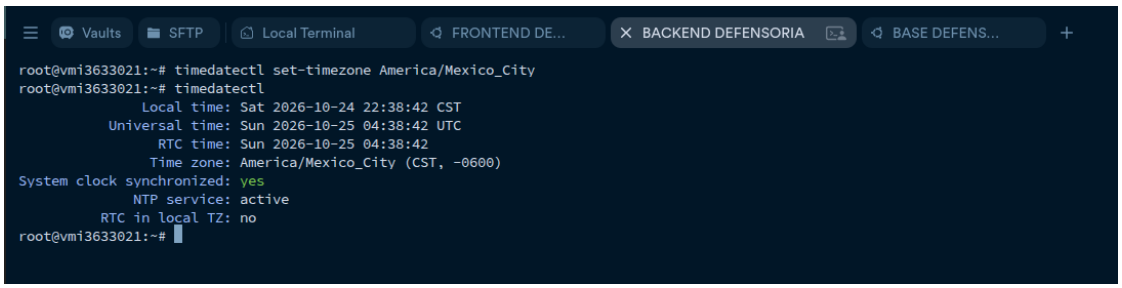


Figura 2.7: Zona horaria America/Mexico_City en el VPS Backend

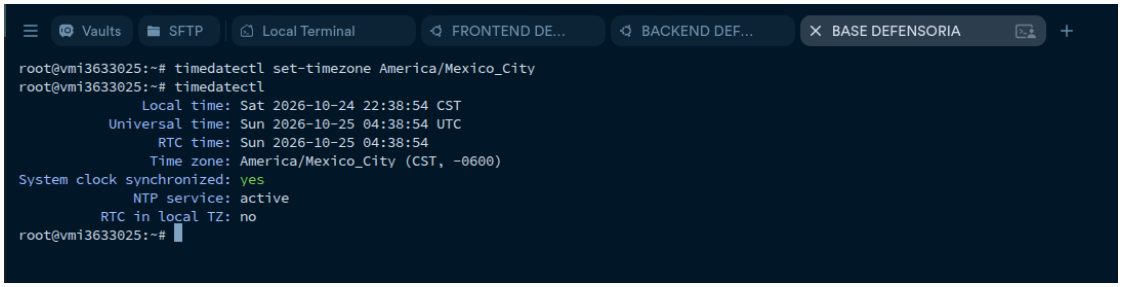


Figura 2.8: Zona horaria America/Mexico_City en el VPS Base de Datos

Nota: La base de datos también trabaja en esta zona horaria: el contenedor `defensoria-db` se crea con `TZ=America/Mexico_City` y el script de inicialización fija la misma zona en `defensoria_db` (capítulo 5).

2.2. Instalación y aprovisionamiento de Podman

Todos los componentes de la plataforma (base de datos, microservicios, modelo de IA, frontends y nginx) se ejecutan en contenedores. Como motor de contenedores se usa Podman, que es compatible con las imágenes y los Dockerfile de Docker pero no necesita un proceso servidor (*daemon*) permanente: cada contenedor es un proceso más del sistema, lo que reduce el consumo de memoria en VPS pequeñas.

2.2.1. Instalación de Podman y utilerías (podman, curl, iptables, python3, certbot)

En las **tres** VPS se instalan Podman y las utilerías que usan los scripts de la plataforma:

```
# apt install -y podman curl iptables
# podman --version
```

- `podman`: motor de contenedores. Ubuntu 24.04 instala la versión 4.9.
- `curl`: pruebas de los servicios desde la terminal (`curl http://127.0.0.1:<puerto>`).
- `iptables`: reglas de firewall que aplican los scripts de la plataforma (capítulo 3).

Además, según el servidor:

```
# apt install -y python3            # solo VPS Backend
# apt install -y certbot            # solo VPS Frontend
```

- `python3` (Backend): lo usa el generador de la carga masiva de quejas (`generar-csv.py`).
- `certbot` (Frontend): emisión y renovación del certificado HTTPS (sección 7.3).

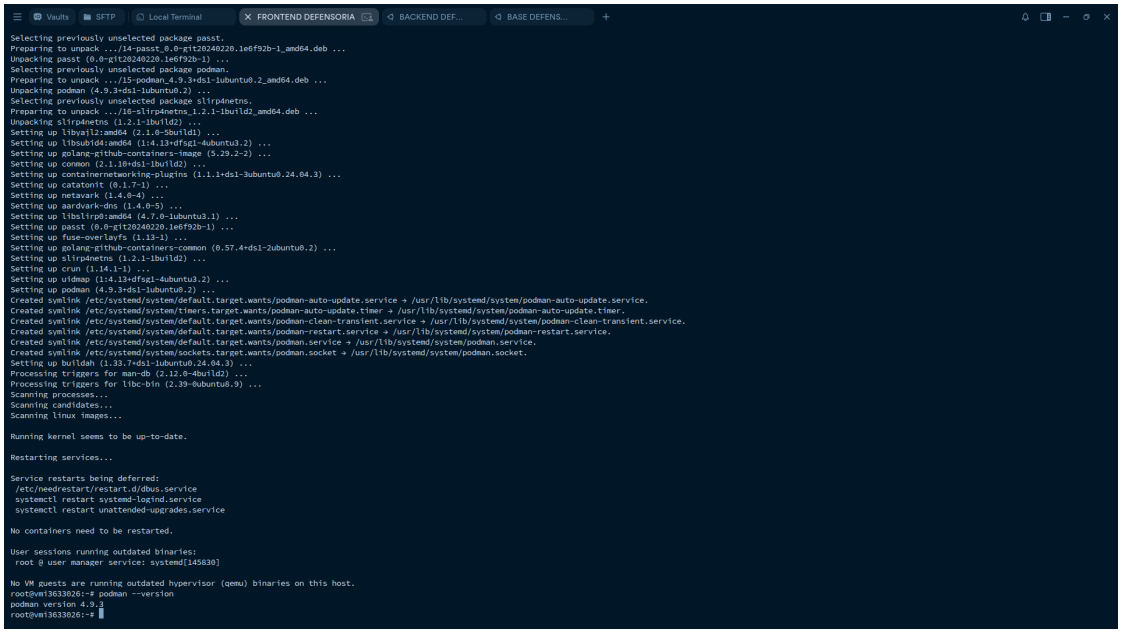


Figura 2.9: Instalación de Podman y versión en el VPS Frontend

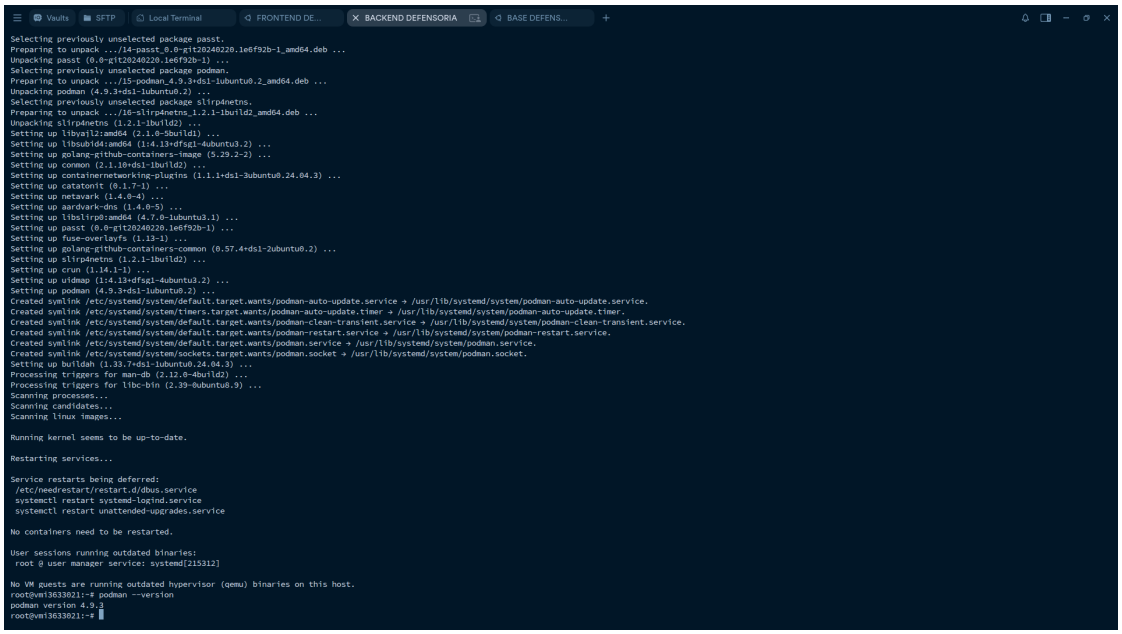


Figura 2.10: Instalación de Podman y versión en el VPS Backend

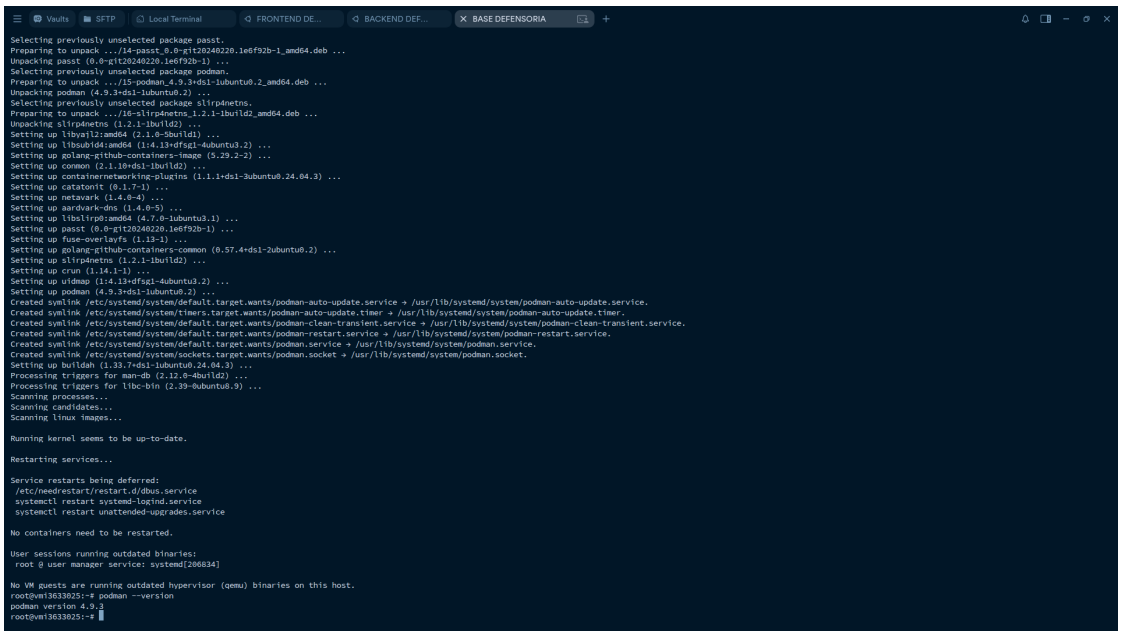


Figura 2.11: Instalación de Podman y versión en el VPS Base de Datos

2.2.2. Imágenes base con nombre completo (docker.io/library/...)

Podman puede buscar una imagen en varios registros cuando se le da un nombre corto (por ejemplo `postgres`); en ese caso pregunta de forma interactiva de cuál registro descargarla, lo que detiene un script. Para evitarlo, todos los scripts y Dockerfile de la plataforma usan el nombre completo de la imagen, con registro incluido, por lo que no es necesario modificar `/etc/containers/registries.conf`:

Tabla 2.2: Imágenes base de la plataforma

Imagen	VPS	Uso
`docker.io/library/postgres:16-alpine`	Base de datos	`defensoria-db`
`docker.io/library/eclipse-temurin:21-jre-jammy`	Backend	Microservicios y modelo de IA
`docker.io/library/nginx:alpine`	Frontend	`router-nginx` y los 5 frontends

Para comprobar que la VPS puede descargar imágenes de Docker Hub, se descarga la imagen que le corresponde:

```
# podman pull docker.io/library/nginx:alpine                    # Frontend
# podman pull docker.io/library/eclipse-temurin:21-jre-jammy    # Backend
# podman pull docker.io/library/postgres:16-alpine              # BD
# podman images
```

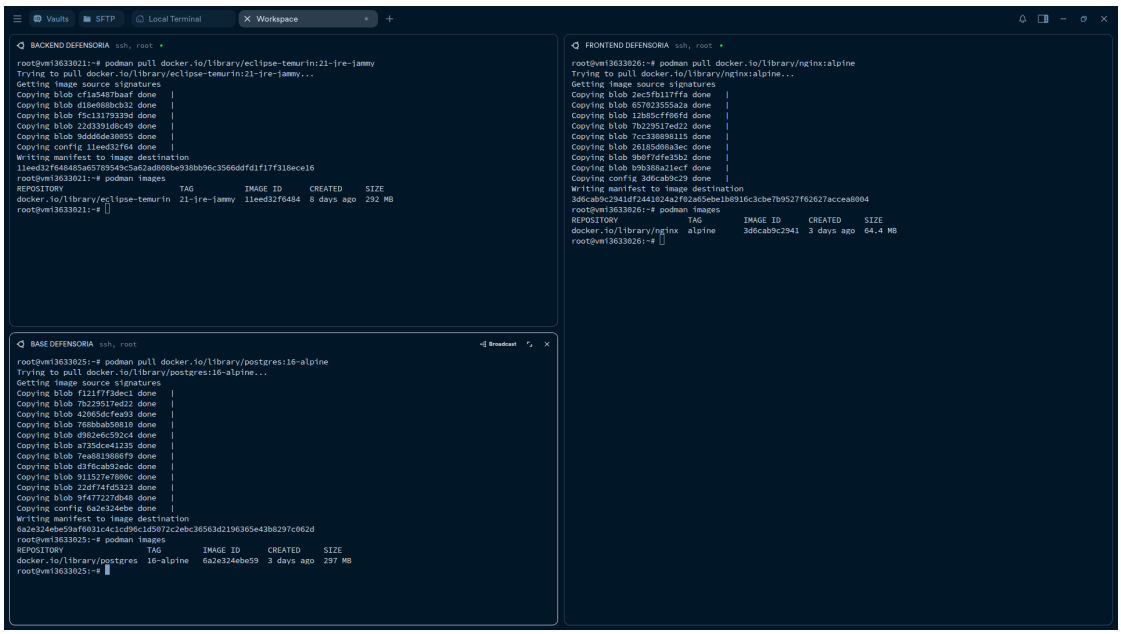


Figura 2.12: Imagen base descargada en cada VPS (salida de podman images)

2.2.3. Contenedores ejecutados como root con podman run

Los contenedores de la plataforma se crean como `root` (modo *rootful*) con el comando `podman run`, que los scripts de cada servidor ejecutan con todos sus parámetros (nombre, puertos, memoria, volúmenes y variables). Se eligió este modo porque:

- `router-nginx` necesita escuchar en los puertos 80 y 443, que en Linux están reservados para `root`.
- Los volúmenes montados (datos de PostgreSQL, configuración, respaldos) pertenecen a `root`, igual que el resto de `/apps`.
- Todos los contenedores, imágenes y reglas de firewall quedan administrados desde la misma cuenta con la que se conecta Termius.

En modo *rootful*, Podman guarda las imágenes y contenedores en `/var/lib/containers/storage`. Se verifica con:

```
# podman info --format '{{.Store.GraphRoot}}'
# podman ps -a
```

La primera línea debe mostrar `/var/lib/containers/storage`; `podman ps -a` aparece vacío hasta que se levanten los componentes en los capítulos 5 a 7.

2.2.4. Arranque automático de contenedores (podman-restart.service)

Como Podman no tiene un *daemon*, los contenedores no vuelven a arrancar solos después de reiniciar la VPS. Para lograrlo, Podman incluye el servicio de systemd `podman-restart`, que al encender el servidor levanta todos los contenedores creados con la política `--restart always`. Se habilita en las tres VPS:

```
# systemctl enable --now podman-restart.service
# systemctl is-enabled podman-restart.service
```

La salida debe ser `enabled`. Los scripts de la plataforma crean cada contenedor con `--restart always`, por lo que este servicio es el que garantiza que la plataforma se recupere sola tras un reinicio.

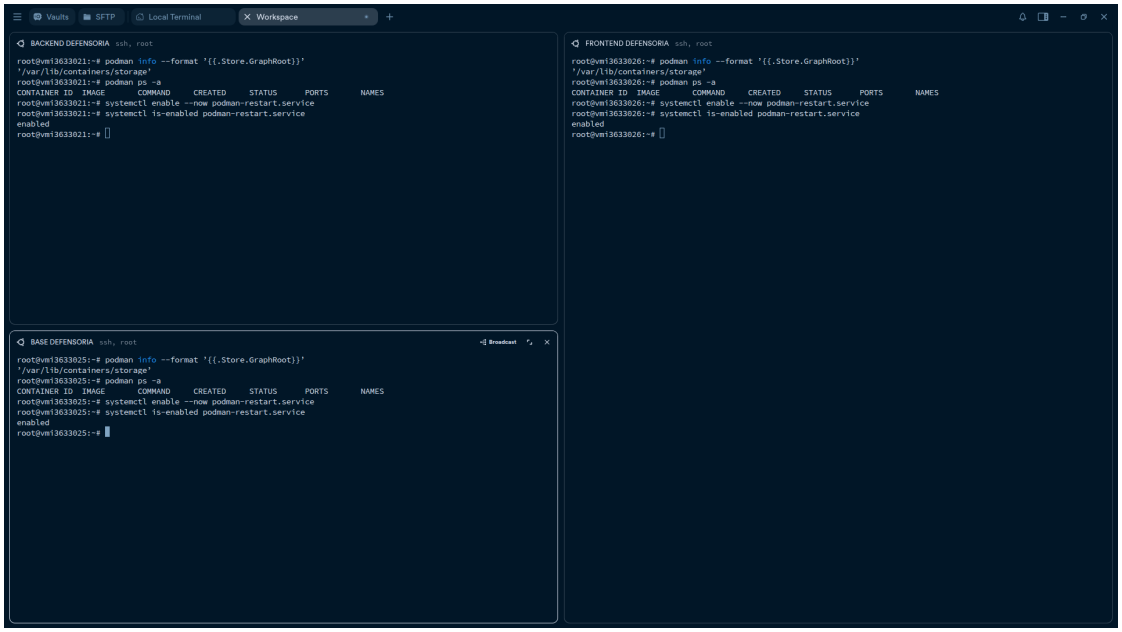


Figura 2.13: Servicio podman-restart habilitado

2.3. Estructura de directorios y gestión de permisos

Todos los archivos de la plataforma viven bajo `/apps` en cada VPS. Las carpetas se crean en este paso, vacías y con sus permisos; los archivos (scripts, JAR, configuración, frontends) se colocan en ellas en los capítulos 5 a 7.

2.3.1. Árbol de carpetas en /apps (aplicaciones/defensoria, data/postgresql, respaldos, utiles/respaldos)

Tabla 2.3: Carpetas principales

Carpeta	Contenido
`/apps/aplicaciones/defensoria`	Scripts, configuración y artefactos de la plataforma (las tres VPS)
`/apps/data/postgresql`	Datos de PostgreSQL (solo VPS Base de Datos)
`/apps/respaldos/bd`	Respaldos de la base generados por `podman-compose-bd.sh backup` (VPS Base de Datos)
`/apps/utiles/respaldos`	Respaldos generados desde el Panel de la Defensora (VPS Backend)

VPS Base de Datos (169.58.62.99).

```
/apps/
|-- aplicaciones/defensoria/
|   |-- database/              podman-compose-bd.sh
|   `-- database-scripts/
```

```
|       |-- init/              00-init.sql (se ejecuta al crear la base)
|       `-- seeds/             01..05-*.sql (datos semilla)
|-- data/postgresql/           defensoria_db/ (la crea el script)
`-- respaldos/bd/              respaldos .sql.gz
```

VPS Backend (156.67.26.73).

```
/apps/
|-- aplicaciones/defensoria/back/
|   |-- Dockerfile, podman-compose.sh, podman-compose-modelo.sh, firewall-
    backend.sh
|   |-- artifact/              11 archivos .jar
|   |-- admin-service/         Dockerfile propio de admin-service
|   |-- config-files/          <servicio>/config/<servicio>.yml
|   |-- modelo/Dataset/        corpus JSON del modelo de IA
|   |-- modelo/pruebas/        peticiones de prueba del modelo
|   `-- carga-masiva/          cargar-quejas.sh, generar-csv.py
`-- utiles/respaldos/          respaldos del Panel de la Defensora
```

VPS Frontend (169.58.62.111).

```
/apps/aplicaciones/defensoria/
|-- front/                     defensoria-web (portal)
|   |-- config/                static.conf
|   `-- dist/browser/          archivos compilados de Angular
|-- front-admin/dist/browser/            admin-web
|-- front-revision/dist/browser/         revision-web
|-- front-primer-contacto/dist/browser/  primer-contacto-web
|-- front-subdefensoria/dist/browser/    subdefensoria-web
`-- router/
    |-- config/                defensoria.conf (router-nginx)
    `-- certbot-webroot/       reto HTTP-01 de Let's Encrypt
```

Creación de las carpetas. En el VPS Base de Datos:

```
# A=/apps/aplicaciones/defensoria
# mkdir -p $A/database $A/database-scripts/init $A/database-scripts/seeds \
           /apps/data/postgresql /apps/respaldos/bd
```

En el VPS Backend:

```
# B=/apps/aplicaciones/defensoria/back
# mkdir -p $B/artifact $B/admin-service $B/config-files \
           $B/modelo/Dataset $B/modelo/pruebas $B/carga-masiva \
           /apps/utiles/respaldos
```

En el VPS Frontend:

```
# F=/apps/aplicaciones/defensoria
# mkdir -p $F/front/config $F/front/dist/browser \
           $F/front-admin/dist/browser $F/front-revision/dist/browser \
           $F/front-primer-contacto/dist/browser $F/front-subdefensoria/dist/
   browser \
           $F/router/config $F/router/certbot-webroot
```

2.3.2. Políticas de propiedad y permisos (root:root, 755 carpetas, 750 scripts, 750/640 config-files, 700 respaldos)

Todo `/apps` pertenece a `root:root`, porque los contenedores se ejecutan como `root` (sección 2.2). Los permisos se asignan según lo que contiene cada carpeta o archivo:

Tabla 2.4: Política de permisos

Carpeta o archivo	Permiso	Motivo
Carpetas generales	755	Cualquiera puede recorrerlas y leerlas; solo `root` escribe
Scripts `.sh`	750	Solo `root` los ejecuta
`config-files/` (carpetas / archivos)	750 / 640	Contienen la contraseña de la BD, el secreto JWT y la contraseña SMTP
`podman-compose-bd.sh` y su carpeta	700	El script contiene la contraseña de PostgreSQL
JAR, Dockerfile, SQL, JSON, archivos web	644	Solo lectura; se copian dentro de las imágenes
Carpetas de respaldos	700	Contienen copias completas de la base de datos
`database-scripts/init`	755 / 644	PostgreSQL ejecuta `00-init.sql` con su propio usuario dentro del contenedor y necesita leerlo
`/apps/data/postgresql/*`	–	No se modifica a mano: el contenedor asigna su propio dueño a la carpeta de datos

En el VPS Base de Datos:

```
# chown -R root:root /apps
# chmod 755 /apps /apps/aplicaciones /apps/aplicaciones/defensoria /apps/data /
    apps/data/postgresql
# chmod 755 $A/database-scripts $A/database-scripts/init $A/database-scripts/
    seeds
# chmod 700 $A/database
# chmod 755 /apps/respaldos && chmod 700 /apps/respaldos/bd
```

En el VPS Backend:

```
# chown -R root:root /apps
# chmod 755 /apps /apps/aplicaciones /apps/aplicaciones/defensoria $B \
            $B/artifact $B/admin-service $B/modelo $B/modelo/Dataset \
            $B/modelo/pruebas $B/carga-masiva /apps/utiles
# chmod 750 $B/config-files
# chmod 700 /apps/utiles/respaldos
```

En el VPS Frontend:

```
# chown -R root:root /apps
# find /apps -type d -exec chmod 755 {} +
```

Los permisos de los archivos (750 para scripts, 640 para configuración, 644 para el resto) se aplican cuando se colocan en los capítulos 5 a 7.

Verificación. En cada VPS se listan las carpetas con su dueño y permisos:

```
# find /apps -type d -printf "%M %u %g %p\n"
```

La salida muestra, por cada carpeta, sus permisos (`%M`, por ejemplo `drwxr-xr-x` = 755), su dueño (`%u`), su grupo (`%g`) y su ruta (`%p`). Cada carpeta debe aparecer con dueño `root root` y con el permiso de la tabla anterior.

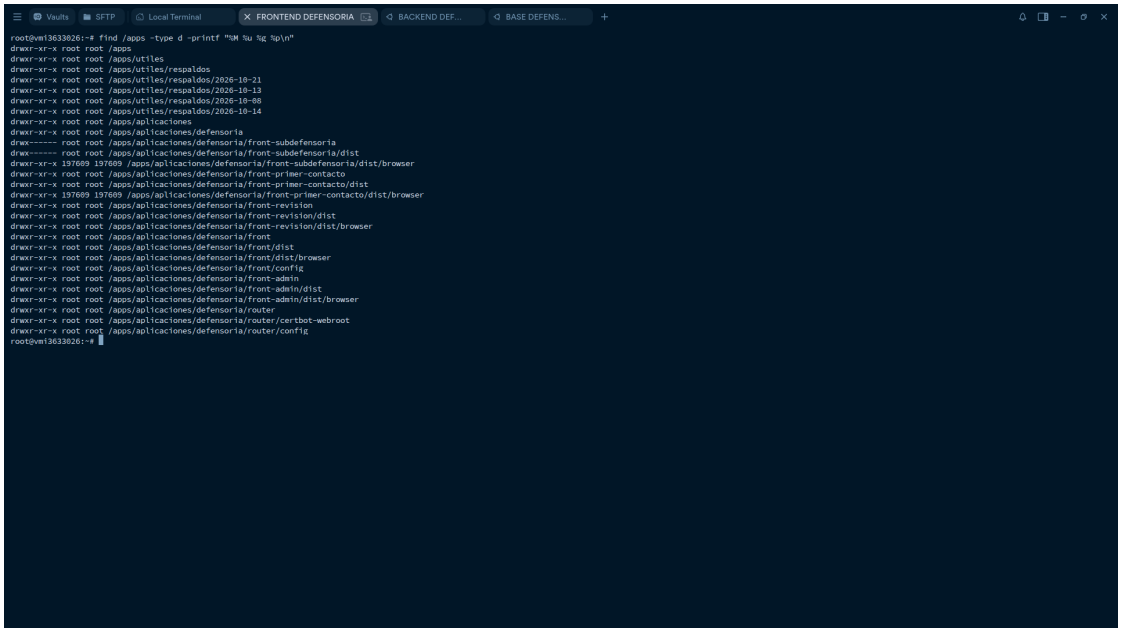


Figura 2.14: Dueño y permisos de las carpetas en el VPS Frontend

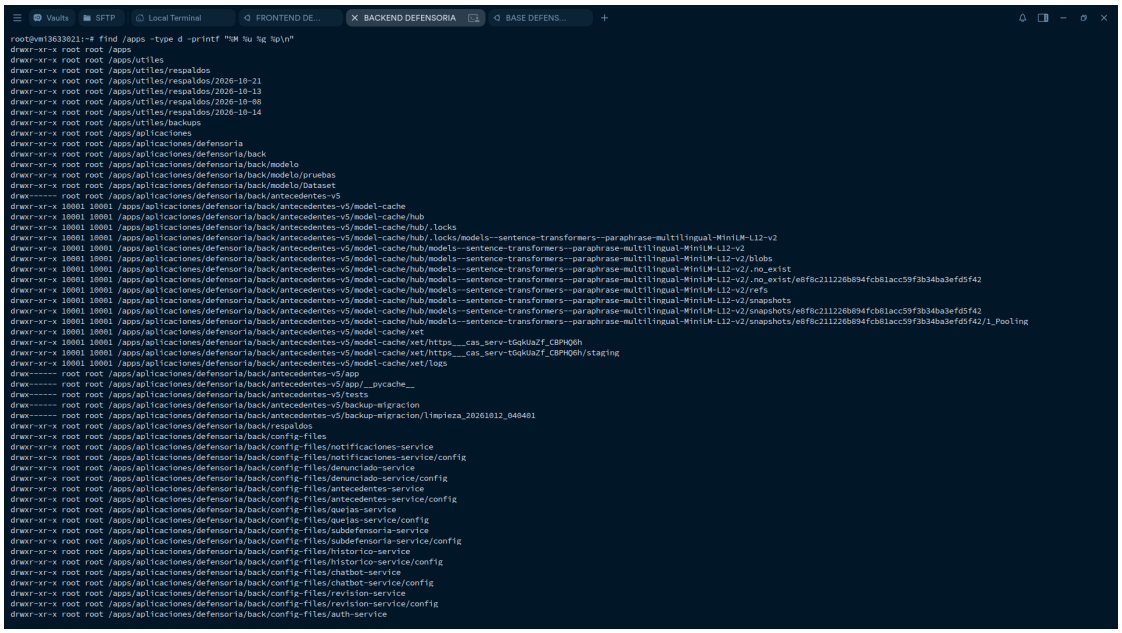


Figura 2.15: Dueño y permisos de las carpetas en el VPS Backend

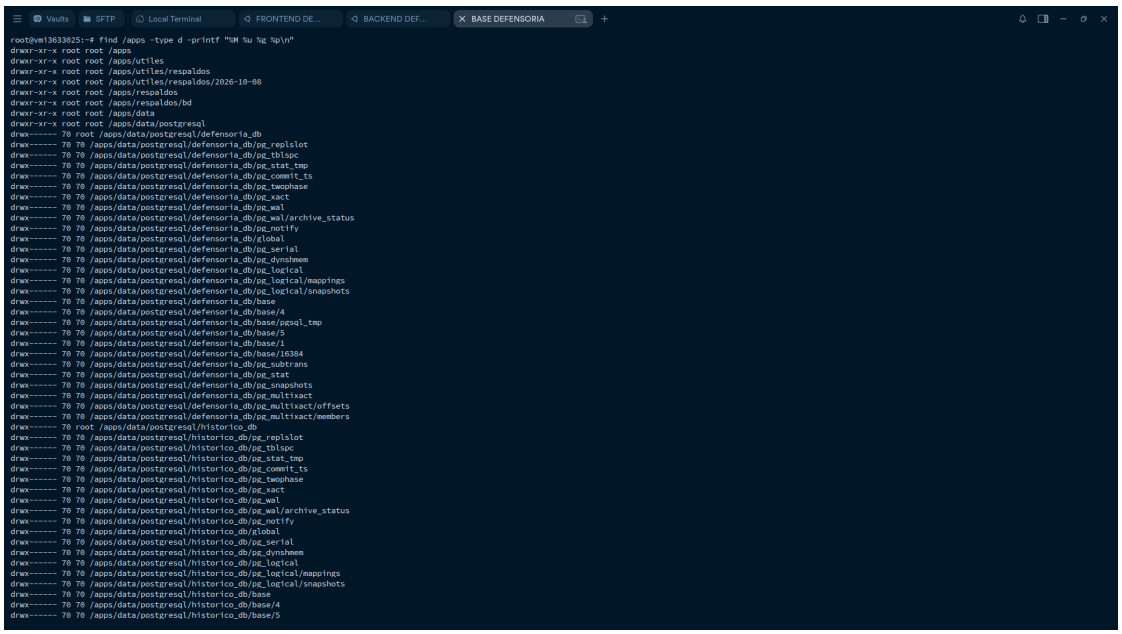


Figura 2.16: Dueño y permisos de las carpetas en el VPS Base de Datos

3. Seguridad de Red y Configuración de Firewalls (ufw / iptables)

Este capítulo define qué tráfico puede llegar a cada VPS. La regla general es que cada capa solo acepta conexiones de la capa anterior: Internet solo llega al Frontend, el Frontend es el único que llega al Backend y el Backend es el único que llega a la Base de Datos. El puerto 22 (SSH) queda abierto en las tres VPS para la administración con Termius.

La protección se configura en dos capas que trabajan juntas:

1. **ufw** (*Uncomplicated Firewall*), con política de rechazar todo lo entrante salvo lo permitido explícitamente.
2. Una **regla en la tabla raw de iptables**, aplicada por los scripts de la plataforma, que descarta las conexiones a los puertos de los contenedores que no vengan de la IP autorizada.

3.1. Política de firewall con Podman

3.1.1. ufw: política por defecto y reglas de entrada (IN) y de reenvío (FWD)

ufw es la interfaz de firewall incluida en Ubuntu. Se configura con una política por defecto y una lista de excepciones:

```
# ufw default deny incoming      # rechazar todo lo que entra
# ufw default allow outgoing     # permitir todo lo que sale
# ufw default deny routed        # rechazar el trafico reenviado (valor por
    defecto)
```

Con Podman hay que distinguir dos tipos de tráfico, porque ufw los filtra en lugares distintos:

Tabla 3.1: Tipos de reglas de ufw

Tipo	Comando	A qué tráfico se aplica
Entrada (ALLOW IN)	`ufw allow ...`	Conexiones dirigidas a un proceso del propio servidor, como `sshd` en el puerto 22. Cadena `INPUT`.
Reenvío (ALLOW FWD)	`ufw route allow ...`	Conexiones que el servidor reenvía a otra red. Es el caso de los contenedores: cuando Podman publica un puerto (`-p 5432:5432`), redirige la conexión a la IP interna del contenedor (*DNAT*) y el paquete pasa por la cadena `FORWARD`, no por `INPUT`.

Por eso una regla de entrada como `ufw allow 5432` por sí sola no basta para un contenedor: también se necesita la regla de reenvío correspondiente (`ufw route allow`). Del mismo modo, una regla `deny` de entrada no bloquea el puerto de un contenedor.

3.1.2. Capa adicional: regla en la tabla raw de iptables

Como segunda capa, independiente de ufw, los scripts de la plataforma agregan una regla en la cadena `PREROUTING` de la tabla `raw`. Esta tabla se evalúa **antes** que cualquier otra (antes de la redirección de Podman y antes de las reglas de ufw), por lo que el paquete no autorizado se descarta en cuanto llega al servidor, sin importar cómo estén configuradas las demás reglas.

```
iptables -t raw -I PREROUTING -i eth0 -p tcp --dport 5432 ! -s 156.67.26.73 -j
    DROP
```

Tabla 3.2: Partes de la regla raw

Parte	Significado
`-t raw`	Tabla que iptables evalúa primero.
`-I PREROUTING`	Inserta la regla en la cadena por la que pasa todo paquete al llegar.
`-i eth0`	Solo el tráfico que entra por la interfaz pública. El tráfico local y el de la red interna de Podman no se toca.
`-p tcp --dport 5432`	Conexiones TCP a ese puerto (o rango, como `8082:8094`).
`! -s 156.67.26.73`	"Cuyo origen *no* sea esta IP". La IP autorizada queda exenta.
`-j DROP`	Descartar el paquete sin responder; desde fuera el puerto parece no existir.

La interfaz pública en las VPS de Contabo es `eth0`:

```
# ip route show default | awk '{print $5; exit}'
eth0
```

Nota: En Ubuntu 24.04, iptables y ufw escriben sus reglas en *nftables*, el filtro de paquetes actual del kernel. Por eso todas las reglas (las de ufw, la regla raw y las que crea Podman) pueden verse juntas con `nft list ruleset`.

3.2. Reglas perimetrales por nodo

Tabla 3.3: Resumen de la protección por VPS

VPS	ufw	Permitido	Regla raw
Base de datos	Activo	22 desde cualquiera; 5432 solo desde 156.67.26.73	5432 solo desde 156.67.26.73
Backend	Activo	22 desde cualquiera; puertos de los microservicios solo desde 169.58.62.111	8082–8094 solo desde 169.58.62.111
Frontend	Inactivo	Todo (solo `router-nginx` debe atender en 80/443)	Ninguna

3.2.1. VPS Base de Datos: puerto 5432 exclusivamente desde la IP del VPS Backend

ufw. En el VPS Base de Datos (169.58.62.99):

```
# ufw default deny incoming
# ufw default allow outgoing
# ufw allow 22/tcp
# ufw allow from 156.67.26.73 to any port 5432 proto tcp
# ufw route allow from 156.67.26.73 to any port 5432 proto tcp
# ufw enable
```

Tabla 3.4: Reglas de ufw del VPS Base de Datos

Regla	Tipo	Motivo
`22/tcp` desde cualquiera	IN	Administración por SSH con Termius. Se agrega **antes** de `ufw enable` para no perder la conexión.
`5432/tcp` desde 156.67.26.73	IN	Permite al Backend llegar al puerto publicado de PostgreSQL.
`5432/tcp` desde 156.67.26.73	FWD	Permite que esa conexión se reenvíe al contenedor `defensoria-db`. Sin esta regla el Backend no puede conectarse.
Todo lo demás	–	Rechazado por la política `deny incoming`.

Regla raw. La aplica el script de la base de datos (capítulo 5):

```
# cd /apps/aplicaciones/defensoria/database
# bash podman-compose-bd.sh firewall        # aplica la regla
# bash podman-compose-bd.sh firewall-boot   # la reaplica en cada reinicio
```

El script descarta en la tabla raw todo lo que llegue a 5432 (y a 5433, puerto de la antigua base histórica) desde una IP distinta del Backend.

3.2.2. VPS Backend: puertos de los microservicios exclusivamente desde la IP del VPS Frontend

ufw. En el VPS Backend (156.67.26.73):

```
# ufw default deny incoming
# ufw default allow outgoing
# ufw allow 22/tcp
# for p in 8082 8083 8084 8085 8086 8087 8088 8089 8091 8094; do
    ufw allow from 169.58.62.111 to any port $p proto tcp
  done
# ufw route allow in on eth0 out on podman0
# ufw route allow in on podman0 out on eth0
# ufw route allow in on eth0 out on podman1
# ufw route allow in on podman1 out on eth0
# ufw enable
```

Tabla 3.5: Reglas de ufw del VPS Backend

Regla	**Tipo**	**Motivo**
22/tcp desde cualquiera	IN	Administración por SSH.
8082–8089, 8091, 8094 desde 169.58.62.111	IN	Un puerto por microservicio con ruta pública: Primer Contacto (8082), auth (8083), quejas (8084), notificaciones (8085), catálogo (8086), admin (8087), revisión (8088), chatbot (8089), Subdefensoría (8091) y denunciado (8094).
eth0 → podman0	FWD	Permite que el tráfico que llega por la interfaz pública se reenvíe a la red de los contenedores.
podman0 → eth0	FWD	Permite que los contenedores salgan a Internet y a otras VPS: conexión a la base de datos (169.58.62.99:5432) y envío de correos por SMTP.
eth0 ↔ podman1	FWD	Lo mismo para la segunda red de Podman (10.89.0.0/24).

Nota: Las reglas de reenvío eth0 → podman0/podman1 permiten llegar a **cualquier** puerto de los contenedores desde cualquier origen. Por eso, en el Backend, quien realmente limita los microservicios al VPS Frontend es la regla raw del siguiente párrafo; las reglas de entrada por puerto documentan qué puertos son públicos para el Frontend.

Regla raw. La aplica `firewall-backend.sh` (capítulo 6):

```
# cd /apps/aplicaciones/defensoria/back
# bash firewall-backend.sh          # aplica la regla
# bash firewall-backend.sh boot     # la reaplica en cada reinicio
```

Descarta todo lo que llegue por eth0 a los puertos 8082–8094 desde una IP distinta del Frontend. Se limita a eth0 a propósito: los microservicios se llaman entre sí por la IP pública (por ejemplo 156.67.26.73:8085), pero ese tráfico entra por la red de Podman; una regla sin -i eth0 lo bloquearía y las peticiones quedarían colgadas.

Tabla 3.6: Puertos del VPS Backend

Puerto	Servicio	Quién puede entrar	Ruta en nginx
22	SSH	Cualquiera	–
8082	primer-contacto-service	169.58.62.111	`/api/primer-contacto/`
8083	auth-service	169.58.62.111	`/api/auth/`
8084	quejas-service	169.58.62.111	`/api/quejoso/`
8085	notificaciones-service	169.58.62.111	`/api/notificaciones/`
8086	catalogo-service	169.58.62.111	`/api/catalogos/`
8087	admin-service	169.58.62.111	`/api/admin/`
8088	revision-service	169.58.62.111	`/api/revision/`
8089	chatbot-service	169.58.62.111	`/api/chatbot/`
8091	subdefensoria-service	169.58.62.111	`/api/subdefensoria/`
8093	antecedentes-service (IA)	Solo otros microservicios	Sin ruta pública
8094	denunciado-service	169.58.62.111	`/api/denunciado/`

3.2.3. VPS Frontend: tráfico público 80 (HTTP) y 443 (HTTPS)

En el VPS Frontend (169.58.62.111) ufw está **inactivo** y no hay regla raw:

```
# ufw status
Status: inactive
```

Es la única VPS que debe atender a Internet: `router-nginx` recibe a los usuarios en los puertos 80 y 443, y los cinco frontends (8090–8095) solo deben consumirse a través de él por `127.0.0.1`.

Configuración recomendada. Para dejar abiertos únicamente SSH, HTTP y HTTPS, y cerrar a Internet los puertos de los frontends:

```
# ufw default deny incoming
# ufw default allow outgoing
# ufw allow 22/tcp
# ufw allow 80/tcp
# ufw allow 443/tcp
# ufw enable
# IFACE=$(ip route show default | awk '{print $5; exit}')
# iptables -t raw -I PREROUTING -i $IFACE -p tcp --dport 8090:8095 -j DROP
```

`router-nginx` usa la red del host y llega a los frontends por la interfaz local `lo`, así que ninguna de estas reglas le afecta. La regla raw no sobrevive a un reinicio; para hacerla permanente se agrega a un servicio de systemd como los que crean los scripts del Backend y la Base de Datos.

3.2.4. Rutas internas bloqueadas en nginx (403)

Además del firewall, `router-nginx` niega desde Internet los endpoints que solo deben usarse entre microservicios. Estas líneas forman parte de `defensoria.conf` (sección 7.4):

```
location = /api/notificaciones/enviar    { return 403; }
location = /api/notificaciones/registrar { return 403; }
location /api/primer-contacto/ingesta/   { return 403; }
```

```
location /api/subdefensoria/ingesta/     { return 403; }
location /api/subdefensoria/acuerdos-conclusion/interno/ { return 403; }
```

Los microservicios siguen pudiendo usarlos porque se llaman directamente por `156.67.26.73:<puerto>`, sin pasar por nginx.

3.2.5. Persistencia de las reglas tras reiniciar

- **ufw**: al ejecutar `ufw enable` queda habilitado el servicio `ufw.service`, que carga las reglas guardadas en cada arranque. No requiere nada adicional.
- **Regla raw**: iptables no la guarda por sí mismo. Los comandos `firewall-boot` (BD) y `boot` (Backend) de los scripts crean un servicio de systemd que la vuelve a aplicar al encender la VPS:

Tabla 3.7: Servicios de systemd del firewall

VPS	Servicio	Lo crea
Base de datos	`ufw.service`, `defensoria-bd-firewall.service`	`ufw enable`, `podman-compose-bd.sh firewall-boot`
Backend	`ufw.service`, `defensoria-backend-firewall.service`	`ufw enable`, `firewall-backend.sh boot`

```
# systemctl is-enabled ufw defensoria-bd-firewall        # VPS Base de Datos
# systemctl is-enabled ufw defensoria-backend-firewall   # VPS Backend
```

Verificación de las reglas de cada VPS. En cada VPS se listan las reglas de ufw numeradas y la regla raw:

```
# ufw status numbered
# iptables -t raw -L PREROUTING -n -v --line-numbers
```

- `ufw status numbered`: lista cada regla con su número, el puerto (`To`), la acción (`ALLOW IN` o `ALLOW FWD`) y el origen (`From`). El número sirve para borrar una regla con `ufw delete <número>`.
- `iptables -t raw -L PREROUTING -n -v`: muestra la regla raw y sus contadores `pkts`/`bytes`, que indican cuántos paquetes no autorizados ha descartado.

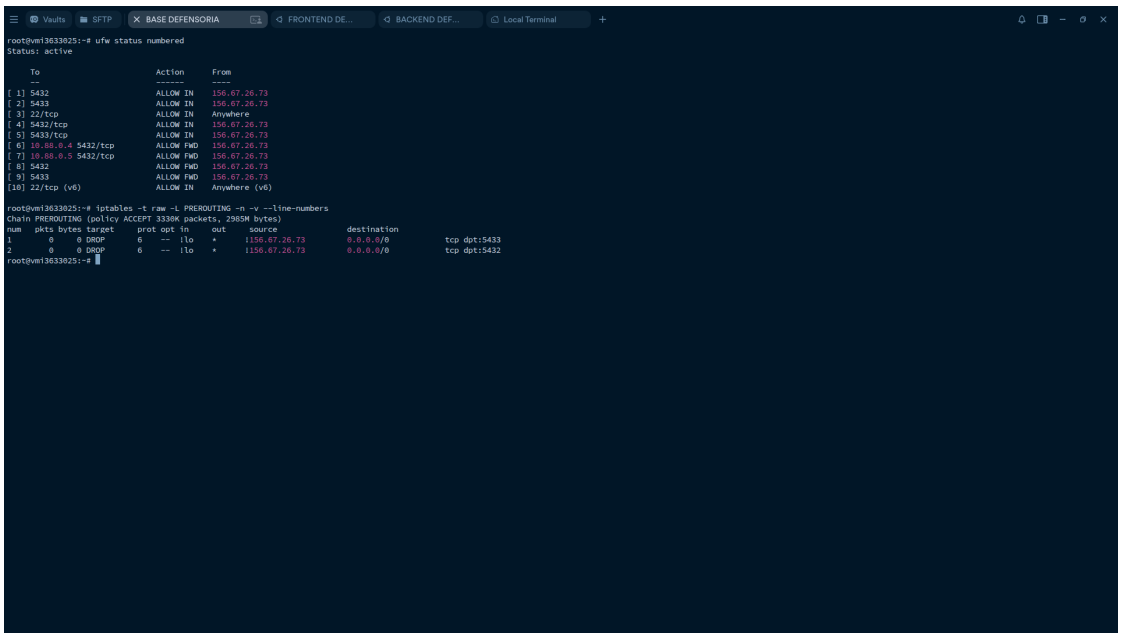


Figura 3.1: Reglas de ufw y regla raw del VPS Base de Datos

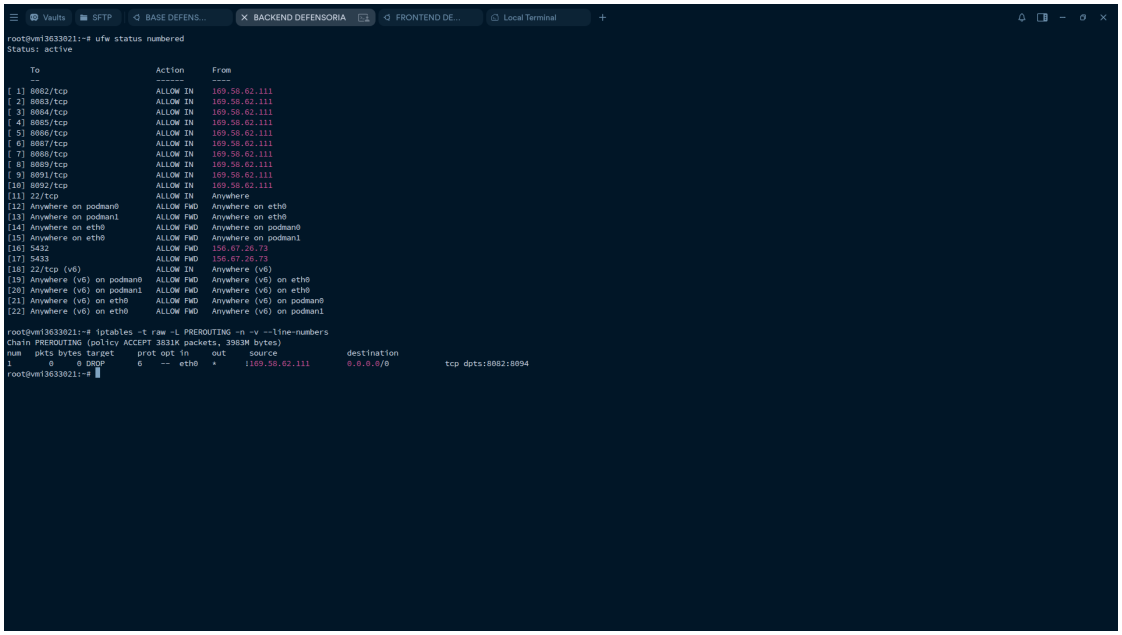


Figura 3.2: Reglas de ufw y regla raw del VPS Backend

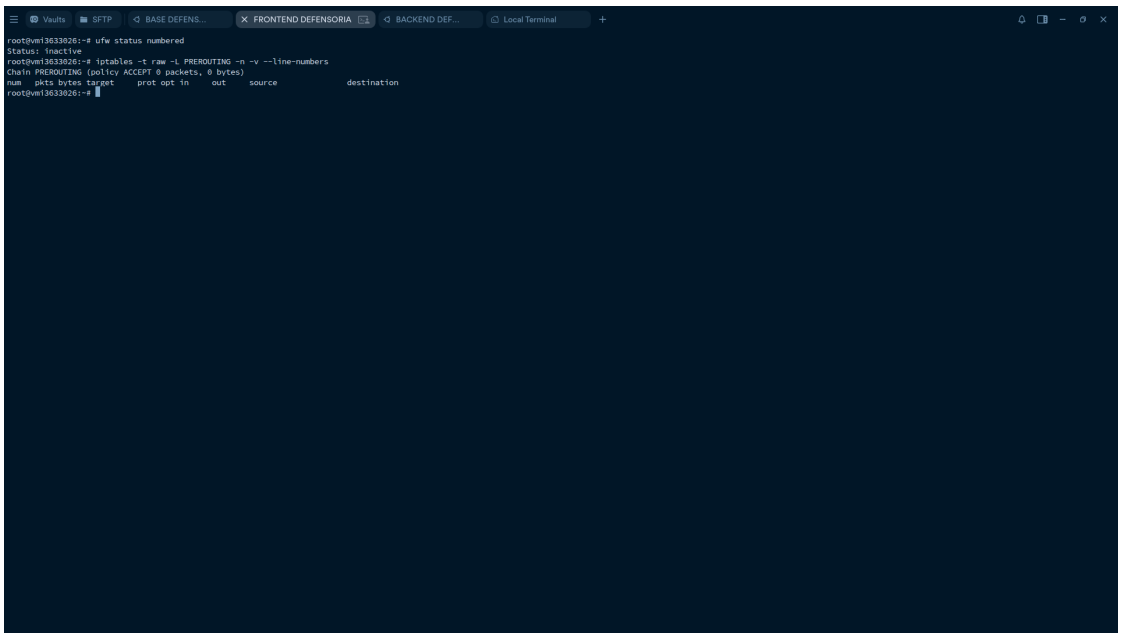


Figura 3.3: Estado del firewall del VPS Frontend

3.3. Diagnóstico de conectividad

Estas pruebas sirven para confirmar que el firewall funciona y para diagnosticar cuando un servicio no responde. Mientras no se hayan levantado los contenedores (capítulos 5 a 7), los puertos de la plataforma aparecerán cerrados en todos los casos.

3.3.1. Verificación de puertos y sockets (ss -tulpn, ufw status, iptables -t raw)

`ss` muestra qué puertos están escuchando en el servidor y qué proceso los atiende:

```
# ss -tulpn
```

Tabla 3.8: Puertos que deben aparecer en escucha con la plataforma instalada

VPS	**Puertos en escucha** (`LISTEN`)
Frontend	22 (sshd), 80 y 443 (nginx de router-nginx), 8090–8095 (frontends)
Backend	22 (sshd), 8082–8089, 8091, 8093 y 8094 (microservicios y modelo de IA)
Base de datos	22 (sshd), 5432 (defensoria-db)

Que un puerto aparezca en `LISTEN` solo indica que el servicio está levantado; quién puede llegar a él lo deciden ufw y la regla raw (sección 3.2).

ufw registra las conexiones que rechaza con el prefijo `[UFW BLOCK]`, útil para saber si el firewall está bloqueando algo que debería pasar:

```
# journalctl -k --since "1 hour ago" | grep "UFW BLOCK" | tail
```

3.3.2. Pruebas de conexión entre nodos y desde Internet

Con `nc` (incluido en Ubuntu) se prueba si un puerto remoto acepta la conexión; `-z` solo comprueba sin enviar datos, `-v` muestra el resultado y `-w 3` espera como máximo 3 segundos:

```
# nc -zv -w 3 169.58.62.99 5432      # desde el Backend: debe conectar
# nc -zv -w 3 156.67.26.73 8084      # desde el Frontend: debe conectar
```

Desde el equipo de administración (fuera de las VPS), los mismos puertos deben fallar por tiempo de espera:

```
$ nc -zv -w 3 169.58.62.99 5432      # debe fallar (timed out)
$ nc -zv -w 3 156.67.26.73 8084      # debe fallar (timed out)
$ nc -zv -w 3 169.58.62.111 443      # debe conectar
```

Tabla 3.9: Matriz de pruebas de conectividad (con la plataforma levantada)

Desde	Hacia	Puerto	Resultado esperado
Internet	Frontend 169.58.62.111	80 / 443	Conecta
Internet	Backend 156.67.26.73	8084	Falla
Internet	BD 169.58.62.99	5432	Falla
Frontend	Backend 156.67.26.73	8084	Conecta
Frontend	BD 169.58.62.99	5432	Falla
Backend	BD 169.58.62.99	5432	Conecta

Un resultado `succeeded` donde se espera `timed out` indica que falta una regla en esa VPS; uno `Connection refused` indica que el firewall deja pasar el tráfico pero el contenedor no está levantado.

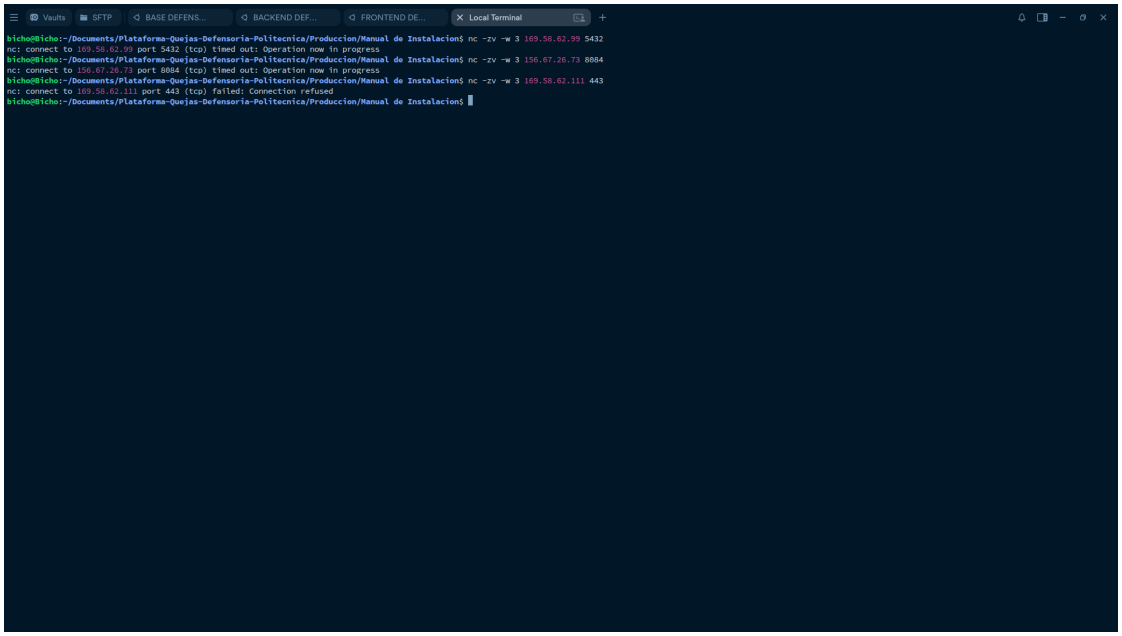


Figura 3.4: Pruebas de conexión entre nodos y desde Internet

4. Compilación Local, Empaquetado y Construcción de Contenedores

Este capítulo se realiza en el **equipo de desarrollo**, no en las VPS. Se descarga el código fuente desde GitHub, se revisa la configuración de producción, se compilan los microservicios y los frontends, se describen los Dockerfile con los que se construyen las imágenes y, por último, se transfieren los artefactos a cada VPS. Los comandos de este capítulo que empiezan con $ se ejecutan en el equipo de desarrollo; los que empiezan con #, en la VPS indicada.

4.1. Código fuente del proyecto

El código fuente de la plataforma se encuentra en GitHub, en la rama `Produccion` del repositorio:

https://github.com/JoseJimenez2004/Plataforma-Quejas-Defensoria-Politecnica

Todo lo que se instala está dentro de la carpeta `Produccion` de esa rama.

Descarga del repositorio. En el equipo de desarrollo se clona únicamente la rama de producción y se entra a la carpeta:

```
$ git clone --branch Produccion https://github.com/JoseJimenez2004/Plataforma-
    Quejas-Defensoria-Politecnica.git
$ cd Plataforma-Quejas-Defensoria-Politecnica/Produccion
$ git log -1 --format='%H %cd %s'
```

El último comando muestra el *hash* del commit descargado. Ese identificador se anota en la bitácora de la instalación, porque indica con exactitud qué versión del código se puso en producción. Si el repositorio ya estaba clonado, en lugar de clonar se actualiza con `git pull origin Produccion`.

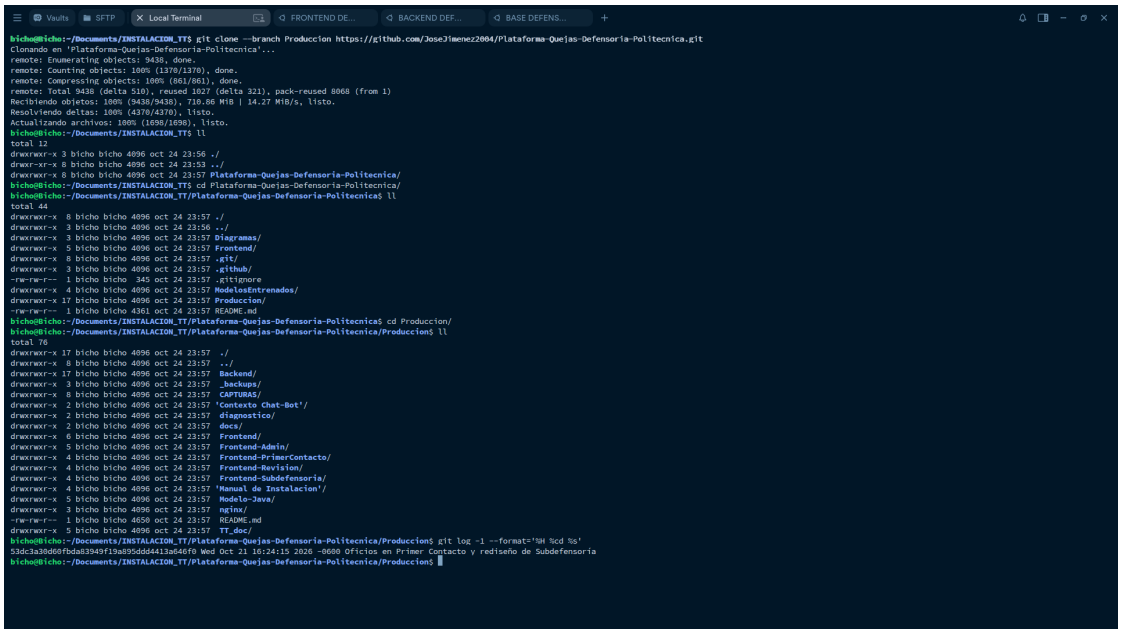


Figura 4.1: Clonado de la rama Produccion y hash del commit descargado

4.1.1. Estructura de la carpeta Produccion

Tabla 4.1: Contenido de la carpeta Produccion

Carpeta o archivo	Contenido
`Backend/auth-service`, `queja-service`, `notificaciones-service`, `catalogo-service`, `admin-service`, `revision-service`, `chatbot-service`, `denunciado-service`, `primercontacto`, `subdefensoria`	Código fuente de los 10 microservicios (proyectos Maven, cada uno con su `pom.xml` y su carpeta `src`)
`Backend/config-files`	Configuración de producción de cada microservicio (sección 4.1.2)
`Backend/database-scripts`	`init/00-init.sql` y los scripts semilla `seeds/01..05-*.sql`
`Backend/Dockerfile`, `Backend/admin-service/Dockerfile`	Dockerfile de los microservicios
`Backend/podman-compose.sh`, `podman-compose-bd.sh`, `firewall-backend.sh`	Scripts de despliegue del Backend y de la Base de Datos
`Backend/carga-masiva`	Scripts de carga masiva de quejas
`Modelo-Java`	Modelo de IA (`antecedentes-service`): código, corpus `Dataset/`, `pruebas/` y su script `podman-compose-modelo.sh`
`Frontend`, `Frontend-Admin`, `Frontend-Revision`, `Frontend-PrimerContacto`, `Frontend-Subdefensoria`	Los 5 proyectos Angular, cada uno con su `Dockerfile`, su configuración de nginx y su script `podman-compose-front*.sh`
`nginx/config/defensoria.conf`	Configuración de `router-nginx`
`docs`	Documentación técnica del proyecto

El repositorio solo contiene código fuente. Las carpetas de compilación (`target/` de Maven, `dist/` y `node_modules/` de Angular) están excluidas en `.gitignore` y se generan en la sección 4.2.

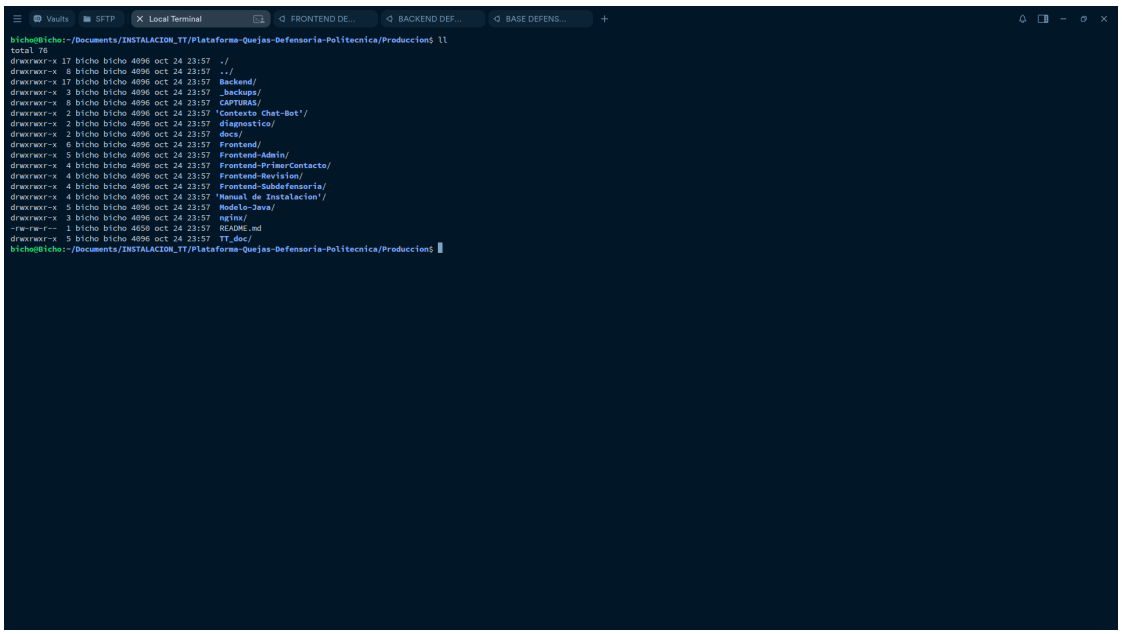


Figura 4.2: Estructura de la carpeta Produccion

4.1.2. Revisión de los archivos de configuración de producción (config-files)

Los JAR de los microservicios no llevan la configuración de producción. Cada servicio tiene un archivo YAML en `Backend/config-files/<servicio>/config/<servicio>.yml` que se copia al VPS Backend y se monta en el contenedor en `/app/config`; Spring Boot lo lee al arrancar gracias a las variables `SPRING_CONFIG_ADDITIONAL_LOCATION` y `SPRING_CONFIG_NAME` que define `podman-compose.sh`. De esta forma un mismo JAR sirve para cualquier ambiente y solo cambia su archivo de configuración.

Secciones de un config-file. Todos los archivos comparten la misma estructura base:

Tabla 4.2: Secciones comunes de los config-files

Sección	Para qué sirve
`server.port`	Puerto interno del contenedor. Siempre `8080`; el puerto externo (8082–8094) lo asigna `podman-compose.sh`.
`spring.datasource`	Conexión a PostgreSQL: URL `jdbc:postgresql://169.58.62.99:5432/defensoria_db`, usuario y contraseña.
`spring.jpa.hibernate.ddl-auto`	En `update`: al arrancar, el servicio crea o actualiza sus tablas.
`spring.servlet.multipart`	Tamaño máximo de archivos (evidencias y credenciales).
`spring.mail`	Servidor SMTP (Gmail) para enviar correos; solo en `auth-service`, `notificaciones-service` y `subdefensoria-service`.
`jwt.secret`	Secreto con el que se firman y validan los tokens de sesión. Debe ser **idéntico** en todos los servicios.
`<servicio>.url`	Direcciones de los otros microservicios a los que llama, por ejemplo `notificaciones.service.url: http://156.67.26.73:8085`.
`springdoc`	Ruta de la documentación Swagger (`/swagger-ui.html`).
`logging`	Formato de los mensajes del log.

Secciones particulares.

- `antecedentes-service`: sección `ia` con las rutas del corpus (`/app/data/*.json`), el umbral de similitud, el número de antecedentes y de oraciones del resumen. No usa base de datos.
- `admin-service`: secciones `respaldos` (carpeta `/app/respaldos`) y `db` (datos de conexión que usa `pg_dump` para los respaldos del Panel de la Defensora).
- `primer-contacto-service`: URLs de Subdefensoría, notificaciones y del modelo de IA; `cors`; y `historico.datasource.url`, que se deja vacía porque la base histórica ya no forma parte de la plataforma.
- `subdefensoria-service`: `plazos` y `correo` para los avisos de la Subdefensoría.
- `historico-service`: su carpeta existe en el repositorio pero no se instala.

Ejemplo: quejas-service.yml. Con los valores sensibles reemplazados por <...>:

```
server:
  port: 8080                      # puerto interno (afuera: 8084)
spring:
  application:
    name: quejas-service
  datasource:
    url: jdbc:postgresql://169.58.62.99:5432/defensoria_db
    username: postgres
    password: <CONTRASENA_BD>     # igual a POSTGRES_PASSWORD
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update            # crea/actualiza sus tablas al arrancar
  servlet:
    multipart:
      max-file-size: 30MB         # por archivo
      max-request-size: 100MB     # por queja completa
jwt:
  secret: <SECRETO_JWT>           # el mismo en todos los servicios
notificaciones:
  service:
    url: http://156.67.26.73:8085  # notificaciones-service
springdoc:
  swagger-ui:
    path: /swagger-ui.html
```

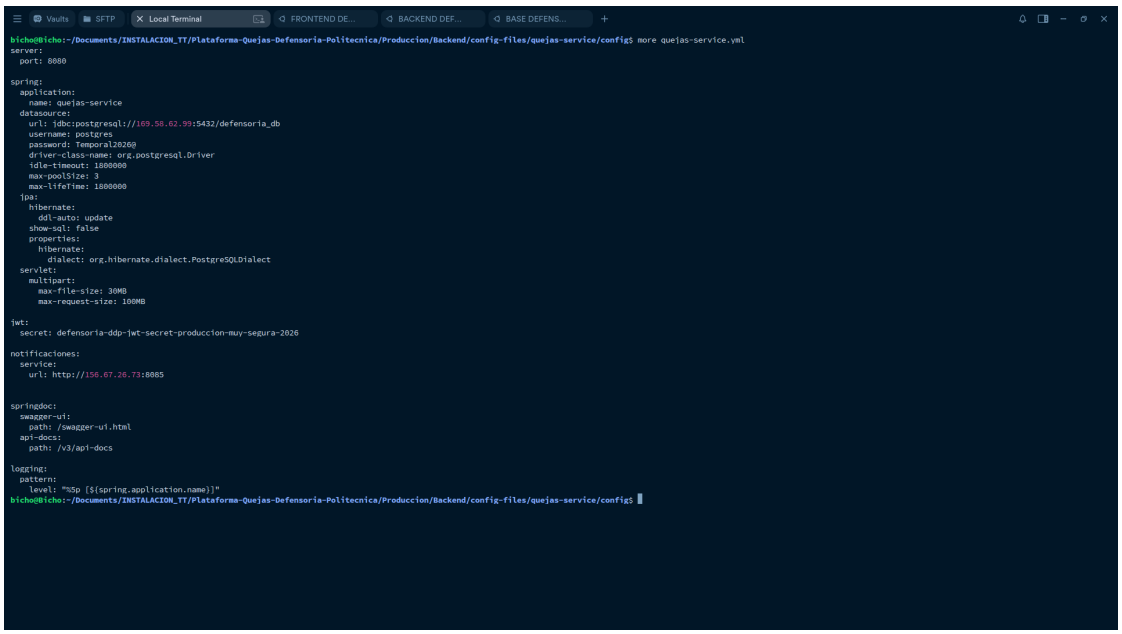


Figura 4.3: Contenido de un config-file de ejemplo (quejas-service.yml, con contraseña y secreto ocultos)

Validación antes de compilar. Se comprueba que todos los servicios apunten a la misma base de datos y compartan un único secreto JWT:

```
$ grep -rh "jdbc:postgresql" Backend/config-files | sort | uniq -c
$ grep -rh -A1 "^jwt:" Backend/config-files | grep secret | sort | uniq -c
```

Todas las URL deben ser `169.58.62.99:5432/defensoria_db` y debe aparecer un solo valor de `secret`.

Nota: Los config-files contienen la contraseña de la base de datos, la contraseña de aplicación de Gmail y el secreto JWT. En el repositorio público deben ir con valores de ejemplo; los valores reales se colocan solo en el VPS Backend, donde la carpeta `config-files` tiene permisos 750/640 (sección 2.3).

4.2. Compilación local

Tabla 4.3: Herramientas requeridas en el equipo de desarrollo

Herramienta	Versión	Comprobación
Java (JDK)	21	`java -version`
Maven	3.9 o superior	`mvn -v`
Node.js y npm	compatible con Angular 21	`node -v && npm -v`
Git	cualquiera reciente	`git --version`

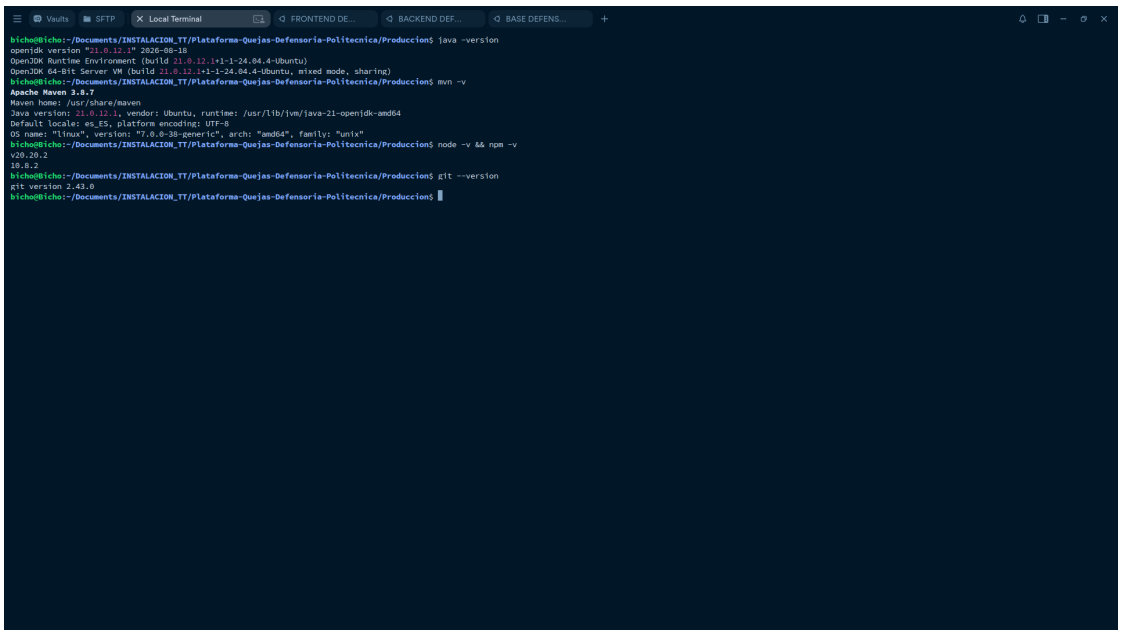


Figura 4.4: Versiones de Java, Maven, Node.js y npm en el equipo de desarrollo

4.2.1. Compilación de los 11 JAR con Maven

Cada microservicio es un proyecto Maven independiente. `mvn clean package` borra la compilación anterior, compila el código y empaqueta el servicio con todas sus dependencias en un solo JAR ejecutable dentro de `target/`. Se usa `-DskipTests` porque las pruebas se ejecutan en desarrollo, no al generar el artefacto de producción.

La compilación se hace en dos pasos, desde la carpeta `Produccion`:

1. Primero el modelo de IA (`Modelo-Java`) con `mvn clean install`, que además de generar su JAR lo registra en el repositorio local de Maven (`~/.m2`). Si falla, el proceso se detiene (`exit 1`).

2. Después, en un ciclo, los diez microservicios con `mvn clean package`. Si alguno falla, se muestra cuál fue y el ciclo se detiene (`break`) para no seguir compilando sobre un error.

```
# 1. Compilar e instalar el modelo de IA
echo "=== Compilando e instalando Modelo-Java ==="
(cd Modelo-Java && mvn -q clean install -DskipTests) || { echo "Error en Modelo-
    Java"; exit 1; }

# 2. Compilar y empaquetar cada microservicio dependiente
for p in auth-service queja-service notificaciones-service catalogo-service \
         admin-service revision-service chatbot-service denunciado-service \
         primercontacto subdefensoria; do
    echo "=== Compilando $p ==="
    (cd Backend/"$p" && mvn -q clean package -DskipTests) || { echo "Fallo al
    compilar $p"; break; }
done
```

La opción `-q` (*quiet*) oculta el detalle de Maven, así que en pantalla solo aparece el mensaje `=== Compilando ... ===` de cada proyecto y, si lo hay, el error.

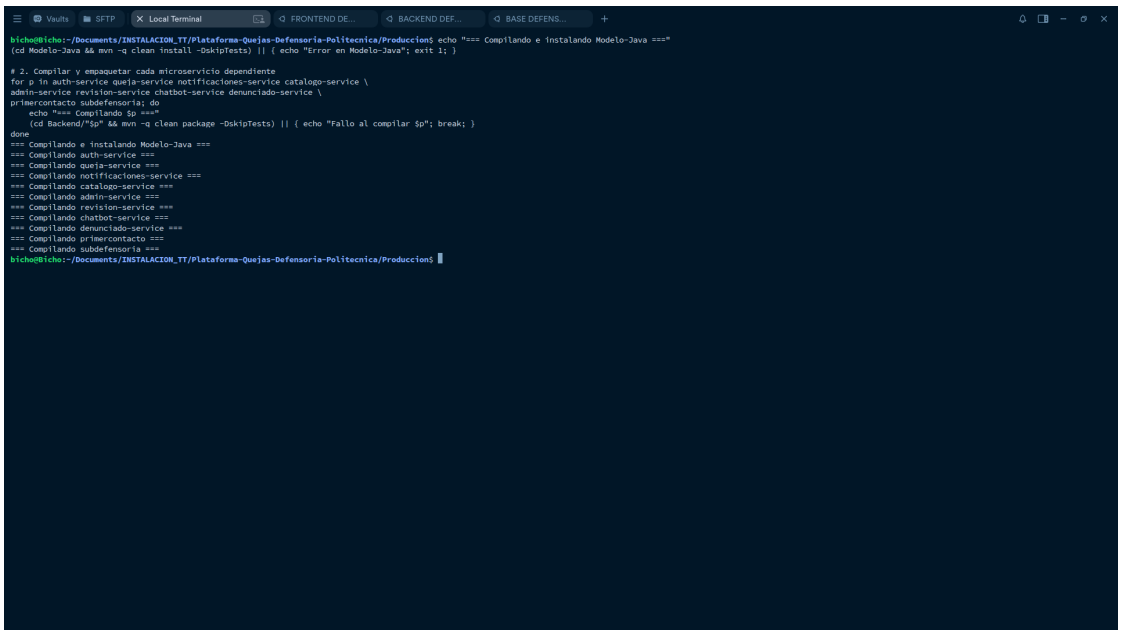


Figura 4.5: Compilación de los microservicios con Maven

Cada `pom.xml` fija el nombre final del JAR (`finalName`), así que sale con el nombre que esperan los scripts del servidor:

Tabla 4.4: JAR generados

Proyecto	JAR	Puerto
`Backend/primercontacto`	`primer-contacto-service.jar`	8082
`Backend/auth-service`	`auth-service.jar`	8083
`Backend/queja-service`	`quejas-service.jar`	8084
`Backend/notificaciones-service`	`notificaciones-service.jar`	8085
`Backend/catalogo-service`	`catalogo-service.jar`	8086
`Backend/admin-service`	`admin-service.jar`	8087
`Backend/revision-service`	`revision-service.jar`	8088
`Backend/chatbot-service`	`chatbot-service.jar`	8089
`Backend/subdefensoria`	`subdefensoria-service.jar`	8091
`Modelo-Java`	`antecedentes-service.jar`	8093
`Backend/denunciado-service`	`denunciado-service.jar`	8094

```
$ ls -lh Backend/*/target/*-service.jar Modelo-Java/target/antecedentes-service.
    jar
```

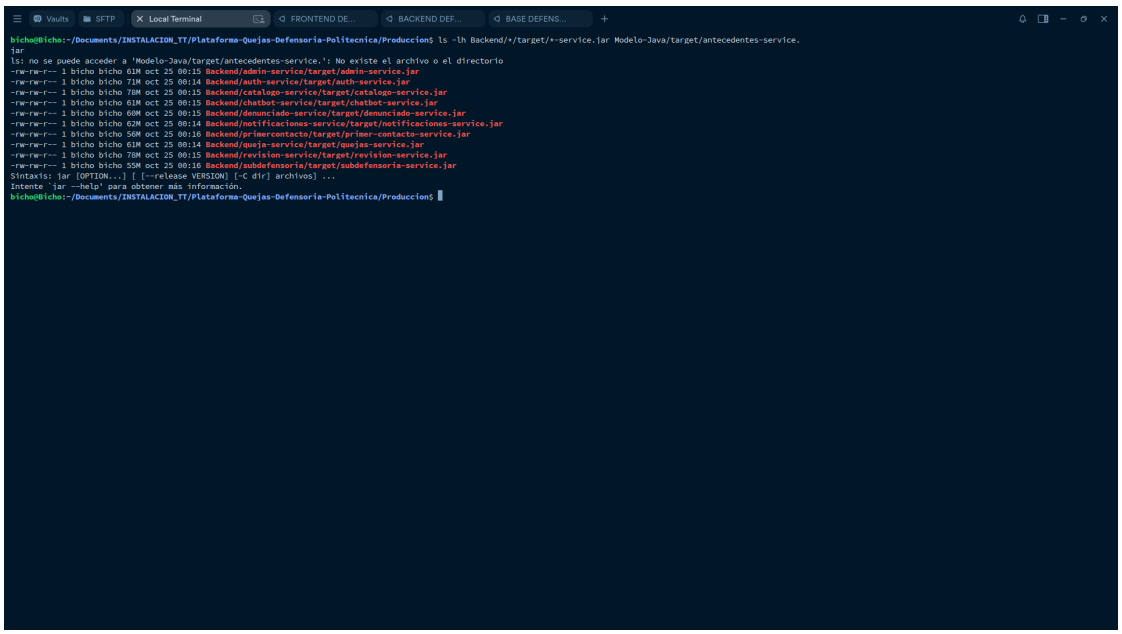


Figura 4.6: Los 11 JAR generados en las carpetas target

4.2.2. Compilación de los 5 frontends Angular (ng build y baseHref)

Cada frontend se compila a archivos estáticos (HTML, CSS y JavaScript). `npm ci` instala exactamente las versiones de dependencias registradas en `package-lock.json`, y `ng build --configuration production` genera la versión optimizada (código minificado y archivos con *hash* en el nombre) en `dist/<proyecto>/browser`. Los cinco se compilan en un ciclo; si uno falla, se muestra cuál y el ciclo se detiene:

```
for f in Frontend Frontend-Admin Frontend-Revision \
         Frontend-PrimerContacto Frontend-Subdefensoria; do
    echo "=== Mangrugi ti panag-build para iti: $f ==="
    (cd "$f" && npm ci && npx ng build --configuration production) || { echo "
   Napaay ti panag-build iti: $f"; break; }
done
```

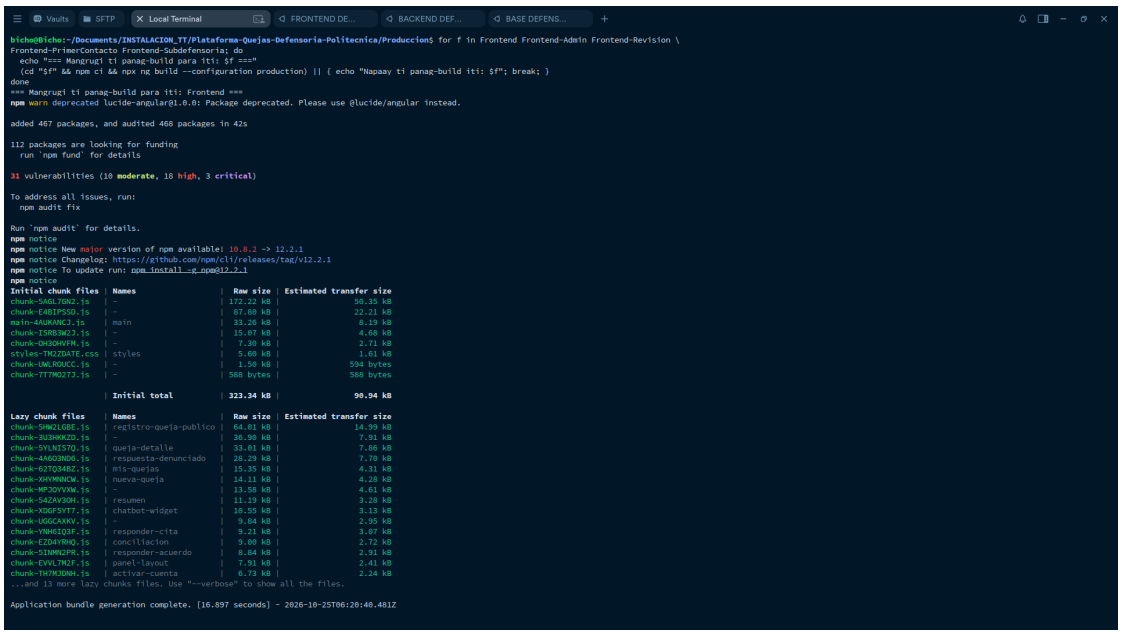


Figura 4.7: Compilación de los frontends con ng build

baseHref. Cuatro de los frontends no se sirven en la raíz del dominio sino en una subruta (`/admin/`, `/revision/`...). Cada `angular.json` define su `baseHref`, que se escribe en el `index.html` como `<base href="...">` y le indica al navegador desde dónde cargar los archivos. Debe coincidir con la ruta que le asigna `router-nginx`; si no, la página carga en blanco.

Tabla 4.5: Salida de compilación de los frontends

Proyecto	baseHref	Carpeta generada
`Frontend`	`/`	`dist/defensoria-front/browser`
`Frontend-Admin`	`/admin/`	`dist/defensoria-admin/browser`
`Frontend-Revision`	`/revision/`	`dist/defensoria-revision/browser`
`Frontend-PrimerContacto`	`/primer-contacto/`	`dist/frontend-primer-contacto/browser`
`Frontend-Subdefensoria`	`/subdefensoria/`	`dist/frontend-subdefensoria/browser`

```
$ grep -o '<base href="[^"]*"' Frontend*/dist/*/browser/index.html
```

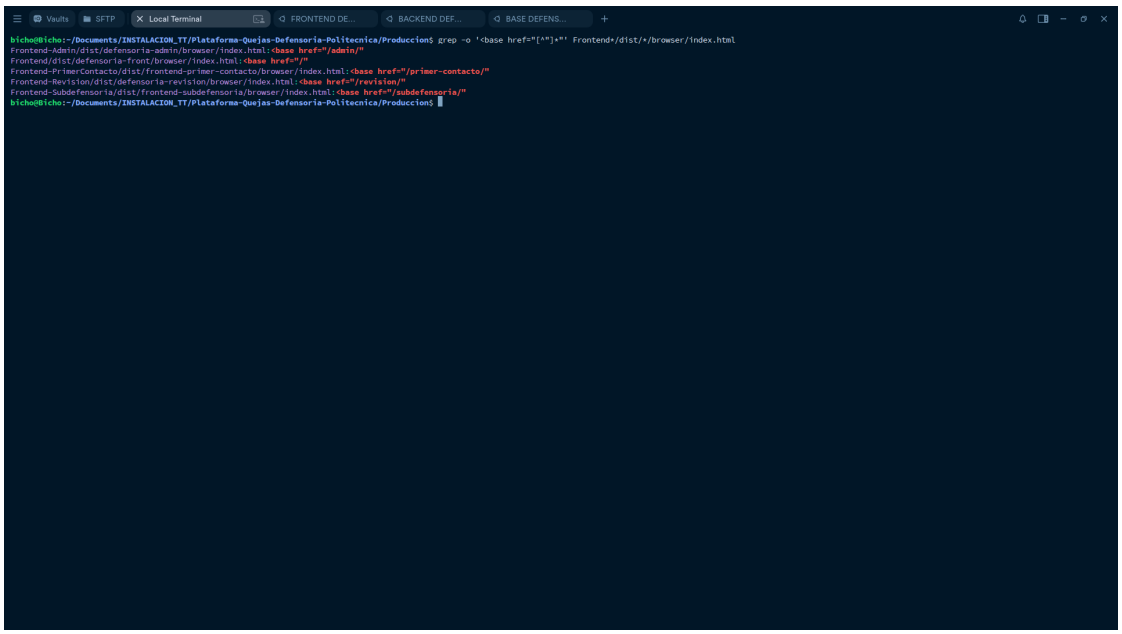


Figura 4.8: Carpetas dist generadas y base href de cada frontend

4.3. Construcción de imágenes con Podman

Una imagen es la plantilla a partir de la cual Podman crea un contenedor; se describe en un archivo `Dockerfile`. La plataforma usa tres Dockerfile. Las imágenes **no** se construyen en el equipo de desarrollo: se construyen en cada VPS con sus scripts (sección 4.3.4), a partir de los artefactos que se transfieren en la sección 4.4.

4.3.1. Dockerfile de los microservicios (eclipse-temurin:21-jre-jammy)

`Backend/Dockerfile` es común a todos los microservicios y al modelo de IA:

```
FROM docker.io/library/eclipse-temurin:21-jre-jammy
WORKDIR /app
ARG JAR_FILE
ARG SERVICE_PORT=8080
COPY ${JAR_FILE} app.jar
EXPOSE ${SERVICE_PORT}
ENTRYPOINT ["java", "-jar", "app.jar"]
```

- `FROM`: parte de una imagen oficial que solo trae el entorno de ejecución de Java 21 (JRE), más ligera que un JDK completo.
- `ARG JAR_FILE`: el JAR se recibe como parámetro al construir, por eso un solo Dockerfile sirve para los once servicios.
- `COPY`: copia el JAR dentro de la imagen como `/app/app.jar`.
- `ENTRYPOINT`: al arrancar el contenedor ejecuta `java -jar app.jar`.

4.3.2. Dockerfile de admin-service (cliente pg_dump 16)

`admin-service` genera y restaura los respaldos del Panel de la Defensora con `pg_dump` y `psql`, así que su imagen necesita además el cliente de PostgreSQL. Ubuntu Jammy (la base de `eclipse-temurin`) trae la versión 14 y `pg_dump` 14 se niega a respaldar un servidor PostgreSQL 16, por lo que `Backend/admin-service/Dockerfile` instala la versión 16 desde el repositorio oficial de PostgreSQL:

```
FROM docker.io/library/eclipse-temurin:21-jre-jammy
WORKDIR /app
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl ca-certificates \
 && (se agrega el repositorio apt.postgresql.org con su llave) \
 && apt-get install -y --no-install-recommends postgresql-client-16 \
 && rm -rf /var/lib/apt/lists/* && pg_dump --version
ARG JAR_FILE
COPY ${JAR_FILE} app.jar
RUN mkdir -p /app/respaldos
ENTRYPOINT ["java", "-jar", "app.jar"]
```

La carpeta `/app/respaldos` se monta como volumen (`/apps/utiles/respaldos`) para que los respaldos sobrevivan a la reconstrucción del contenedor.

4.3.3. Dockerfile de los frontends (nginx:alpine)

Los frontends se sirven con un nginx mínimo que solo entrega los archivos estáticos:

```
FROM docker.io/library/nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY dist/browser /usr/share/nginx/html
EXPOSE 80
```

La configuración interna de nginx de cada frontend contiene `try_files $uri $uri/ /index.html`, para que al recargar una página interna de Angular (por ejemplo `/admin/personal`) se devuelva `index.html` y Angular resuelva la ruta. El portal (`Frontend`) usa la misma estructura con su archivo `config/static.conf` y escucha en el 8090 en lugar del 80.

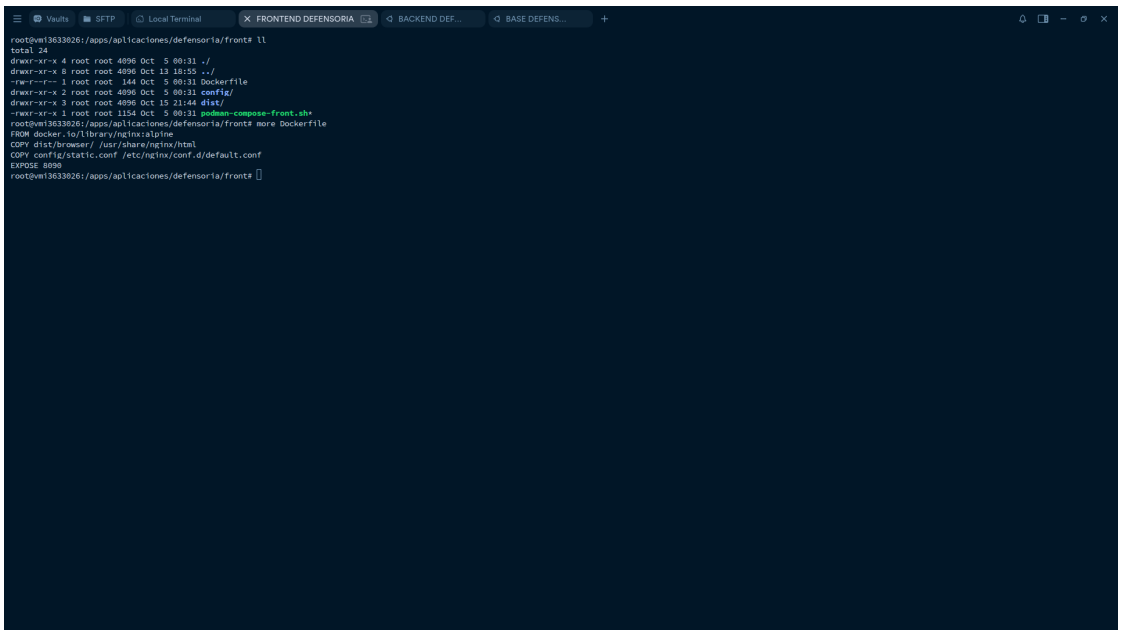


Figura 4.9: Contenido de los Dockerfile de microservicios y frontends

4.3.4. Construcción de las imágenes en el servidor con los scripts podman-compose

Cada script construye la imagen con `podman build` y crea el contenedor con `podman run`. Por ejemplo, para `quejas-service`, `podman-compose.sh` ejecuta:

```
podman build -f Dockerfile --build-arg JAR_FILE=artifact/quejas-service.jar \
             -t defensoria-quejas-service .
```

Tabla 4.6: Imágenes que construye cada script

VPS	Script	Imágenes
Backend	`podman-compose.sh`	`defensoria-<servicio>` (una por microservicio)
Backend	`podman-compose-modelo.sh`	`defensoria-antecedentes-service`
Frontend	`podman-compose-front*.sh`	`defensoria-front-img`, `defensoria-admin-img`, `defensoria-revision-img`, `defensoria-primer-contacto-img`, `defensoria-subdefensoria-img`
Base de datos	`podman-compose-bd.sh`	No construye: usa directamente `postgres:16-alpine`

La ejecución de cada script se describe en los capítulos 5, 6 y 7.

4.4. Transferencia de artefactos a los VPS

4.4.1. Copia segura mediante scp hacia cada servidor de destino

Los artefactos se copian con `scp`, que transfiere los archivos cifrados por SSH. Cada comando pide la contraseña de `root` de la VPS de destino. También se puede usar el explorador **SFTP**

de Termius arrastrando los archivos a las mismas rutas. Antes de copiar, las carpetas de destino ya deben existir (sección 2.3).

VPS Base de Datos (169.58.62.99).

```
$ D=root@169.58.62.99:/apps/aplicaciones/defensoria
$ scp Backend/podman-compose-bd.sh            $D/database/
$ scp Backend/database-scripts/init/*.sql     $D/database-scripts/init/
$ scp Backend/database-scripts/seeds/*.sql    $D/database-scripts/seeds/
```

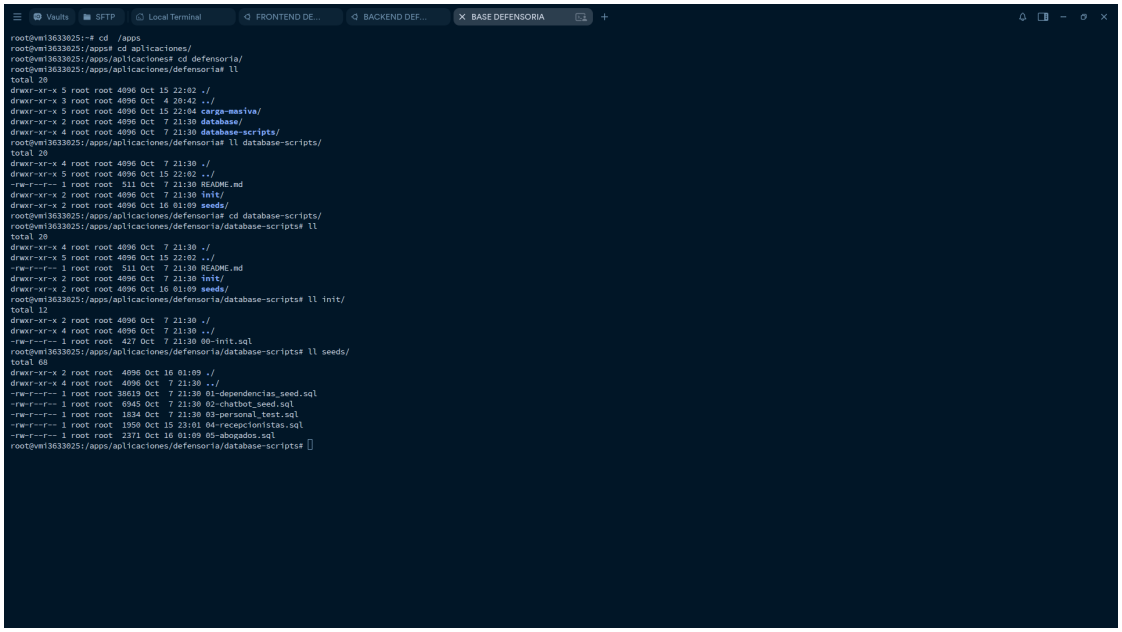


Figura 4.10: Transferencia de archivos al VPS Base de Datos

VPS Backend (156.67.26.73).

```
$ B=root@156.67.26.73:/apps/aplicaciones/defensoria/back
# JAR
$ scp Backend/{auth-service,queja-service,notificaciones-service,catalogo-
    service}/target/*-service.jar \
      Backend/{admin-service,revision-service,chatbot-service,denunciado-service
    }/target/*-service.jar \
      Backend/{primercontacto,subdefensoria}/target/*-service.jar \
      Modelo-Java/target/antecedentes-service.jar   $B/artifact/
# Configuracion (sin historico-service)
$ scp -r Backend/config-files/{auth-service,quejas-service,notificaciones-
    service,catalogo-service} \
         Backend/config-files/{admin-service,revision-service,chatbot-service,
    denunciado-service} \
         Backend/config-files/{primer-contacto-service,subdefensoria-service,
    antecedentes-service} \
         $B/config-files/
# Dockerfile y scripts
$ scp Backend/Dockerfile Backend/podman-compose.sh Backend/firewall-backend.sh \
      Modelo-Java/podman-compose-modelo.sh   $B/
$ scp Backend/admin-service/Dockerfile       $B/admin-service/
# Modelo de IA y carga masiva
$ scp Modelo-Java/Dataset/quejas_sinteticas.json Modelo-Java/Dataset/
    quejas_prueba_30.json \
```

```
      $B/modelo/Dataset/
$ scp Modelo-Java/pruebas/*.json              $B/modelo/pruebas/
$ scp -r Backend/carga-masiva/cargar-quejas.sh Backend/carga-masiva/generar-csv.
    py \
        Backend/carga-masiva/quejas-ejemplo.csv Backend/carga-masiva/
    descripciones \
        $B/carga-masiva/
```

Nota: `quejas_aprendidas.json` no se copia: lo genera el propio modelo de IA al aprender quejas nuevas y, en un servidor en operación, sobrescribirlo borraría lo aprendido.

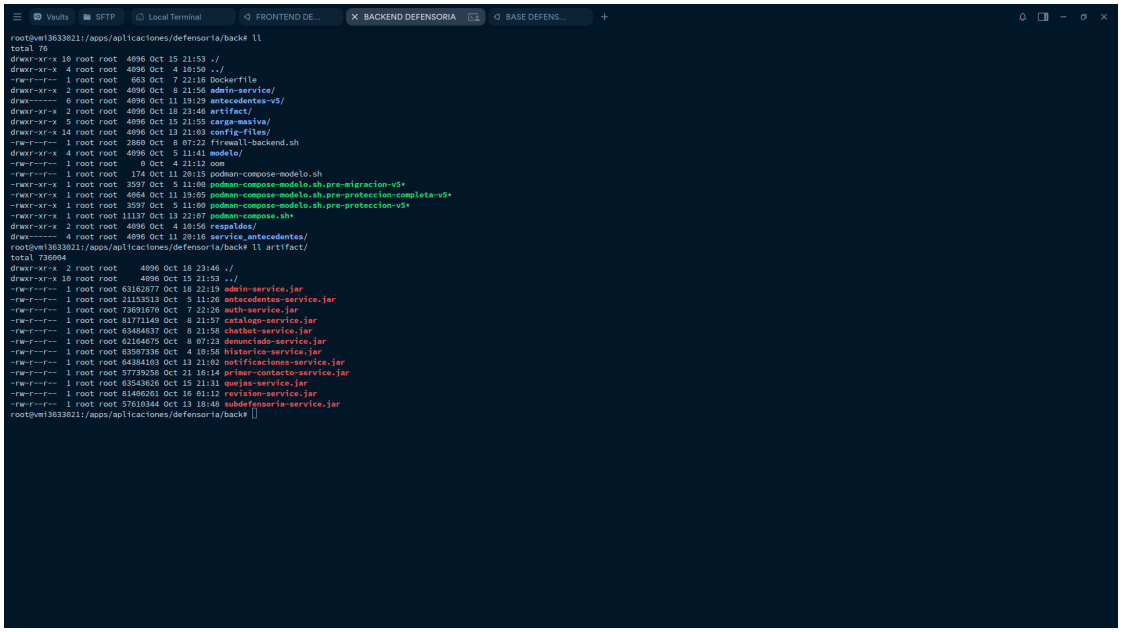


Figura 4.11: Transferencia de archivos al VPS Backend

VPS Frontend (169.58.62.111). Para cada frontend se copian su Dockerfile, su configuración de nginx, su script y el contenido de `dist/.../browser`. Si es una actualización, primero se vacía la carpeta `dist/browser` del servidor para no dejar archivos de la versión anterior:

```
$ ssh root@169.58.62.111 'rm -rf /apps/aplicaciones/defensoria/*/dist/browser/*'
$ F=root@169.58.62.111:/apps/aplicaciones/defensoria
# Portal (defensoria-web)
$ scp Frontend/Dockerfile Frontend/podman-compose-front.sh      $F/front/
$ scp Frontend/config/static.conf                              $F/front/config/
$ scp -r Frontend/dist/defensoria-front/browser/*              $F/front/dist/
    browser/
# Panel de la Defensora (admin-web)
$ scp Frontend-Admin/{Dockerfile,nginx.conf,podman-compose-front-admin.sh}   $F/
    front-admin/
$ scp -r Frontend-Admin/dist/defensoria-admin/browser/*        $F/front-admin/
    dist/browser/
# Revision (revision-web)
$ scp Frontend-Revision/{Dockerfile,nginx.conf,podman-compose-front-revision.sh}
      $F/front-revision/
$ scp -r Frontend-Revision/dist/defensoria-revision/browser/*   $F/front-
    revision/dist/browser/
```

```
# Primer Contacto (primer-contacto-web)
$ scp Frontend-PrimerContacto/{Dockerfile,nginx.conf,podman-compose-front-primer
    -contacto.sh} \
      $F/front-primer-contacto/
$ scp -r Frontend-PrimerContacto/dist/frontend-primer-contacto/browser/* \
      $F/front-primer-contacto/dist/browser/
# Subdefensoria (subdefensoria-web)
$ scp Frontend-Subdefensoria/{Dockerfile,nginx.conf,podman-compose-front-
    subdefensoria.sh} \
      $F/front-subdefensoria/
$ scp -r Frontend-Subdefensoria/dist/frontend-subdefensoria/browser/* \
      $F/front-subdefensoria/dist/browser/
# router-nginx
$ scp nginx/config/defensoria.conf                                  $F/router/config
    /
```

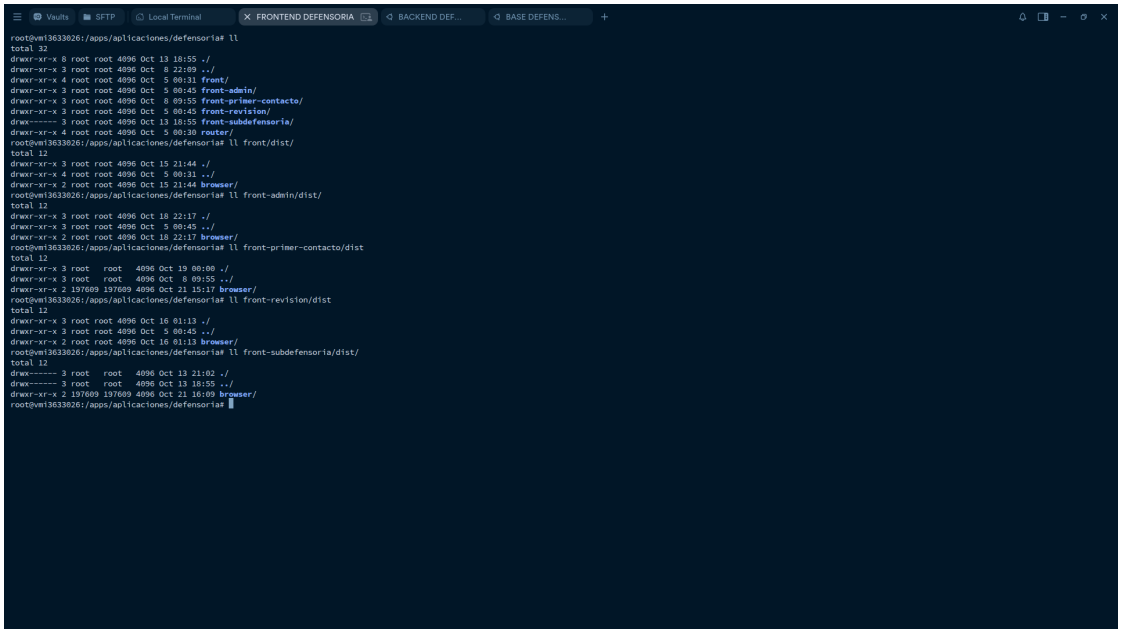


Figura 4.12: Transferencia de archivos al VPS Frontend

Permisos de los archivos copiados. Después de copiar se aplica la política de permisos de la sección 2.3:

```
# chmod 700 /apps/aplicaciones/defensoria/database/podman-compose-bd.sh        #
    BD
# chmod 644 /apps/aplicaciones/defensoria/database-scripts/*/*.sql             #
    BD

# B=/apps/aplicaciones/defensoria/back                                         #
    Backend
# chmod 750 $B/*.sh $B/carga-masiva/*.sh
# chmod 644 $B/artifact/*.jar $B/Dockerfile $B/admin-service/Dockerfile $B/
    modelo/*/*.json
# find $B/config-files -type d -exec chmod 750 {} + ; find $B/config-files -type
    f -exec chmod 640 {} +
```

```
# F=/apps/aplicaciones/defensoria                                          #
    Frontend
# find $F -type d -exec chmod 755 {} + ; find $F -type f -exec chmod 644 {} +
# chmod 750 $F/*/podman-compose-front*.sh
```

5. Despliegue del Nodo de Base de Datos (VPS 169.58.62.99)

La base de datos es la primera capa que se levanta, porque los microservicios necesitan conectarse a ella al arrancar. Todos los comandos de este capítulo se ejecutan en el VPS Base de Datos, en la carpeta `/apps/aplicaciones/defensoria/database`, donde quedó el script `podman-compose-bd.sh` (sección 4.4).

5.1. Imagen de PostgreSQL

5.1.1. Descarga de la imagen oficial postgres:16-alpine

La base de datos no necesita una imagen propia: se usa la imagen oficial de PostgreSQL 16 publicada en Docker Hub, en su variante `alpine`, que es más pequeña porque está construida sobre Alpine Linux. La imagen ya se descargó al preparar la VPS (sección 2.2); si no, el script la descarga automáticamente la primera vez. Se comprueba que esté disponible y su versión exacta:

```
# podman images docker.io/library/postgres
# podman run --rm docker.io/library/postgres:16-alpine postgres --version
```

El segundo comando crea un contenedor temporal (`--rm` lo borra al terminar) solo para imprimir la versión, que debe ser `postgres (PostgreSQL) 16.x`.

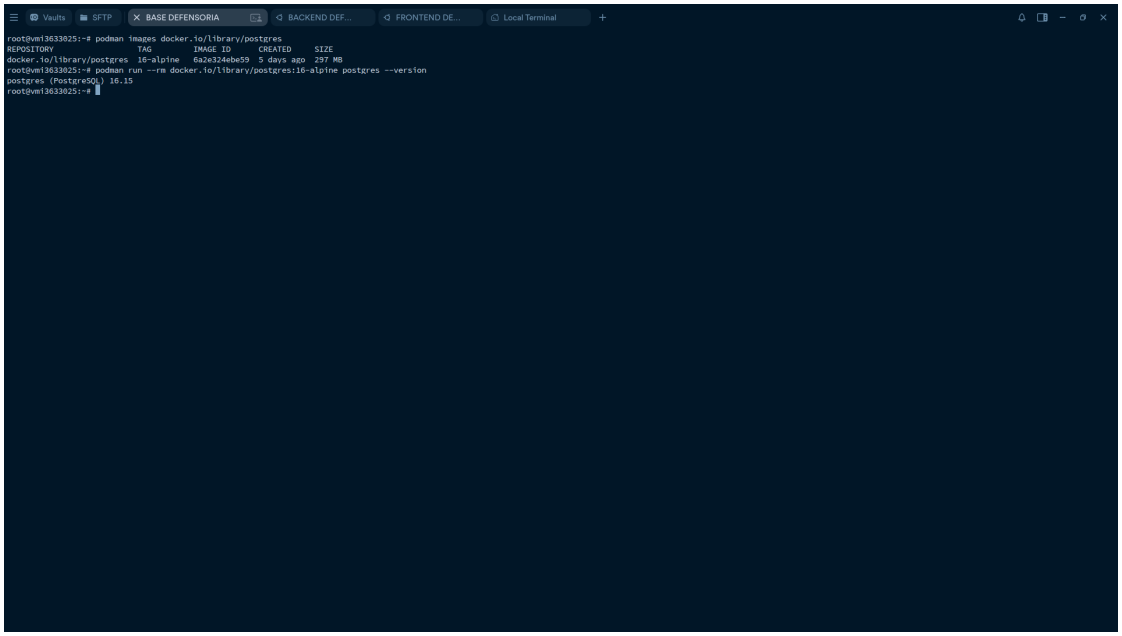


Figura 5.1: Imagen postgres:16-alpine disponible y versión de PostgreSQL

5.2. Volúmenes persistentes y variables

5.2.1. Volumen de datos en /apps/data/postgresql/defensoria_db

Un contenedor se puede borrar y volver a crear en cualquier momento; si los datos vivieran dentro de él, se perderían. Por eso la carpeta donde PostgreSQL guarda la base (`/var/lib/postgresql/data` dentro del contenedor) se monta desde el disco de la VPS:

```
-v /apps/data/postgresql/defensoria_db:/var/lib/postgresql/data:Z
```

- La carpeta del host la crea el script la primera vez; al iniciar, PostgreSQL le asigna su propio usuario (`postgres`, UID 70), por eso no se modifica a mano (sección 2.3).
- Detener, borrar o recrear el contenedor (`down` / `up-defensoria`) **conserva** todos los datos.
- Solo el comando `reset` del script borra esta carpeta, y pide escribir `BORRAR` para confirmarlo.
- La opción `:Z` ajusta la etiqueta de seguridad del volumen para sistemas con SELinux; en Ubuntu no tiene efecto, pero se conserva para que el script funcione igual en otras distribuciones.

5.2.2. Script de inicialización 00-init.sql y scripts semilla

La carpeta `/apps/aplicaciones/defensoria/database-scripts` tiene dos subcarpetas con propósitos distintos:

Tabla 5.1: Scripts SQL de la base de datos

Carpeta	Archivos	Cuándo se ejecutan
`init/`	`00-init.sql`	Solo una vez, automáticamente, cuando el contenedor se crea con la carpeta de datos vacía. Se monta en `/docker-entrypoint-initdb.d` (solo lectura).
`seeds/`	`01-dependencias_seed.sql`, `02-chatbot_seed.sql`, `03-personal_test.sql`, `04-recepcionistas.sql`, `05-abogados.sql`	A mano, después de levantar los microservicios, porque necesitan que las tablas ya existan (sección 6.5).

`00-init.sql` solo configura la base; no crea tablas, porque las crea cada microservicio al arrancar (Hibernate con `ddl-auto: update`):

```
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
ALTER DATABASE defensoria_db SET timezone TO 'America/Mexico_City';
```

- `uuid-ossp`: extensión que genera identificadores únicos (UUID) que usan algunas tablas.
- `timezone`: todas las fechas que guarda la base quedan en hora del centro de México.

```
# ls -l /apps/aplicaciones/defensoria/database-scripts/*
# cat /apps/aplicaciones/defensoria/database-scripts/init/00-init.sql
```

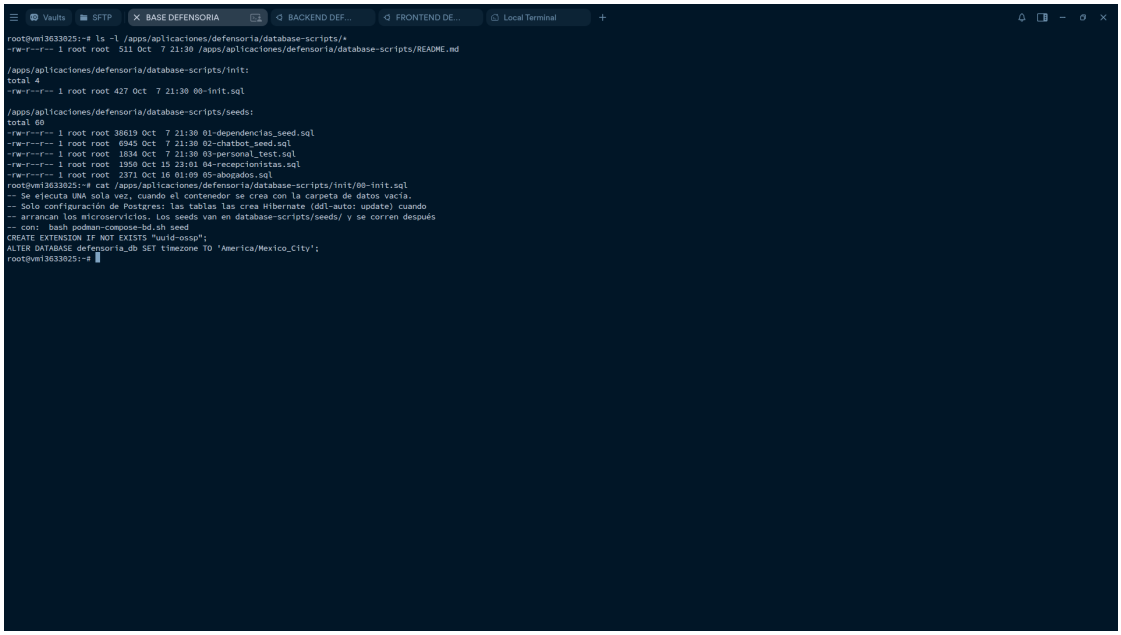


Figura 5.2: Scripts de inicialización y semilla en el VPS Base de Datos

5.3. Creación y ejecución del contenedor de BD

5.3.1. El script podman-compose-bd.sh

`podman-compose-bd.sh` reúne en un solo archivo todas las operaciones sobre la base de datos. Se ejecuta como `root` con `bash podman-compose-bd.sh <comando>`; sin comando muestra la ayuda.

Tabla 5.2: Comandos de podman-compose-bd.sh usados en la plataforma

Comando	Qué hace
`up-defensoria`	Crea (o recrea) el contenedor `defensoria-db` y espera a que acepte conexiones
`status`	Muestra el estado del contenedor y sus puertos
`logs defensoria-db`	Muestra el log de PostgreSQL en vivo
`psql-defensoria`	Abre la consola SQL dentro de la base `defensoria_db`
`seed`	Carga los datos semilla (sección 6.5)
`backup / restore <archivo>`	Respaldo de la base en `/apps/respaldos/bd` y restauración a partir de un respaldo
`firewall / firewall-boot`	Aplican la regla raw que limita el puerto 5432 al Backend y la hacen permanente (sección 5.4)
`down`	Elimina el contenedor conservando los datos
`reset`	Elimina contenedor **y datos** y crea la base vacía (pide confirmación)

Nota: El script también contiene los comandos up, up-historico y psql-historico para la base histórica (historico-db, puerto 5433). Esa base ya no forma parte de la plataforma, por lo que se usa up-defensoria en lugar de up.

5.3.2. Creación de defensoria-db (puerto 5432, límite de memoria)

```
# cd /apps/aplicaciones/defensoria/database
# bash podman-compose-bd.sh up-defensoria
```

Internamente el script ejecuta:

```
podman run -d \
  --name defensoria-db \
  --restart always \
  -p 5432:5432 \
  -m 1g \
  -e TZ=America/Mexico_City \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=<CONTRASENA_BD> \
  -e POSTGRES_DB=defensoria_db \
  -v /apps/data/postgresql/defensoria_db:/var/lib/postgresql/data:Z \
  -v /apps/aplicaciones/defensoria/database-scripts/init:/docker-entrypoint-
    initdb.d:ro,Z \
  docker.io/library/postgres:16-alpine
```

Tabla 5.3: Parámetros del contenedor defensoria-db

Parámetro	Significado
--name defensoria-db	Nombre fijo con el que se administra el contenedor
--restart always	Se reinicia si se detiene y al encender la VPS (sección 5.4)
-p 5432:5432	Publica el puerto de PostgreSQL en la VPS; el firewall lo limita al Backend
-m 1g	Límite de 1 GB de memoria para el contenedor
-e TZ=...	Zona horaria del contenedor (fechas de los logs)
-e POSTGRES_USER, _PASSWORD, _DB	Usuario, contraseña y base que PostgreSQL crea la primera vez
-v.../defensoria_db:...	Volumen persistente de datos (sección 5.2)
-v.../init: /docker-entrypoint-initdb.d:ro	Scripts que se ejecutan al crear la base, montados en solo lectura

El script espera hasta 60 segundos a que PostgreSQL acepte conexiones (pg_isready) y muestra defensoria-db ->:5432 / defensoria_db. Después se revisa el estado:

```
# bash podman-compose-bd.sh status
NAMES          STATUS         PORTS
defensoria-db  Up 2 minutes   0.0.0.0:5432->5432/tcp
```

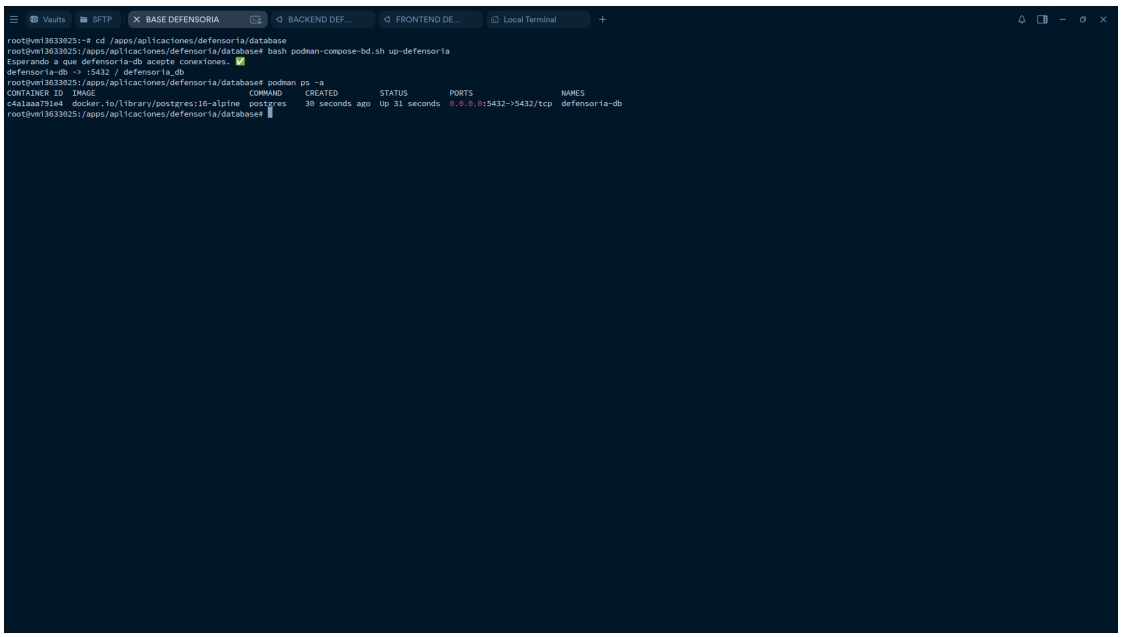


Figura 5.3: Creación de defensoria-db y estado del contenedor

5.3.3. Verificación de la base defensoria_db

Se abre la consola SQL y se comprueba que la base, la extensión y la zona horaria estén configuradas:

```
# bash podman-compose-bd.sh psql-defensoria
defensoria_db=# \l                  -- lista de bases: debe aparecer
    defensoria_db
defensoria_db=# \dx                 -- extensiones: debe aparecer uuid-ossp
defensoria_db=# SHOW timezone;      -- America/Mexico_City
defensoria_db=# \q
```

En este momento la base está vacía (`\dt` no muestra tablas); las tablas aparecen cuando arrancan los microservicios en el capítulo 6.

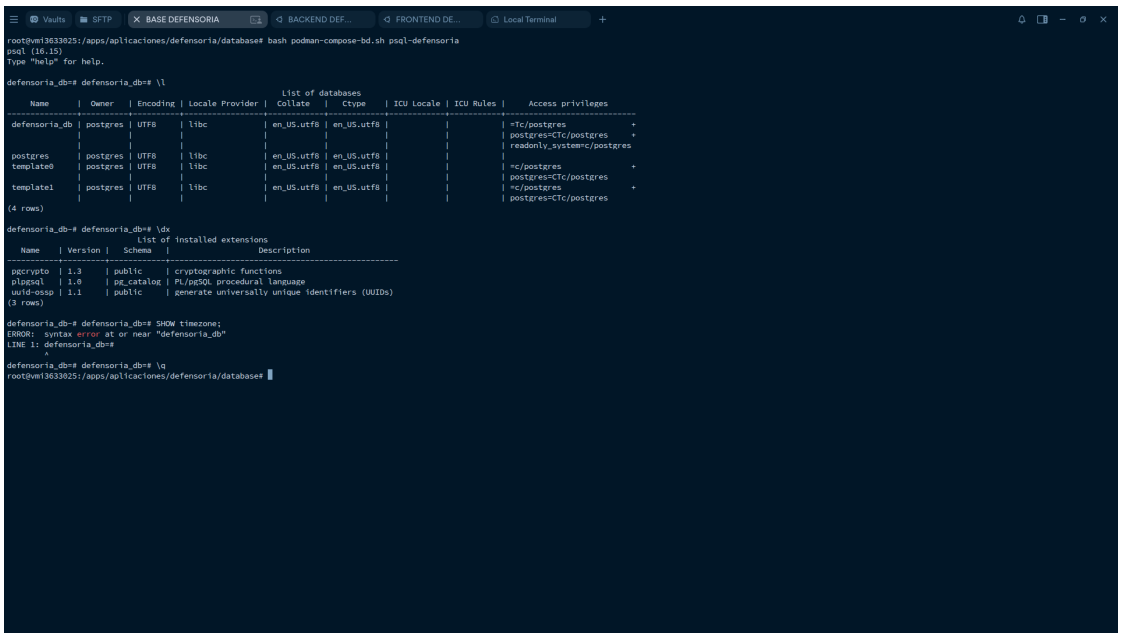


Figura 5.4: Base defensoria_db, extensión uuid-ossp y zona horaria

Por último se confirma que el puerto está escuchando y que el firewall del capítulo 3 lo protege: ufw debe permitir el 5432 solo desde 156.67.26.73, como entrada (`ALLOW IN`) y como reenvío al contenedor (`ALLOW FWD`):

```
# ss -tlnp | grep 5432
# ufw status numbered
# iptables -t raw -L PREROUTING -n -v --line-numbers
```

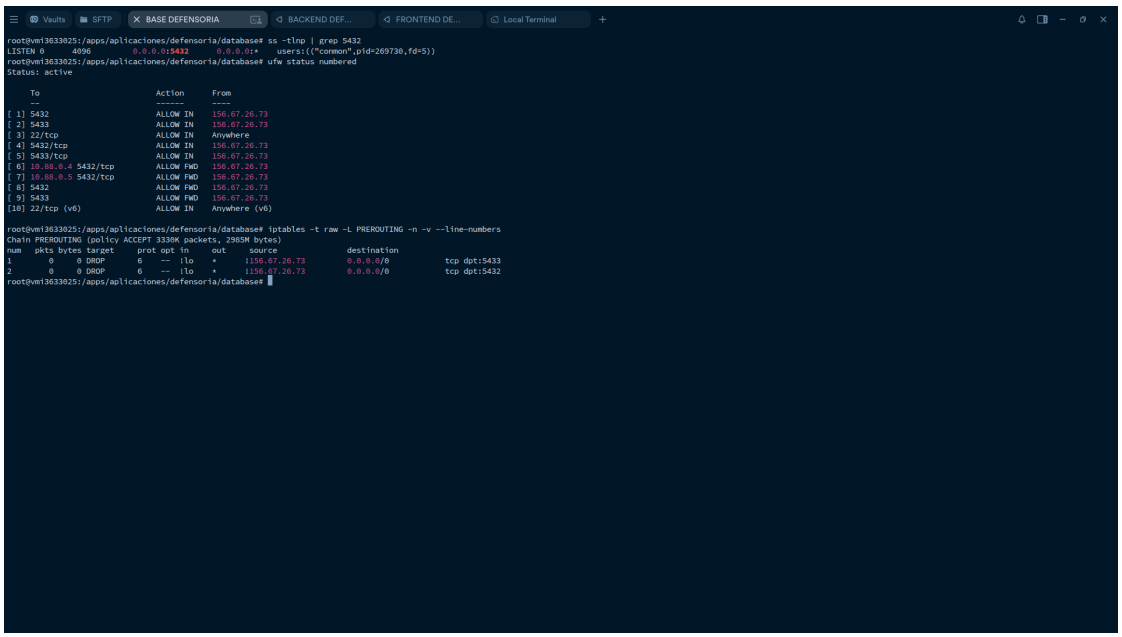


Figura 5.5: Puerto 5432 en escucha y protegido por la regla de firewall

5.4. Arranque automático y respaldos

5.4.1. Política --restart always y podman-restart.service

`defensoria-db` se crea con `--restart always`: si PostgreSQL termina inesperadamente, Podman lo vuelve a iniciar, y al encender la VPS lo levanta el servicio `podman-restart` habilitado en la sección 2.2. Se comprueban ambos:

```
# podman inspect --format '{{.HostConfig.RestartPolicy.Name}}' defensoria-db   #
    always
# systemctl is-enabled podman-restart.service                                  #
    enabled
```

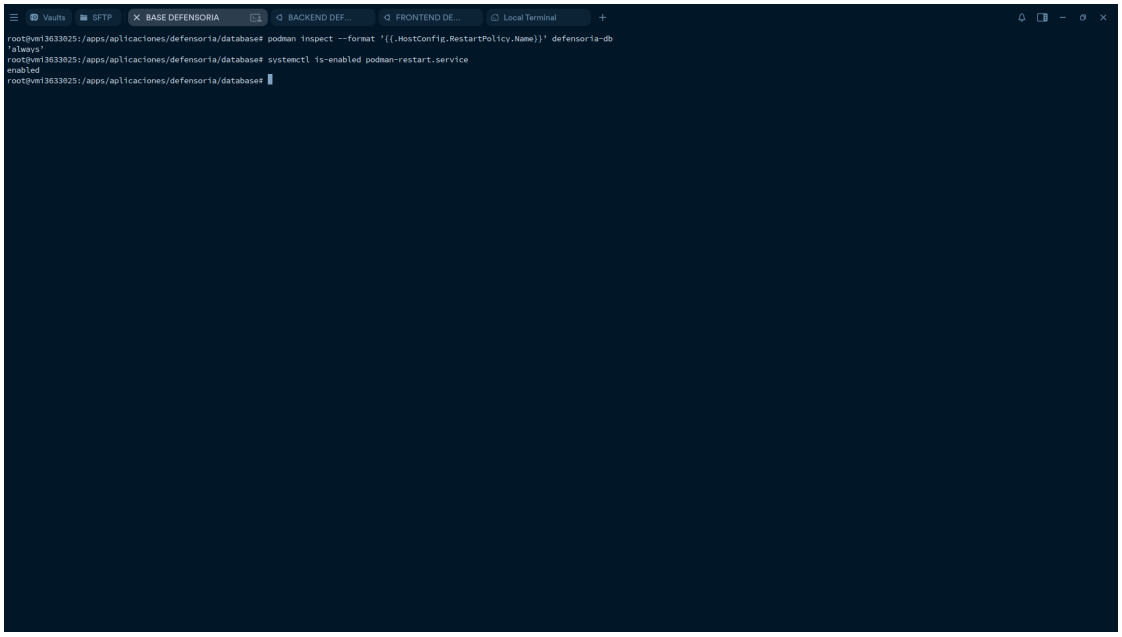


Figura 5.6: Política de reinicio de defensoria-db y servicio podman-restart

5.4.2. Servicio systemd del firewall (firewall-boot)

El VPS Base de Datos queda protegido por las dos capas descritas en el capítulo 3: ufw y la regla raw. ufw recarga sus reglas solo en cada arranque (`ufw.service`); la regla raw, en cambio, se pierde al reiniciar si no se instala su servicio. El script de la base la aplica y la deja permanente:

```
# cd /apps/aplicaciones/defensoria/database
# bash podman-compose-bd.sh firewall
# bash podman-compose-bd.sh firewall-boot
```

- `firewall`: detecta la interfaz pública y agrega en la tabla raw la regla que descarta todo lo que llegue a los puertos 5432 y 5433 desde una IP distinta de 156.67.26.73. Antes de agregarla comprueba si ya existe, así que se puede ejecutar varias veces sin duplicarla.
- `firewall-boot`: crea el servicio `/etc/systemd/system/defensoria-bd-firewall.service`, que ejecuta `podman-compose-bd.sh firewall` en cada arranque, y lo habilita.

Se comprueba que ambos servicios estén habilitados y que la regla esté cargada:

```
# systemctl is-enabled ufw defensoria-bd-firewall
# systemctl status defensoria-bd-firewall --no-pager
# iptables -t raw -L PREROUTING -n -v --line-numbers
```

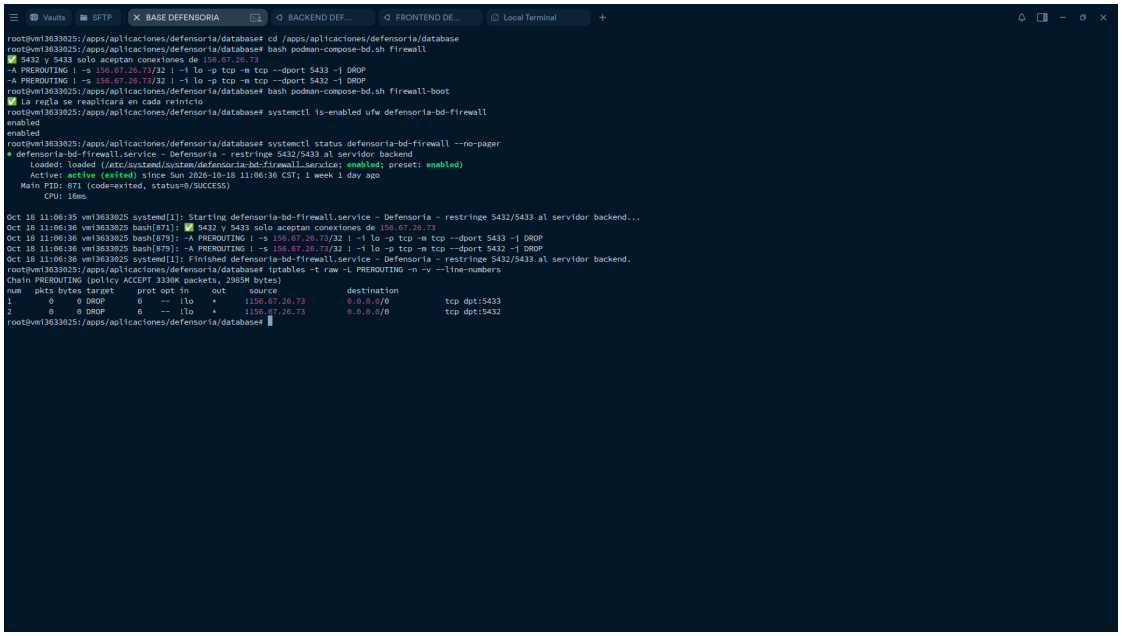


Figura 5.7: Servicios ufw y defensoria-bd-firewall habilitados y regla raw cargada

6. Despliegue del Nodo de Backend (VPS 156.67.26.73)

Con la base de datos en funcionamiento se levantan los diez microservicios y el modelo de IA. Todos los comandos se ejecutan en el VPS Backend, en la carpeta `/apps/aplicaciones/defensoria/back`, salvo la carga de datos semilla, que se hace en el VPS Base de Datos.

6.1. Construcción de las imágenes del Backend

6.1.1. Ubicación de los JAR en artifact/

Los scripts buscan cada JAR en `/apps/aplicaciones/defensoria/back/artifact/` con el nombre exacto `<servicio>.jar`; si falta alguno, se detienen con el mensaje `Error: No se encontro el archivo artifact/<servicio>.jar`. Antes de construir se confirma que estén los once:

```
# cd /apps/aplicaciones/defensoria/back
# ls -lh artifact/
```

Deben aparecer `admin-service.jar`, `antecedentes-service.jar`, `auth-service.jar`, `catalogo-service.jar`, `chatbot-service.jar`, `denunciado-service.jar`, `notificaciones-service.jar`, `primer-contacto-service.jar`, `quejas-service.jar`, `revision-service.jar` y `subdefensoria-service.jar`.

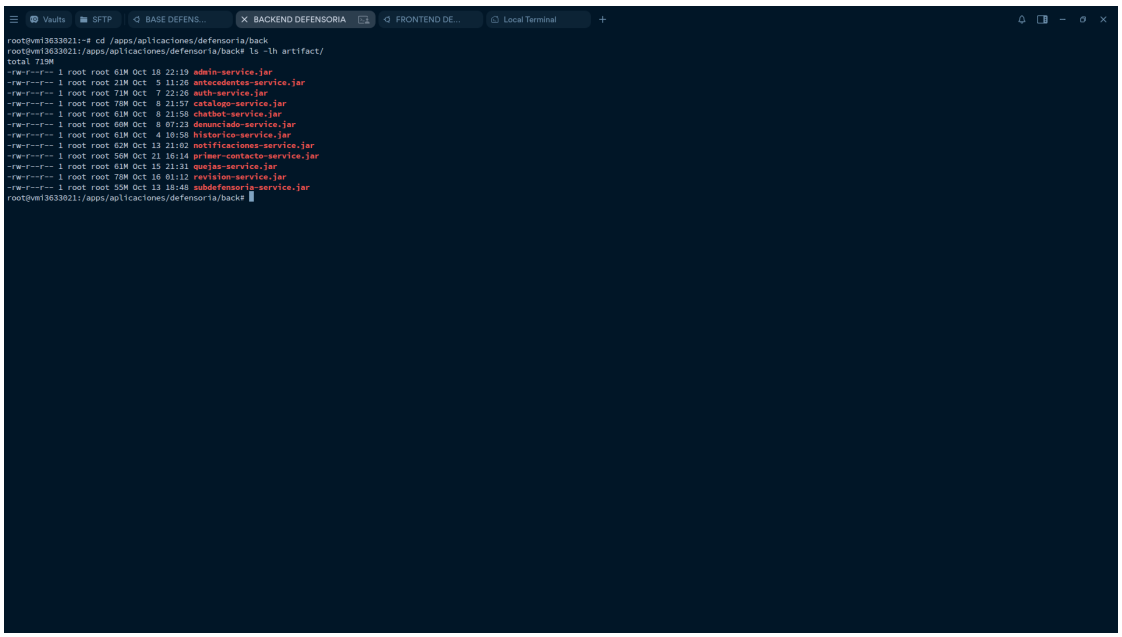


Figura 6.1: Los 11 JAR en la carpeta artifact del VPS Backend

6.1.2. Construcción de las imágenes con podman-compose.sh

`podman-compose.sh` administra los diez microservicios. Para cada servicio realiza tres pasos:

1. **Elimina** el contenedor anterior, si existe (`podman stop` y `podman rm`).
2. **Construye** la imagen `defensoria-<servicio>` con el Dockerfile común, o con `admin-service/Dockerfile` en el caso de `admin-service` (sección 4.3):

```
podman build -q -f Dockerfile --build-arg JAR_FILE=artifact/<servicio>.jar
    \
              --build-arg SERVICE_PORT=8080 -t defensoria-<servicio> .
```

3. **Crea** el contenedor con `podman run` y sus parámetros (sección 6.3).

Tabla 6.1: Comandos de podman-compose.sh

Comando	Qué hace
`up`	Construye y levanta los diez microservicios, uno tras otro
`up-container <servicio>`	Construye y levanta solo un microservicio
`delete`	Detiene y elimina los diez contenedores (las imágenes se conservan)
`delete-container <servicio>`	Detiene y elimina un contenedor

El modelo de IA tiene su propio script, `podman-compose-modelo.sh` (sección 6.3). La primera construcción tarda varios minutos porque descarga la imagen base y la de `admin-service` instala además el cliente de PostgreSQL; las siguientes reutilizan las capas ya descargadas.

6.2. Configuración de producción (config-files)

6.2.1. Montaje de la configuración en /app/config

Cada microservicio lee su configuración desde un archivo externo, no desde el JAR. `podman-compose.sh` monta la carpeta del servicio y le indica a Spring Boot dónde buscarla y cómo se llama el archivo:

```
-v /apps/aplicaciones/defensoria/back/config-files/quejas-service/config:/app/
    config:Z
-e SPRING_CONFIG_ADDITIONAL_LOCATION=optional:file:/app/config/
-e SPRING_CONFIG_NAME=quejas-service
```

Con esto Spring Boot carga `/app/config/quejas-service.yml`. Como el archivo vive fuera de la imagen, un cambio de configuración **no requiere recompilar ni reconstruir**: basta con editar el YAML y reiniciar el contenedor (`podman restart quejas-service`).

```
# find /apps/aplicaciones/defensoria/back/config-files -type f -printf "%M %u %
    g %p\n"
```

Cada archivo debe tener permiso `-rw-r-----` (640) y dueño `root`, porque contiene contraseñas (sección 2.3).

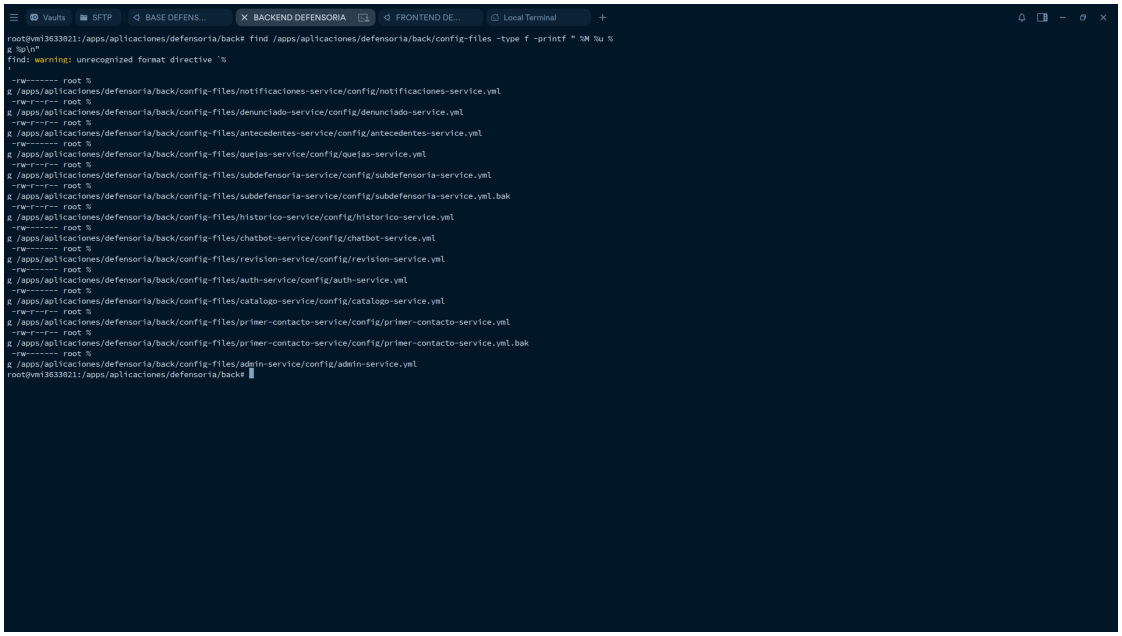


Figura 6.2: Archivos de configuración en el VPS Backend con sus permisos

6.2.2. Conexión a la base de datos (169.58.62.99:5432)

Todos los microservicios, excepto el modelo de IA, se conectan a la misma base:

```
spring:
  datasource:
    url: jdbc:postgresql://169.58.62.99:5432/defensoria_db
    username: postgres
    password: <CONTRASENA_BD>
  jpa:
    hibernate:
```

```
      ddl-auto: update
```

Al arrancar, cada servicio crea o actualiza sus propias tablas (`ddl-auto: update`). Si la contraseña no coincide con la de la base o el firewall del VPS Base de Datos no permite la IP del Backend, el servicio se detiene al iniciar con un error de conexión (`Connection refused` o `password authentication failed`) visible en su log.

Se puede probar la conexión desde el Backend antes de levantar los servicios:

```
# nc -zv -w 3 169.58.62.99 5432
```

6.2.3. Secreto JWT compartido

Cuando un usuario del personal inicia sesión, `auth-service` genera un token JWT firmado con `jwt.secret`. El navegador envía ese token en cada petición y cada microservicio verifica la firma con su propio `jwt.secret`. Por eso el valor debe ser **exactamente el mismo** en todos los config-files; si uno es distinto, ese servicio rechaza todas las peticiones con `401`. Se comprueba que haya un solo valor:

```
# grep -rh -A1 "^jwt:" config-files | grep secret | sort | uniq -c
```

6.2.4. Correo SMTP y URLs entre microservicios

Correo. `auth-service` (activación de cuentas y recuperación de contraseña) y `notificaciones-service` (avisos) envían correos por el SMTP de Gmail (`smtp.gmail.com`, puerto 587) con una contraseña de aplicación de la cuenta de la Defensoría, configurada en la sección `spring.mail`.

Llamadas entre microservicios. Los servicios se llaman entre sí por la IP del Backend y el puerto publicado de cada uno. Estas direcciones están en los config-files:

Tabla 6.2: Dependencias entre microservicios

Servicio	Llama a
`auth-service`	`quejas-service` (8084), `notificaciones-service` (8085)
`quejas-service`	`notificaciones-service` (8085)
`admin-service`	`catalogo-service` (8086)
`revision-service`	`primer-contacto-service` (8082), `notificaciones-service` (8085), `catalogo-service` (8086)
`primer-contacto-service`	`notificaciones-service` (8085), `subdefensoria-service` (8091), `antecedentes-service` (8093)
`subdefensoria-service`	`primer-contacto-service` (8082), `notificaciones-service` (8085)

Este tráfico sale de un contenedor y entra por la red de Podman, no por `eth0`, así que el firewall del Backend no lo bloquea (sección 3.1).

6.3. Creación y ejecución de los contenedores del Backend

6.3.1. Límites de memoria y heap de la JVM

Cada contenedor tiene un límite de memoria (`-m`) y la máquina virtual de Java un tamaño máximo de *heap* (`-Xmx`) menor que ese límite, porque la JVM usa memoria adicional fuera del heap. Si el contenedor supera su límite, el kernel lo termina (código de salida 137).

```
-m 512m
-e JAVA_TOOL_OPTIONS="-Xmx320m -Xms64m -XX:+ExitOnOutOfMemoryError"
```

- `-Xmx320m`: heap máximo de 320 MB (aproximadamente el 60 % del límite del contenedor).
- `-Xms64m`: heap inicial pequeño; crece solo si se necesita.
- `-XX:+ExitOnOutOfMemoryError`: si la JVM se queda sin memoria, el proceso termina en lugar de quedar colgado, y `--restart always` lo vuelve a levantar.

Tabla 6.3: Recursos asignados a cada contenedor del Backend

Contenedor	Puerto	Límite	Heap	Motivo
quejas-service	8084	1 GB	640 MB	Recibe evidencias de hasta 100 MB
denunciado-service	8094	1 GB	640 MB	Recibe credencial y evidencias
auth, notificaciones, catálogo	8083, 8085, 8086	512 MB	320 MB	Carga normal
admin, revisión, chatbot	8087, 8088, 8089	512 MB	320 MB	Carga normal
primer-contacto-service	8082	512 MB	320 MB	Carga normal
subdefensoria-service	8091	512 MB	320 MB	Carga normal
antecedentes-service (IA)	8093	250 MB	150 MB	Índice del corpus en memoria

6.3.2. Microservicios de la plataforma

`podman-compose.sh up` levanta en un solo paso los diez microservicios: los ocho de la plataforma y los de Primer Contacto y Subdefensoría.

```
# cd /apps/aplicaciones/defensoria/back
# bash podman-compose.sh up
```

Por cada servicio se muestra `Construyendo/Actualizando imagen defensoria-<servicio>...`, `Imagen ... actualizada exitosamente` y `Contenedor <servicio>iniciado.`

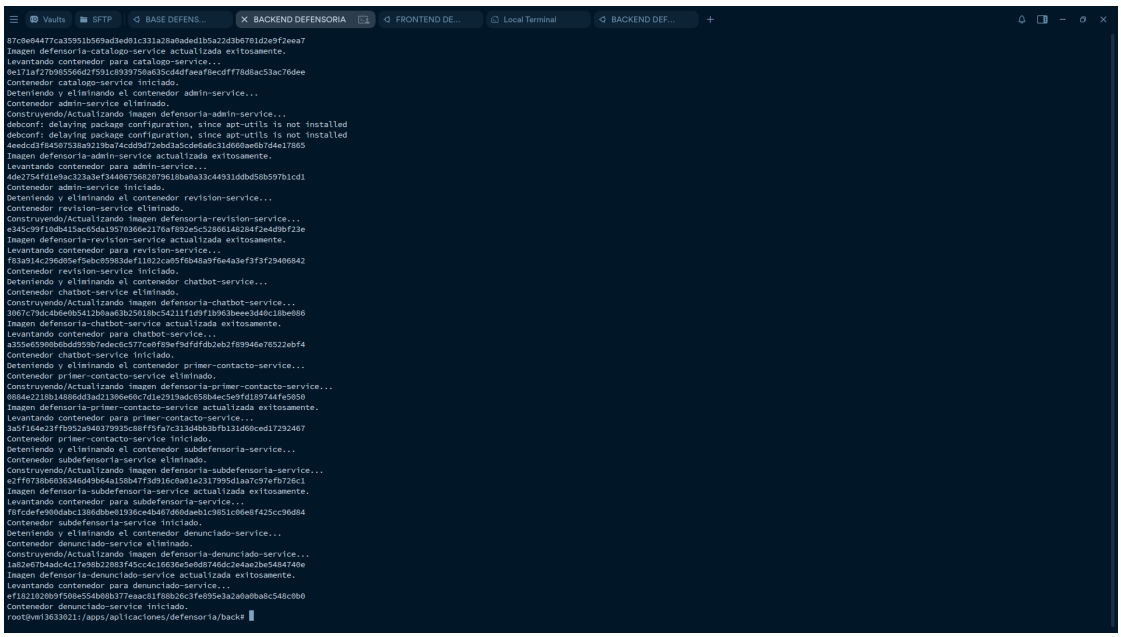


Figura 6.3: Construcción y arranque de los microservicios con podman-compose.sh up

Un servicio tarda entre 20 y 60 segundos en terminar de arrancar (conectarse a la base y crear sus tablas). Se confirma en su log con el mensaje `Started ... in X seconds`:

```
# podman logs --tail 20 quejas-service | grep -i started
```

6.3.3. Primer Contacto y Subdefensoría

`primer-contacto-service` (8082) y `subdefensoria-service` (8091) se levantan con el mismo up. Si se necesita levantarlos o reconstruirlos por separado:

```
# bash podman-compose.sh up-container primer-contacto-service
# bash podman-compose.sh up-container subdefensoria-service
```

Ambos usan la base `defensoria_db`. En `primer-contacto-service.yml`, la URL de la base histórica (`historico.datasource.url`) se deja vacía, porque esa base ya no forma parte de la plataforma; así el servicio busca antecedentes solo en el sistema y en el modelo de IA.

6.3.4. Modelo de IA (antecedentes-service y su corpus)

El modelo se administra con `podman-compose-modelo.sh`, que construye la imagen con el mismo Dockerfile común y crea el contenedor con dos volúmenes: su configuración y la carpeta del corpus.

```
-p 8093:8080 -m 250m
-e JAVA_TOOL_OPTIONS="-Xmx150m -Xms64m -XX:+UseSerialGC"
-v /apps/aplicaciones/defensoria/back/config-files/antecedentes-service/config:/
   app/config:Z
-v /apps/aplicaciones/defensoria/back/modelo/Dataset:/app/data:Z
```

- `/app/data` contiene `quejas_sinteticas.json` (corpus histórico) y `quejas_prueba_30.json`. Al aprender quejas nuevas, el modelo escribe

quejas_aprendidas.json en la misma carpeta, por lo que el aprendizaje se conserva aunque se recree el contenedor.

- -XX:+UseSerialGC usa el recolector de basura más ligero de Java, adecuado para un contenedor pequeño.

El script no crea el contenedor con --restart always; se agrega antes de levantarlo para que también vuelva solo tras un reinicio de la VPS:

```
# sed -i 's/--name "\$SERVICIO"/--restart always --name "$SERVICIO"/' podman-
    compose-modelo.sh
# grep -n "restart always" podman-compose-modelo.sh
# bash podman-compose-modelo.sh up
# bash podman-compose-modelo.sh status
```

status muestra el contenedor y la respuesta de /api/antecedentes/health. Para probar el modelo se envía una de las peticiones de ejemplo:

```
# curl -s -X POST http://127.0.0.1:8093/api/antecedentes/buscar \
    -H "Content-Type: application/json" -d @modelo/pruebas/buscar.json
```

La respuesta es una lista JSON con las quejas más parecidas del corpus y su grado de similitud.

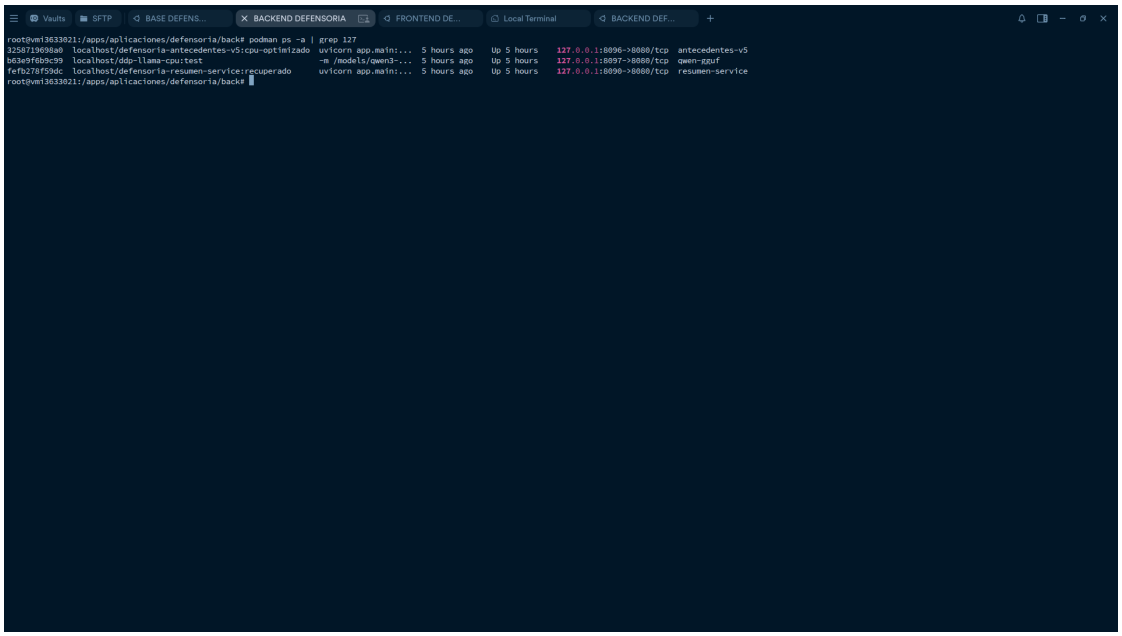


Figura 6.4: Modelo de IA levantado, health check y búsqueda de prueba

Verificación del Backend. Se listan los contenedores y se comprueba que cada puerto responda:

```
# podman ps -a --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
# for p in 8082 8083 8084 8085 8086 8087 8088 8089 8091 8093 8094; do
    printf "%s -> " $p; curl -s -o /dev/null -w "%{http_code}\n" --max-time 5
   http://127.0.0.1:$p/
  done
```

Deben aparecer los 11 contenedores en estado `Up`. En la prueba de puertos cualquier código HTTP (200, 401, 404) indica que el servicio responde; `000` indica que no está escuchando y se revisa su log con `podman logs <servicio>`.

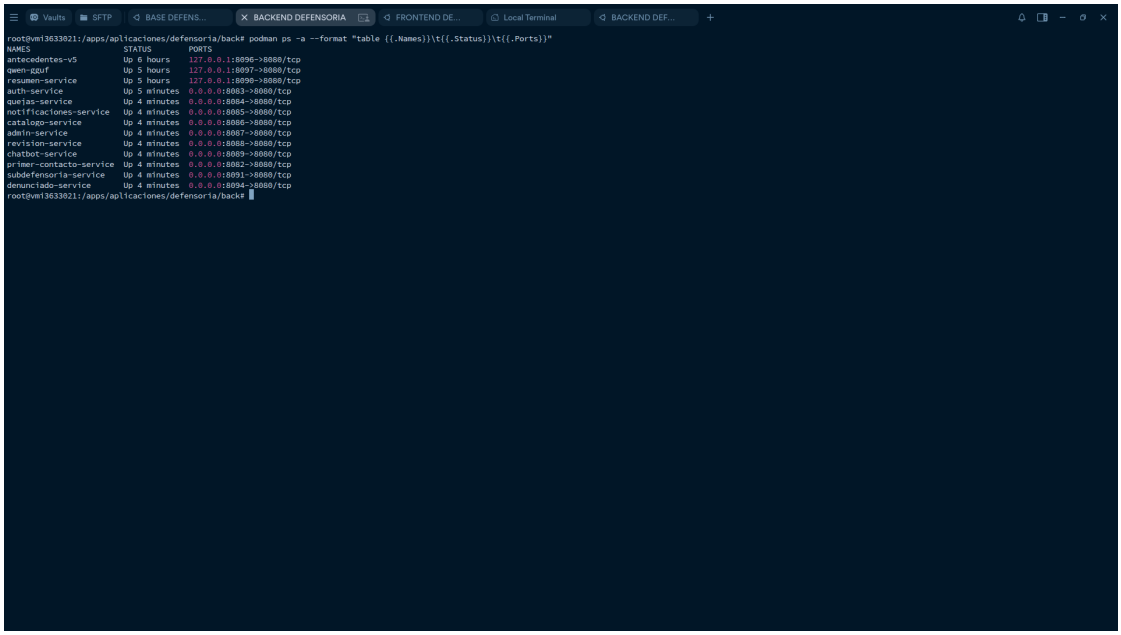


Figura 6.5: Los 11 contenedores del Backend en estado Up

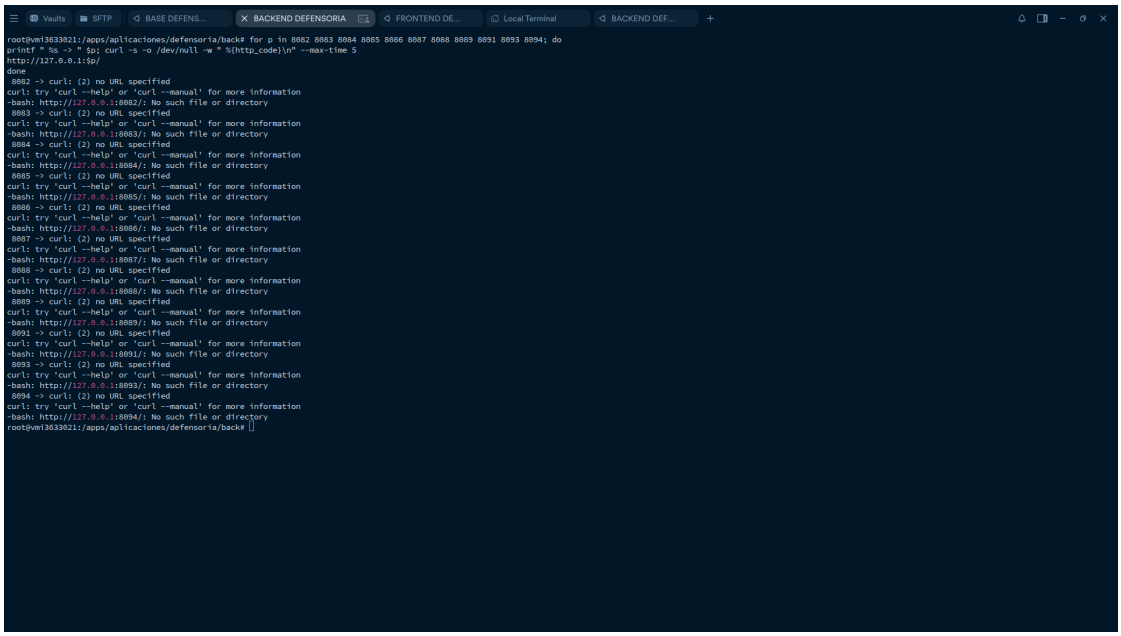


Figura 6.6: Respuesta de cada microservicio en su puerto

```
# podman images
```

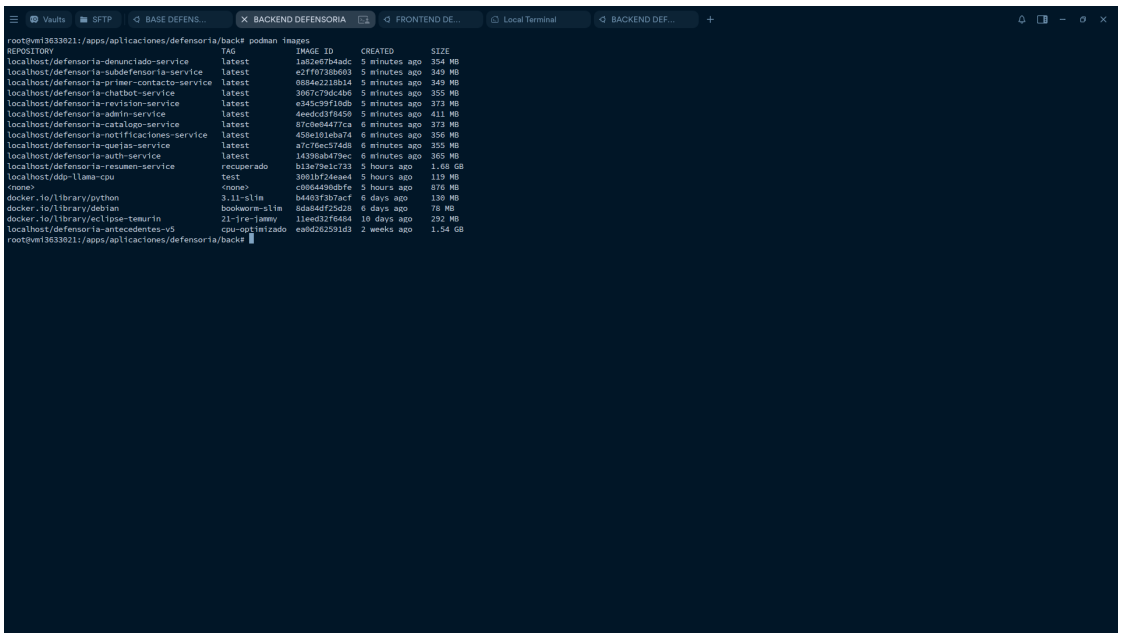


Figura 6.7: Imágenes defensoria-* construidas en el VPS Backend

6.4. Ciclo de vida de los contenedores

6.4.1. Reinicio automático (--restart always)

Los once contenedores del Backend quedan con la política `--restart always`: Podman los vuelve a iniciar si el proceso termina (por ejemplo, por falta de memoria) y el servicio `podman-restart` los levanta al encender la VPS. Se comprueba para todos a la vez:

```
# podman inspect --format '{{.Name}} {{.HostConfig.RestartPolicy.Name}}' $(
    podman ps -aq)
```

Cada línea debe terminar en `always`.

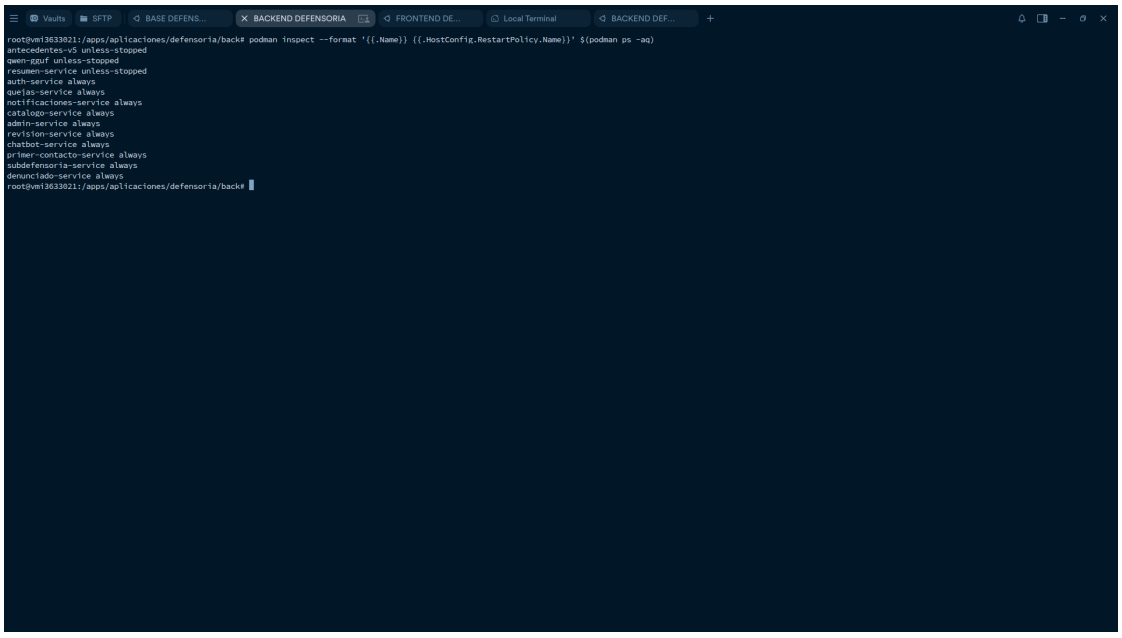


Figura 6.8: Política de reinicio de los contenedores del Backend

6.4.2. Redespliegue de un servicio (up-container y delete-container)

Para actualizar un microservicio se sustituye su JAR y se vuelve a construir solo ese servicio; los demás siguen funcionando:

1. Compilar el servicio en el equipo de desarrollo y copiar el JAR a `artifact/` con el mismo nombre (secciones 4.2 y 4.4).
2. En el VPS Backend:

```
# cd /apps/aplicaciones/defensoria/back
# bash podman-compose.sh up-container quejas-service
# podman logs -f --tail 50 quejas-service        # Ctrl+C para salir
```

`up-container` elimina el contenedor, reconstruye la imagen con el JAR nuevo y crea el contenedor otra vez. Durante ese tiempo (menos de un minuto) solo ese servicio deja de responder.

Tabla 6.4: Operaciones del ciclo de vida

Situación	Comando
Cambió el JAR	`bash podman-compose.sh up-container <servicio>`
Cambió solo el config-file	`podman restart <servicio>`
Detener un servicio	`bash podman-compose.sh delete-container <servicio>`
Detener todo el Backend	`bash podman-compose.sh delete` y `bash podman-compose-modelo.sh delete`
Actualizar el modelo de IA	`bash podman-compose-modelo.sh up`
Liberar espacio de imágenes viejas	`podman image prune`

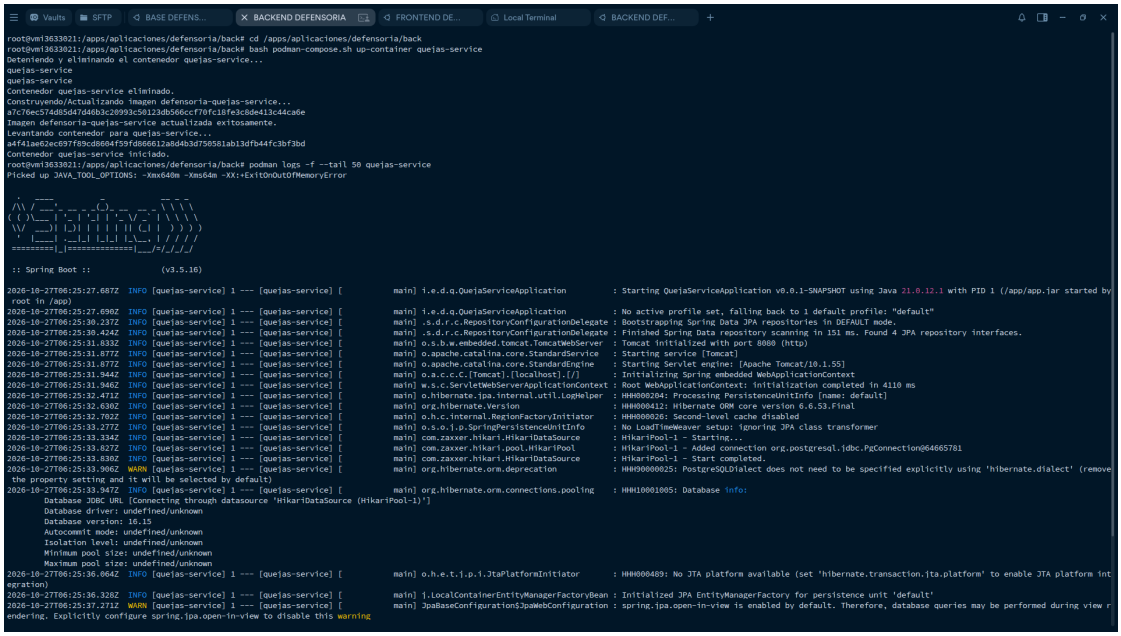


Figura 6.9: Redespliegue de un microservicio con up-container

6.5. Carga de datos iniciales

6.5.1. Datos semilla (dependencias, chatbot y personal)

Con los microservicios en funcionamiento, sus tablas ya existen y se pueden cargar los datos iniciales. Estos comandos se ejecutan en el **VPS Base de Datos**. Primero se confirma que las tablas se crearon:

```
# cd /apps/aplicaciones/defensoria/database
# bash podman-compose-bd.sh psql-defensoria
defensoria_db=# \dt
defensoria_db=# \q
```

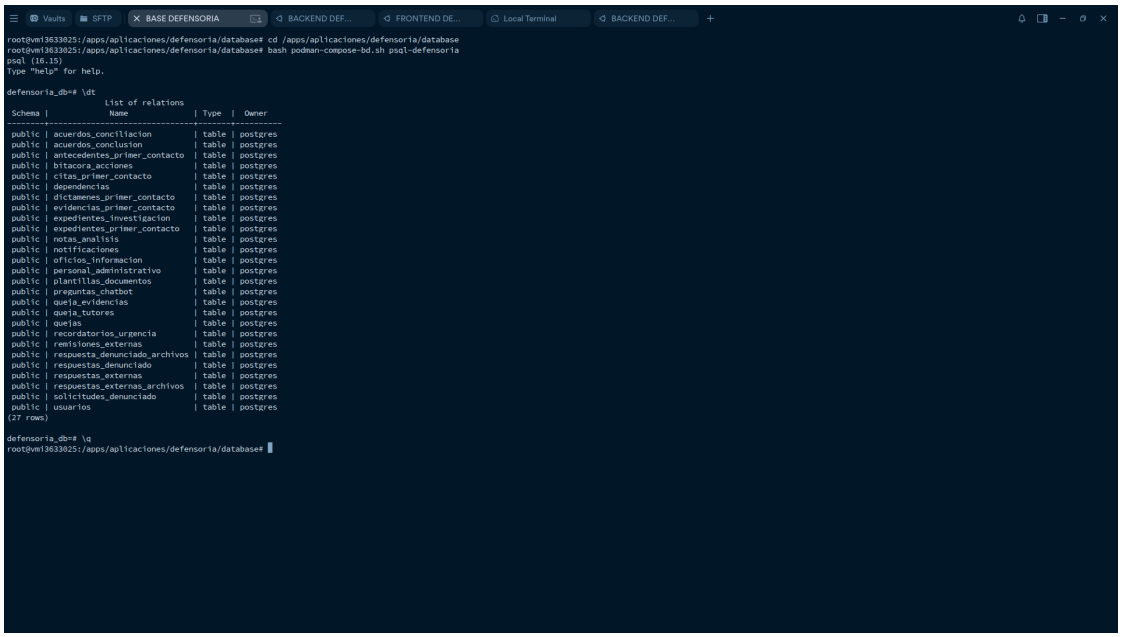


Figura 6.10: Tablas creadas por los microservicios en defensoria_db

Después se cargan los datos semilla:

```
# bash podman-compose-bd.sh seed
```

Tabla 6.5: Datos semilla

Archivo	Tabla	Contenido
`01-dependencias_seed.sql`	`dependencias`	Catálogo de unidades académicas y dependencias del IPN
`02-chatbot_seed.sql`	`preguntas_chatbot`	Preguntas y respuestas del asistente del portal
`03-personal_test.sql`	`personal_administrativo`	Una cuenta por cada rol del personal

El script ejecuta cada archivo en una sola transacción (si algo falla no se carga nada), no vuelve a cargar `dependencias` ni `preguntas_chatbot` si ya tienen datos, y avisa qué microservicio falta levantar si una tabla todavía no existe.

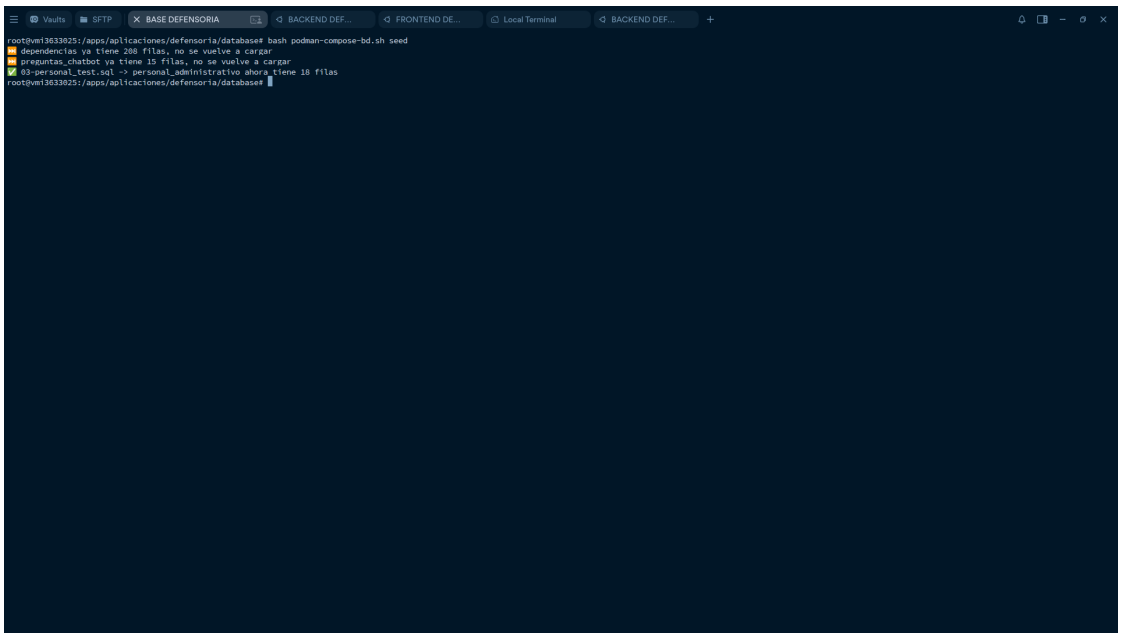


Figura 6.11: Resultado de la carga de datos semilla

Las cuentas de recepcionistas y de abogados de la Subdefensoría se cargan aparte:

```
# cd /apps/aplicaciones/defensoria/database-scripts/seeds
# podman exec -i defensoria-db psql -U postgres -d defensoria_db < 04-
    recepcionistas.sql
# podman exec -i defensoria-db psql -U postgres -d defensoria_db < 05-abogados.
    sql
```

Se comprueba el número de registros cargados:

```
# podman exec defensoria-db psql -U postgres -d defensoria_db -c \
  "SELECT 'dependencias', count(*) FROM dependencias
   UNION ALL SELECT 'preguntas_chatbot', count(*) FROM preguntas_chatbot
   UNION ALL SELECT 'personal_administrativo', count(*) FROM
    personal_administrativo;"
```

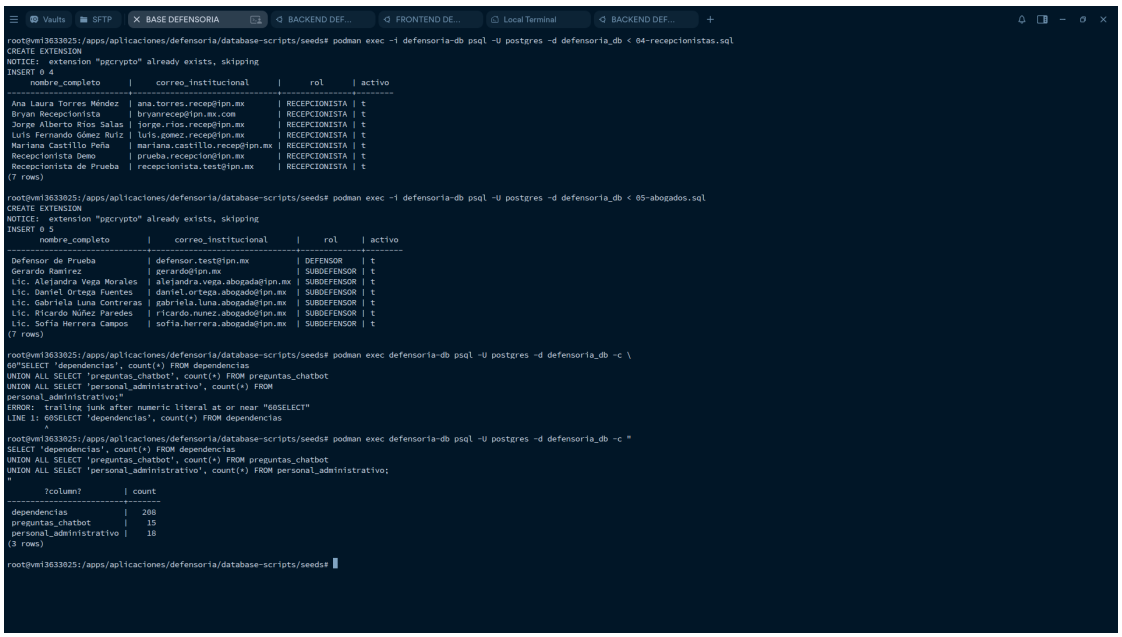


Figura 6.12: Número de registros cargados en cada tabla

Nota: Las cuentas de los scripts semilla son de prueba y tienen contraseñas conocidas. Antes de entregar la plataforma se desactivan:
`UPDATE personal_administrativo SET activo = false WHERE correo_institucional LIKE '%.test@ipn.mx';`

7. Despliegue del Nodo de Frontend, Nginx y Certificados (VPS 169.58.62.111)

El VPS Frontend es la puerta de entrada de la plataforma. En él corren los cinco contenedores de frontends y `router-nginx`, que recibe todo el tráfico de Internet con HTTPS y lo reparte. El orden de este capítulo es: frontends, certificado HTTPS y, al final, `router-nginx`, porque su configuración necesita que el certificado ya exista. Todos los comandos se ejecutan en el VPS Frontend.

7.1. Nginx en contenedor (router-nginx)

7.1.1. Contenedor router-nginx con --network host

`router-nginx` es un nginx oficial (`nginx:alpine`) que funciona como *proxy inverso*: recibe las peticiones de los usuarios, termina la conexión HTTPS y reenvía cada petición al frontend o al microservicio que corresponde según la ruta (sección 7.4). No tiene imagen propia: su comportamiento lo define por completo el archivo `defensoria.conf`, que se monta desde la VPS.

A diferencia de los demás contenedores, se crea con `--network host`: no tiene red propia, sino que usa directamente la red de la VPS. Esto permite que:

- Escuche en los puertos 80 y 443 de la VPS sin necesidad de publicarlos con `-p`.
- Alcance los cinco frontends por `127.0.0.1:8090`–`8095`, que son los puertos que estos publican en la VPS.

- Registre la IP real de cada visitante en sus logs, en lugar de una IP interna de Podman.

El contenedor se crea en la sección 7.3, una vez obtenido el certificado.

7.1.2. Estructura de /apps/aplicaciones/defensoria/router (config y certbot-webroot)

```
/apps/aplicaciones/defensoria/router/
|-- config/
|   `-- defensoria.conf      -> /etc/nginx/conf.d/default.conf (solo lectura)
`-- certbot-webroot/         -> /var/www/certbot (solo lectura)
/etc/letsencrypt/            -> /etc/letsencrypt (solo lectura, certificados)
```

- `config/defensoria.conf`: configuración de nginx, copiada en la sección 4.4.
- `certbot-webroot/`: carpeta donde Certbot deja el archivo temporal del reto de Let's Encrypt y desde donde nginx lo sirve.
- `/etc/letsencrypt`: carpeta del sistema donde Certbot guarda los certificados; se monta en el contenedor para que nginx los lea.

Los tres volúmenes se montan en solo lectura (`:ro`): nginx solo necesita leerlos.

```
# find /apps/aplicaciones/defensoria/router -printf "%M %u %g %p\n"
```

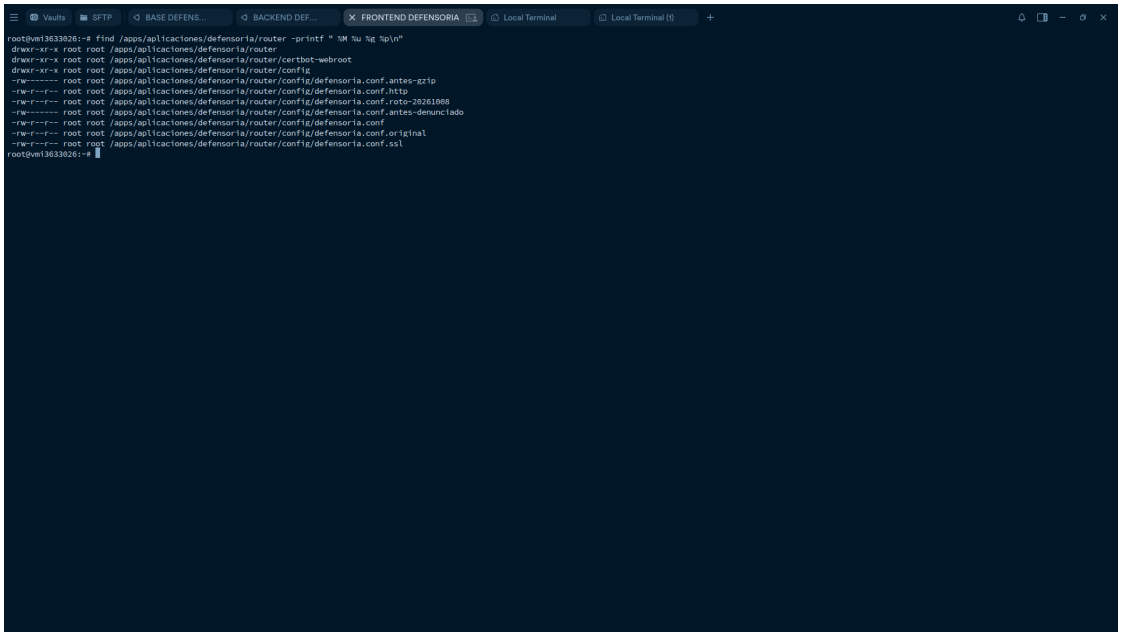


Figura 7.1: Carpeta router con defensoria.conf y certbot-webroot

7.2. Despliegue de los frontends

7.2.1. Ubicación del build web en dist/browser y asignación de permisos (755/644)

Cada frontend tiene su carpeta en `/apps/aplicaciones/defensoria` con su `Dockerfile`, su configuración de nginx, su script y, en `dist/browser`, los archivos compilados (sección 4.4):

Tabla 7.1: Carpetas de los frontends en el VPS Frontend

Carpeta	Contenedor	Script
`front/`	defensoria-web	`podman-compose-front.sh`
`front-admin/`	admin-web	`podman-compose-front-admin.sh`
`front-revision/`	revision-web	`podman-compose-front-revision.sh`
`front-primer-contacto/`	primer-contacto-web	`podman-compose-front-primer-contacto.sh`
`front-subdefensoria/`	subdefensoria-web	`podman-compose-front-subdefensoria.sh`

Los archivos web son públicos (cualquiera los descarga desde el navegador), por eso llevan permiso 644 y sus carpetas 755; los scripts, 750:

```
# F=/apps/aplicaciones/defensoria
# find $F -type d -exec chmod 755 {} +
# find $F -type f -exec chmod 644 {} +
# chmod 750 $F/*/podman-compose-front*.sh
# ls $F/*/dist/browser/index.html
```

El último comando debe listar los cinco `index.html`; si falta alguno, el frontend no se copió completo.

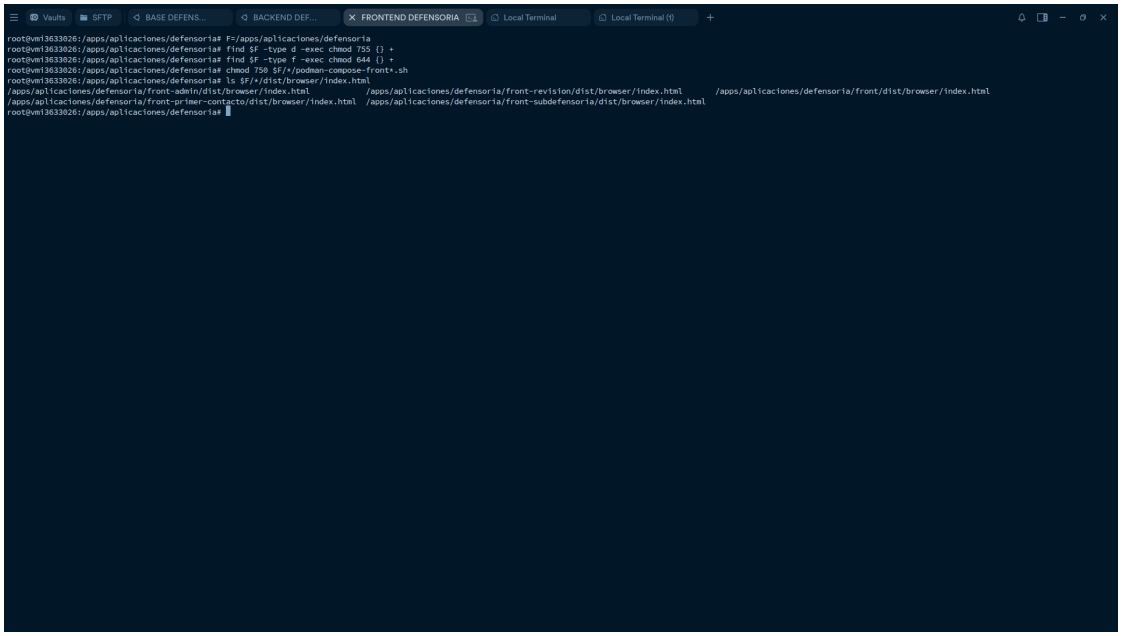


Figura 7.2: Archivos compilados de los cinco frontends en dist/browser

7.2.2. Contenedores de los 5 frontends (nginx:alpine)

Cada script construye la imagen con su Dockerfile (`podman build`) y crea el contenedor publicando su puerto. Los scripts de `defensoria-web`, `admin-web`, `revision-web` y `primer-contacto-web` no incluyen `--restart always`; se agrega antes de ejecutarlos para que los contenedores vuelvan solos tras reiniciar la VPS:

```
# sed -i 's/podman run -d --name/podman run -d --restart always --name/' \
      $F/*/podman-compose-front*.sh
# grep -h "podman run" $F/*/podman-compose-front*.sh
```

Las cinco líneas deben contener `--restart always`. Después se levantan:

```
# cd $F/front                    && bash podman-compose-front.sh up
# cd $F/front-admin              && bash podman-compose-front-admin.sh up
# cd $F/front-revision           && bash podman-compose-front-revision.sh up
# cd $F/front-primer-contacto    && bash podman-compose-front-primer-contacto.sh up
# cd $F/front-subdefensoria      && bash podman-compose-front-subdefensoria.sh up
```

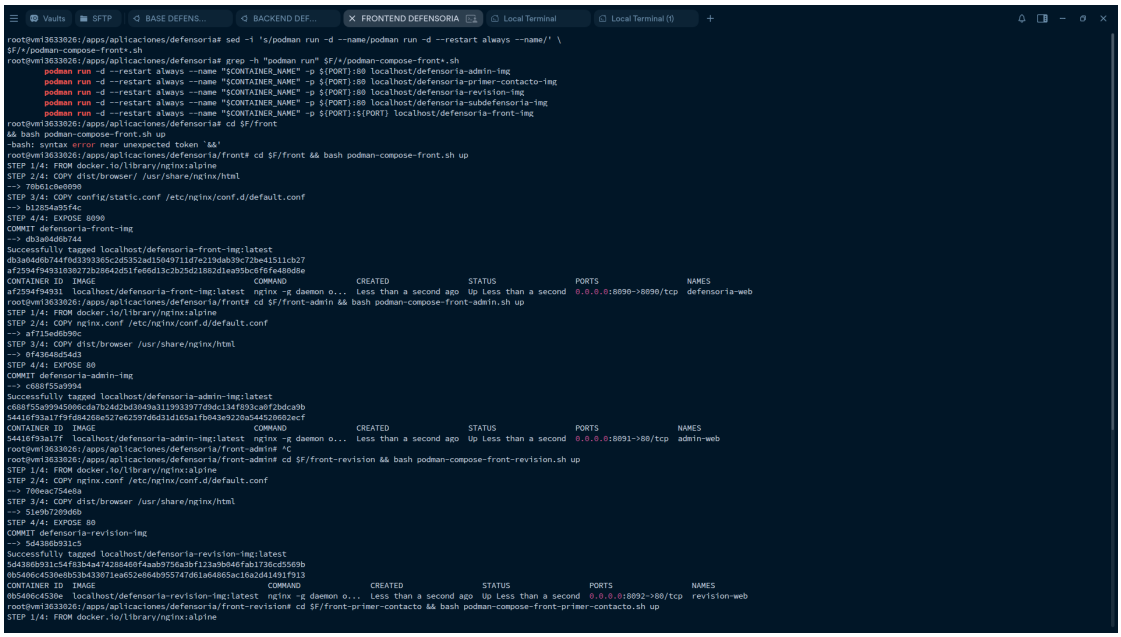


Figura 7.3: Construcción y arranque de los cinco frontends

Tabla 7.2: Contenedores de los frontends

Contenedor	**Imagen**	**Puerto VPS → contenedor**	**Ruta pública**
defensoria-web	defensoria-front-img	8090 → 8090	`/`
admin-web	defensoria-admin-img	8091 → 80	`/admin/`
revision-web	defensoria-revision-img	8092 → 80	`/revision/`
primer-contacto-web	defensoria-primer-contacto-img	8094 → 80	`/primer-contacto/`
subdefensoria-web	defensoria-subdefensoria-img	8095 → 80	`/subdefensoria/`

Verificación desde la propia VPS:

```
# podman ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
# for p in 8090 8091 8092 8094 8095; do
    printf "%s -> " $p; curl -s -o /dev/null -w "%{http_code}\n" http
    ://127.0.0.1:$p/
  done
```

Los cinco deben responder 200. Desde Internet esos puertos no responden, porque la regla del capítulo 3 los cierra; solo se accede a ellos a través de `router-nginx`.

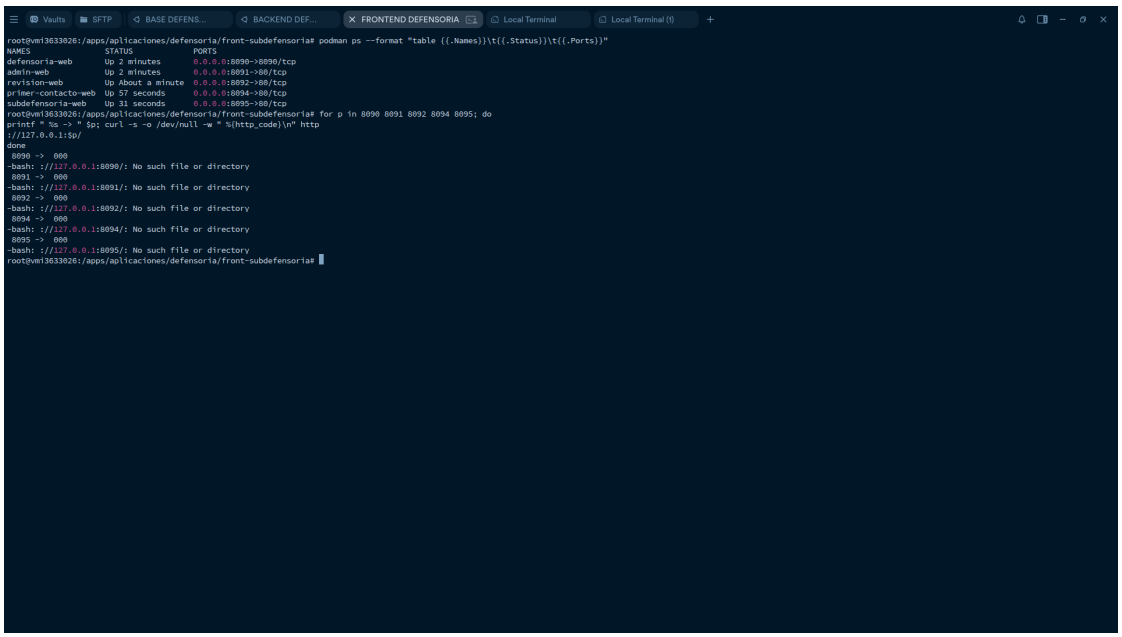


Figura 7.4: Contenedores de los frontends en estado Up y respondiendo 200

7.3. Emisión de Certificados SSL/TLS con Certbot

7.3.1. Instalación de Certbot (método webroot)

El sitio se publica con HTTPS usando un certificado gratuito de Let's Encrypt, que se obtiene y renueva con Certbot (instalado en la sección 2.2):

```
# certbot --version
```

Certbot puede obtener el certificado de varias formas. Se usa el método **webroot**: Certbot no toca nginx, solo escribe un archivo de comprobación en `/apps/aplicaciones/defensoria/router/certbot-webroot`, y `router-nginx` lo sirve en `http://defensoria-escom.ddns.net/.well-known/acme-challenge/`. Así la renovación funciona sin detener el sitio. El plugin `--nginx` de Certbot no se usa porque nginx corre dentro de un contenedor, donde Certbot no puede editar su configuración.

7.3.2. Dominio en No-IP (defensoria-escom.ddns.net)

El dominio `defensoria-escom.ddns.net` es un host gratuito de No-IP. En el panel de No-IP (**Dynamic DNS** → **No-IP Hostnames**) el host debe tener un registro tipo **A** con la IP del VPS Frontend, `169.58.62.111`. Los hosts gratuitos se deben confirmar cada 30 días desde el correo que envía No-IP; si no se confirman, el dominio deja de resolver.

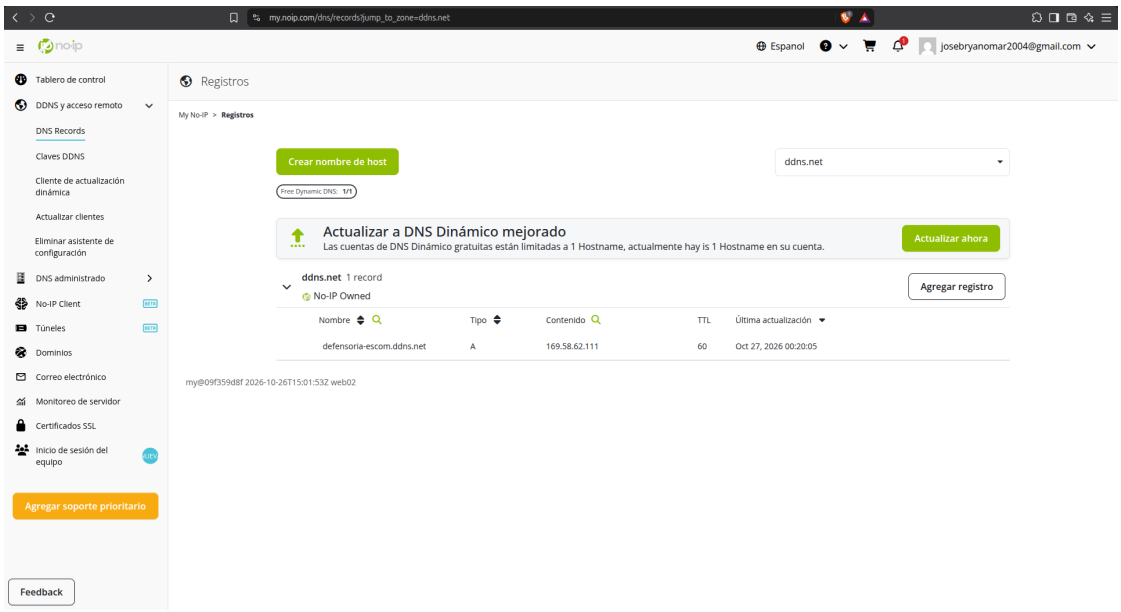


Figura 7.5: Host defensoria-escom.ddns.net apuntando a 169.58.62.111 en el panel de No-IP

Se comprueba que el dominio resuelva a la IP correcta:

```
# getent hosts defensoria-escom.ddns.net
169.58.62.111   defensoria-escom.ddns.net
```

7.3.3. Reto HTTP-01 y emisión inicial del certificado Let's Encrypt

Para emitir el certificado, Let's Encrypt comprueba que quien lo solicita controla el dominio (reto *HTTP-01*): Certbot crea un archivo con un código y Let's Encrypt lo descarga por el puerto 80. `defensoria.conf` hace referencia a los archivos del certificado, por lo que `router-nginx` no puede arrancar mientras no existan. Para la primera emisión se levanta un nginx temporal que solo atiende el reto: Primero se crea la configuración mínima de ese nginx temporal, que solo atiende la ruta del reto en el puerto 80:

```
echo 'server {'                                          >  /tmp/acme.conf
echo '  listen 80;'                                        >> /tmp/acme.conf
echo '  server_name defensoria-escom.ddns.net;'            >> /tmp/acme.conf
echo '  location /.well-known/acme-challenge/ {'           >> /tmp/acme.conf
echo '    root /var/www/certbot;'                          >> /tmp/acme.conf
echo '  }'                                                 >> /tmp/acme.conf
echo '  location / { return 404; }'                        >> /tmp/acme.conf
echo '}'                                                   >> /tmp/acme.conf
cat /tmp/acme.conf
```

Después se levanta el contenedor temporal:

```
# 1. Definir la variable del directorio base
R=/apps/aplicaciones/defensoria/router

# 2. Asegurar que exista la carpeta webroot para el reto ACME
mkdir -p "$R/certbot-webroot"

# 3. Levantar el contenedor Nginx temporal en una sola instruccion
```

```
podman run -d --name router-temporal --network host \
  -v /tmp/acme.conf:/etc/nginx/conf.d/default.conf:ro \
  -v "$R/certbot-webroot":/var/www/certbot:ro \
  docker.io/library/nginx:alpine
```

Con el nginx temporal en marcha se solicita el certificado y, al terminar, se elimina el contenedor temporal:

```
# 1. Asegurar la variable de ruta
R=/apps/aplicaciones/defensoria/router

# 2. Solicitar el certificado con Certbot
certbot certonly --webroot -w "$R/certbot-webroot" \
  -d defensoria-escom.ddns.net \
  --agree-tos \
  -m tu_correo@dominio.com \
  --no-eff-email

# 3. Eliminar el contenedor Nginx temporal
podman rm -f router-temporal

# 4. Verificar que los archivos del certificado se crearon correctamente
ls -l /etc/letsencrypt/live/defensoria-escom.ddns.net/
```

En `-m` se escribe el correo de contacto al que Let's Encrypt enviará los avisos de vencimiento. Certbot debe mostrar `Successfully received certificate`. En la carpeta quedan `fullchain.pem` (certificado con la cadena de confianza) y `privkey.pem` (llave privada). El certificado es válido por 90 días.



Figura 7.6: Emisión del certificado de Let's Encrypt

Creación de router-nginx. Con el certificado disponible se crea el contenedor definitivo:

```
# 1. Asegurar que la variable exista en la sesion actual
R=/apps/aplicaciones/defensoria/router
```

```
# 2. Levantar el contenedor definitivo de Nginx
podman run -d --name router-nginx --restart always --network host \
  -v "$R/config/defensoria.conf":/etc/nginx/conf.d/default.conf:ro \
  -v "$R/certbot-webroot":/var/www/certbot:ro \
  -v /etc/letsencrypt:/etc/letsencrypt:ro \
  docker.io/library/nginx:alpine
```

Una vez que el comando devuelva el ID del contenedor, se ejecutan las comprobaciones:

```
# Probar la sintaxis de la configuracion cargada en Nginx
podman exec router-nginx nginx -t
```

Debe responder `syntax is ok` y `test is successful`.

```
# Verificar que el contenedor este arriba (Status Up)
podman ps --filter name=router-nginx
```

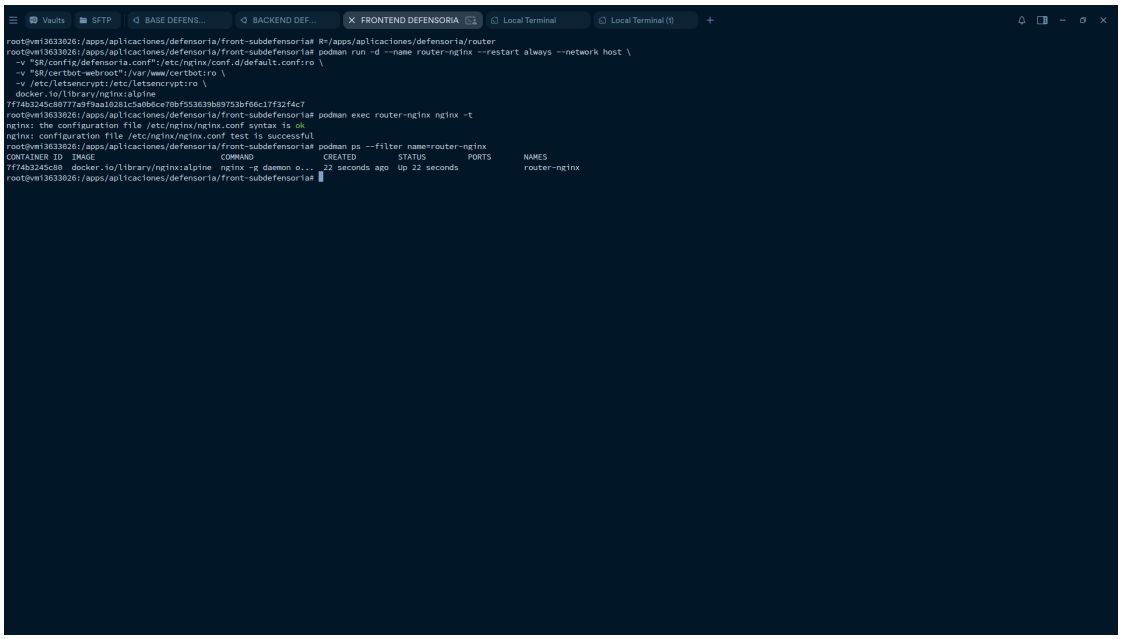


Figura 7.7: Contenedor router-nginx en ejecución y configuración validada

7.3.4. Renovación automática y recarga de nginx (certbot renew --dry-run)

Al instalarse, Certbot crea el temporizador de systemd `certbot.timer`, que dos veces al día revisa los certificados y renueva los que venzan en menos de 30 días usando el mismo método webroot. Después de renovar hay que recargar nginx para que use el certificado nuevo; para eso se agrega un *hook* que Certbot ejecuta tras cada renovación exitosa:

```
# cat > /etc/letsencrypt/renewal-hooks/deploy/recargar-nginx.sh <<'FIN'
#!/bin/bash
podman exec router-nginx nginx -s reload
FIN
# chmod 755 /etc/letsencrypt/renewal-hooks/deploy/recargar-nginx.sh
```

Se comprueba el temporizador y se simula una renovación (no cambia el certificado real):

```
# systemctl list-timers | grep certbot
# certbot renew --dry-run
# certbot certificates
```

`--dry-run` debe terminar con `Congratulations, all simulated renewals succeeded`, y `certbot certificates` muestra la fecha de vencimiento del certificado.

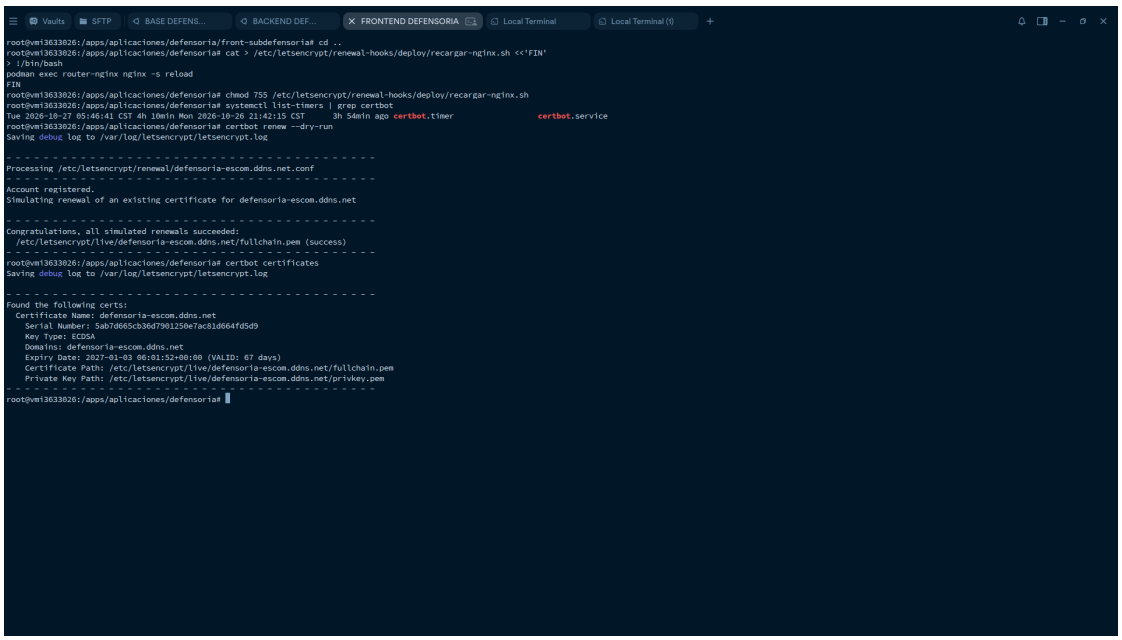


Figura 7.8: Prueba de renovación del certificado y fecha de vencimiento

7.4. Configuración de producción de Nginx (defensoria.conf)

`defensoria.conf` define todo el comportamiento de `router-nginx`. Se compone de dos bloques `server`: uno para el puerto 80 y otro para el 443. Esta sección explica cada parte.

7.4.1. Bloque de redirección forzada de HTTP (80) a HTTPS (443)

```
server {
    listen 80;
    server_name defensoria-escom.ddns.net;
    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }
    location / {
        return 301 https://$host$request_uri;
    }
}
```

El puerto 80 solo hace dos cosas: entrega los archivos del reto de Let's Encrypt (necesarios para renovar el certificado) y responde a cualquier otra petición con una redirección permanente (301) a la misma dirección con `https://`. Así ningún usuario navega sin cifrado.

7.4.2. Bloque 443 y certificados TLS

```
server {
    listen 443 ssl;
    http2 on;
    server_name defensoria-escom.ddns.net;
    ssl_certificate     /etc/letsencrypt/live/defensoria-escom.ddns.net/
    fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/defensoria-escom.ddns.net/privkey.
    pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ...
}
```

- `listen 443 ssl` y `http2 on`: atiende HTTPS con el protocolo HTTP/2, que carga los archivos de las páginas en paralelo por una sola conexión.
- `ssl_certificate` / `ssl_certificate_key`: certificado y llave emitidos por Certbot (sección 7.3).
- `ssl_protocols TLSv1.2 TLSv1.3`: solo se aceptan versiones seguras de TLS.

Dentro del mismo bloque se agregan las cabeceras que nginx envía a los servicios para que conozcan al cliente real:

```
proxy_set_header Host              $host;
proxy_set_header X-Real-IP         $remote_addr;
proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
```

7.4.3. Rutas hacia los frontends

```
location /                { proxy_pass http://127.0.0.1:8090;  }   # defensoria
    -web
location /admin/          { proxy_pass http://127.0.0.1:8091/; }   # admin-web
location /revision/       { proxy_pass http://127.0.0.1:8092/; }   # revision-
    web
location /primer-contacto/ { proxy_pass http://127.0.0.1:8094/; }   # primer-
    contacto-web
location /subdefensoria/   { proxy_pass http://127.0.0.1:8095/; }   #
    subdefensoria-web
```

Cuando `proxy_pass` termina en `/`, nginx quita el prefijo de la ruta antes de reenviar: una petición a `/admin/login` llega al contenedor `admin-web` como `/login`. Por eso cada frontend se compila con su `baseHref` (sección 4.2): el navegador pide los archivos con el prefijo y nginx lo quita al entregarlos.

7.4.4. Reverse proxy: location /api/ hacia el VPS Backend

```
location /api/auth/            { proxy_pass http://156.67.26.73:8083; }
location /api/quejoso/         { proxy_pass http://156.67.26.73:8084; }
location /api/notificaciones/  { proxy_pass http://156.67.26.73:8085; }
location /api/catalogos/       { proxy_pass http://156.67.26.73:8086; }
location /api/admin/           { proxy_pass http://156.67.26.73:8087; }
location /api/revision/        { proxy_pass http://156.67.26.73:8088; }
location /api/chatbot/         { proxy_pass http://156.67.26.73:8089; }
location /api/denunciado/      { proxy_pass http://156.67.26.73:8094; }
location /api/primer-contacto/ { proxy_pass http://156.67.26.73:8082; }
location /api/subdefensoria/   { proxy_pass http://156.67.26.73:8091; }
```

Aquí `proxy_pass` no termina en `/`, así que la ruta llega completa al microservicio: `/api/quejoso/registro` llega a `quejas-service` como `/api/quejoso/registro`. Las peticiones salen de la IP del VPS Frontend, la única que el firewall del Backend acepta.

7.4.5. Rutas bloqueadas, tamaño máximo de carga y tiempos de espera

```
location = /api/notificaciones/enviar    { return 403; }
location = /api/notificaciones/registrar { return 403; }
location /api/primer-contacto/ingesta/   { return 403; }
location /api/subdefensoria/ingesta/     { return 403; }
location /api/subdefensoria/acuerdos-conclusion/interno/ { return 403; }

client_max_body_size 100M;
proxy_read_timeout 120s;
proxy_send_timeout 120s;
```

- Las rutas internas responden `403` desde Internet (sección 3.2). Las rutas exactas (`location =`) y más específicas tienen prioridad sobre las `/api/` generales.
- `client_max_body_size 100M`: una queja puede incluir credencial y varias evidencias; sin este valor, nginx rechaza cualquier petición mayor a 1 MB con el error `413`.
- `proxy_read_timeout` / `proxy_send_timeout`: espera hasta 120 segundos la respuesta del Backend, para cargas de archivos grandes o conexiones lentas.

7.4.6. Compresión gzip y caché de estáticos

```
gzip on;
gzip_comp_level 5;
gzip_min_length 1024;
gzip_types text/css application/javascript image/svg+xml;

map $uri $cache_estaticos {
    "~-[A-Z0-9]{8}\.(js|css)$"  "public, max-age=31536000, immutable";
    default                     "no-cache";
}
add_header Cache-Control $cache_estaticos;
```

- **gzip**: comprime los archivos CSS, JavaScript y SVG, que pesan unas cuatro veces menos al viajar. A propósito no se comprimen las respuestas JSON de la API, porque contienen tokens y datos de las quejas, y comprimir respuestas con secretos facilita el ataque BREACH.
- **Caché**: Angular genera archivos con un *hash* en el nombre (por ejemplo `main-AB12CD34.js`) que nunca cambian de contenido; el navegador los guarda un año. Todo lo demás (`index.html` y la API) se marca `no-cache`, para que cada despliegue nuevo se vea de inmediato sin limpiar la caché del navegador.

Aplicar cambios en defensoria.conf. Después de editar el archivo, se valida y se recarga nginx sin cortar las conexiones activas:

```
# podman exec router-nginx nginx -t
# podman exec router-nginx nginx -s reload
```

Si `nginx -t` marca un error, no se recarga: el contenedor sigue funcionando con la configuración anterior.

Verificación. Desde el equipo de administración:

```
$ curl -I http://defensoria-escom.ddns.net/        # 301 Moved Permanently ->
    https
$ curl -I https://defensoria-escom.ddns.net/       # HTTP/2 200
$ curl -s -o /dev/null -w "%{http_code}\n" -X POST \
    https://defensoria-escom.ddns.net/api/notificaciones/enviar     # 403
```

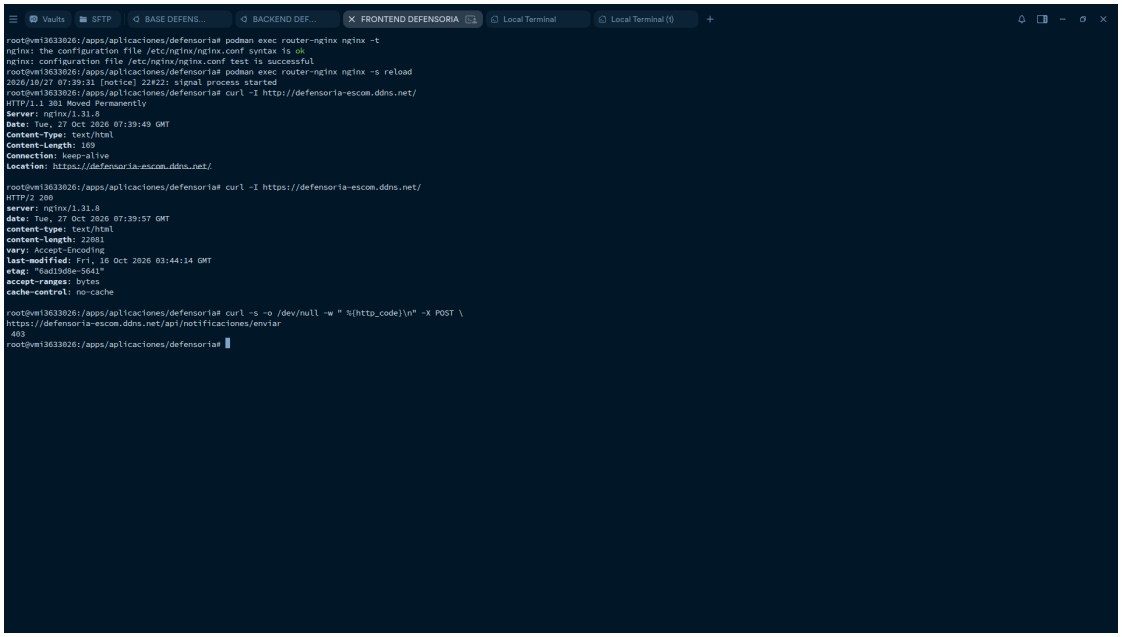


Figura 7.9: Redirección HTTP a HTTPS, respuesta 200 por HTTPS y ruta interna bloqueada